3. Audit Methodology and Criteria

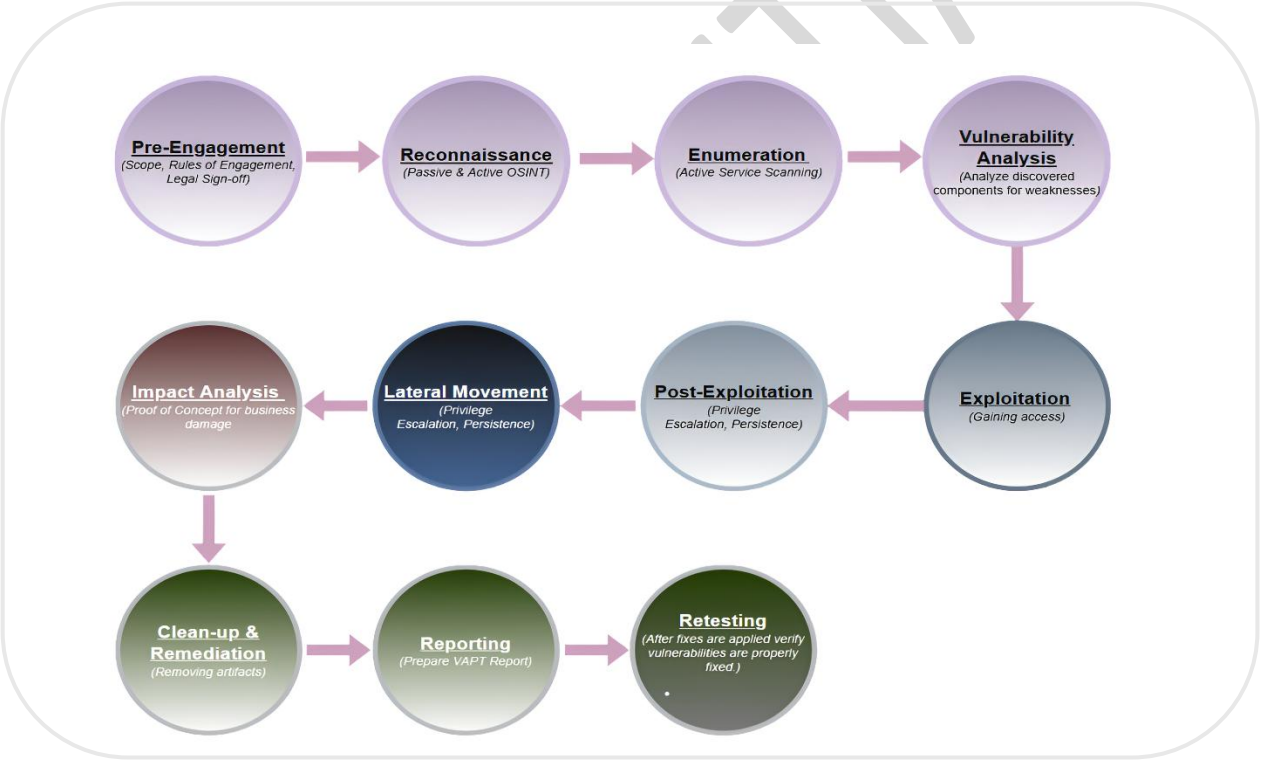
Web Application Penetration testing can never be an exact science where a complete list of all possible issues that should be tested can be defined. Indeed, penetration testing is only an appropriate technique for testing the security of web applications under certain circumstances. The goal is to collect all the possible testing techniques, explain them, and keep the guide updated.

AICERT Web Application Penetration Testing method is based on the Black Box Approach. The tester knows nothing or very little information about the application to be tested. The testing model consists of:

- Tester: Who performs the testing activities
- Tools and methodology: The core of this security testing project
- Application: The black box to test

When conducting a Web Penetration Testing assignment, AICERT adopts a strong technology and process-based approach supported by a well-documented methodology to identify potential security flaws in the application and underlying environment.

3.1 Approach



3.2 Severity Rating

AICERT follows the Common Vulnerability Scoring System (CVSS) scoring system to rate vulnerabilities. The CVSS assessment measures three areas of concern:

- Base Metrics for qualities intrinsic to a vulnerability
- Temporal Metrics for characteristics that evolve over the lifetime of vulnerability
- Environmental Metrics for vulnerabilities that depend on an implementation or environment

For more information on what the base, temporal, and environment metrics in the CVSS scoring system are, please visit NVD - CVSS v3 Calculator. Unless otherwise stated, the findings in the report are scored on the base-metric rating of the vulnerability. AICERT may consider Environmental and Temporal Metrics depending on information provided at project initiation and if this is a mandatory client reporting requirement. Based on the severity of the vulnerability, they are assigned below ratings.

CVSS Severity Rating	CVSS Score
Critical	9.0 - 10.0
High	7.0 - 8.9
Medium	4.0 - 6.9
Low	0.1 - 3.9
Info	0.0

3.3 Risk Rating

By categorizing vulnerabilities into these severity levels, organizations can effectively prioritize and address security issues based on the potential impact on their systems. Here's a guide to interpreting the risk rating.

Risk Rating	Explanation	Recommendations
Critical	This rating is assigned to vulnerabilities that pose an immediate and severe risk to the application and its users. Such issues typically affect publicly accessible or widely exposed environments and involve sensitive data sets (e.g., PII, PHI, financial or proprietary business information) rather than isolated records. Critical vulnerabilities may enable remote code execution, complete system compromise, significant privacy violations, or full application takeover. When identified in a public-facing environment, these findings require immediate escalation and remediation in accordance with the organization's Critical Vulnerability Remediation process.	These issues should be resolved with immediate effect. It is recommended to take the vulnerable system offline until all critical issues are resolved.
HIGH	This rating is assigned to vulnerabilities that can significantly compromise the confidentiality, integrity, or availability of application resources. These issues may allow privilege escalation, unauthorized access to protected resources, execution of arbitrary code by authenticated or unauthenticated users, or the ability to cause denial-of-service conditions. High-severity vulnerabilities present a substantial security risk and require prompt remediation.	Immediate action is recommended to ensure that the weakness or exposure is corrected and removed. In some instances, forensic services may be recommended to ensure that no exploitation has already occurred.
MEDIUM	This rating is assigned to vulnerabilities that could lead to a partial compromise of confidentiality, integrity, or availability under certain conditions. While exploitation may require additional prerequisites, specific configurations, or user interaction, successful exploitation could still result in meaningful security impact. These vulnerabilities are considered important but are generally less straightforward to exploit than high or critical issues.	Action should be taken once high risks have been addressed.
LOW	This rating is assigned to vulnerabilities that have a limited security impact. Exploitation typically requires unlikely conditions, advanced attacker capabilities, or provides minimal benefit to an attacker. While these issues do not pose an immediate threat, they may contribute to risk when combined with other vulnerabilities and should be addressed as part of ongoing security improvements	Whilst lower risk classifications do not pose an immediate threat to the information security, or require immediate attention, they should nevertheless be addressed through remedial work.
INFO	This rating is assigned to issues that do not directly impact the confidentiality, integrity, or availability of	Not applicable.

	the application but may disclose information useful to an attacker during reconnaissance. Examples include disclosure of system details such as server banners, operating system information, network paths, or application configuration details. Although these findings do not represent direct vulnerabilities, they can aid attackers in identifying and exploiting other weaknesses.	

3.4 List of Standards, Frameworks and Other References:

S.No	Standard/Framework Used	Severity Score System Used	Other References
1	OWASP Web Security Testing Guide (WSTG)	1. CVSS (Common Vulnerability Scoring System) 2. OWASP Risk Rating Methodology 3. DREAD (Damage, Reproducibility, Exploitability, Affected Users, Discoverability) 4. EPSS (Exploit Prediction Scoring System)	A comprehensive guide for testing web application security, providing detailed instructions for vulnerability identification. OWASP WSTG https://owasp.org/www-project-web-security-testing-guide/
2	OWASP ASVS (Application Security Verification Standard)	1. CVSS (Common Vulnerability Scoring System) 2. OWASP Risk Rating Methodology 3. DREAD (Damage, Reproducibility, Exploitability, Affected Users, Discoverability)	A framework that defines security requirements for web applications, aimed at verifying secure design and implementation. OWASP ASVS https://owasp.org/www-project-application-security-verification-standard/

		4. EPSS (Exploit Prediction Scoring System)	
3	OWASP SAMM (Software Assurance Maturity Model)	1. CVSS (Common Vulnerability Scoring System) 2. OWASP Risk Rating Methodology 3. DREAD (Damage, Reproducibility, Exploitability, Affected Users, Discoverability) 4. EPSS (Exploit Prediction Scoring System)	A maturity model to help organizations assess and improve their software security practices, focusing on secure software development lifecycle (SDLC). OWASP SAMM https://owasp.org/www-project-samm/
4	PTES (Penetration Testing Execution Standard)	1. CVSS (Common Vulnerability Scoring System) 2. OWASP Risk Rating Methodology 3. DREAD (Damage, Reproducibility, Exploitability, Affected Users, Discoverability) 4. EPSS (Exploit Prediction Scoring System)	A standard methodology for conducting penetration tests, providing guidelines for all phases of the test, including planning, testing, and reporting. PTES
5	OSSTMM (Open-Source Security Testing Methodology Manual)	1. CVSS (Common Vulnerability Scoring System) 2. OWASP Risk Rating Methodology	A comprehensive framework for security testing, including methodology for testing network, application, and human security. OSSTMM

		3. DREAD (Damage, Reproducibility, Exploitability, Affected Users, Discoverability)	
		4. EPSS (Exploit Prediction Scoring System)	
6	OWASP Top 10	1. CVSS (Common Vulnerability Scoring System)	A list of the top ten most critical web application security risks, aimed at guiding developers and organizations in improving security practices. OWASP Top 10 https://owasp.org/www-project-top-ten/
		2. OWASP Risk Rating Methodology	
		3. DREAD (Damage, Reproducibility, Exploitability, Affected Users, Discoverability)	
		4. EPSS (Exploit Prediction Scoring System)	
7	NIST Cybersecurity Framework (CSF)	1. CVSS (Common Vulnerability Scoring System)	A framework designed to guide organizations in managing and reducing cybersecurity risks, including web application security aspects. NIST CSF
		2. OWASP Risk Rating Methodology	
		3. DREAD (Damage, Reproducibility, Exploitability, Affected Users, Discoverability)	
		4. EPSS (Exploit Prediction Scoring System)	
8	CIS Controls (Center for Internet Security)	1. CVSS (Common Vulnerability Scoring System)	https://www.cisecurity.org/controls/cis-controls-list The CIS Controls/framework are designed to be actionable and easy to implement. They provide specific actions for each control that can be
		2. DREAD (Damage,	

		Reproducibility, Exploitability, Affected Users, Discoverability)	executed by organizations to address vulnerabilities and mitigate risks.

3.5 Tools/ Software used

AICERT's Auditors used following tools to conduct Vulnerability Assessment and Penetration Testing (VAPT) for automatic and manual testing.

S.NO	Name of Tool/Software used	Open Source/Licensed
•	Burpsuite	Community Edition
•	OWASP ZAP	Open Source
•	Kali Linux	Open Source
•	Nmap	Open Source
•	Gobuster	Open Source
•	SQLMAP	Open Source
•	XSStrike	Open Source
•	WhatWeb	Open Source
•	Httpx	Open Source
•	Commix	Open Source
•	SSTIMap	Open Source

4. Executive Summary

A security assessment was conducted on the target web application and its associated infrastructure to evaluate the effectiveness of existing security controls and to identify vulnerabilities that could be exploited by malicious actors. The assessment focused on identifying weaknesses that could impact the confidentiality, integrity, and availability of the application and the data it processes. Across the application scope, a list of 21 target assets was provided for assessment.

During the engagement, a total of 29 security vulnerabilities were identified across the target web application and its associated infrastructure. These findings were categorized based on their severity and potential impact:

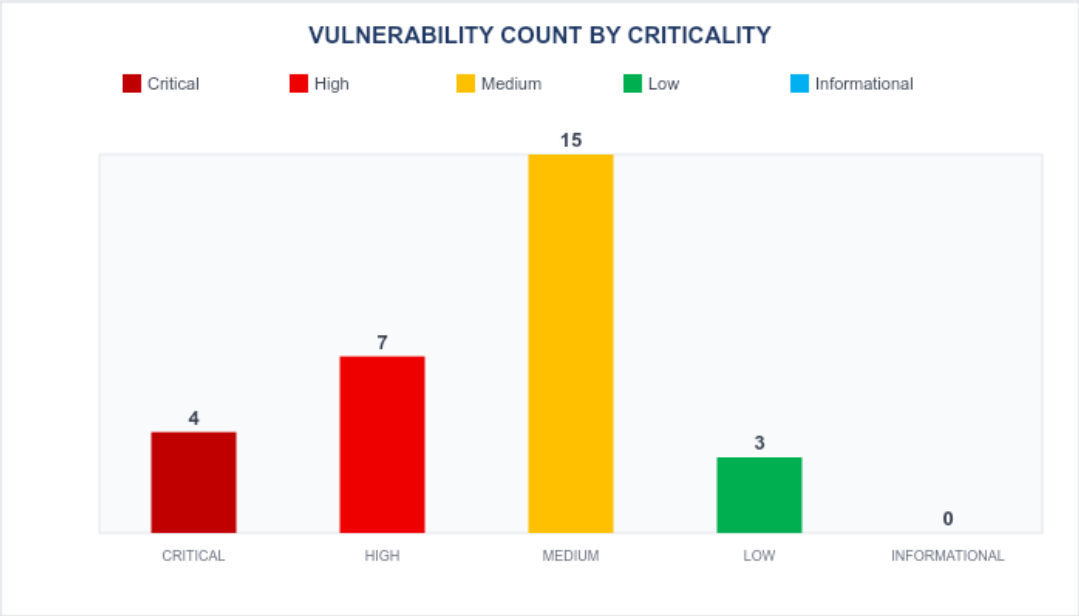
- 4 Critical vulnerabilities
- 7 High vulnerabilities
- 15 Medium vulnerabilities
- 3 Low vulnerabilities
- 0 Informational

Based on the findings, the overall risk rating of the web application is considered Critical.

It is recommended that the organization:

- Perform regular security assessments of web applications and application infrastructure.
- Following remediation, a retest should be conducted to verify that the identified vulnerabilities have been effectively addressed and that no additional security issues remain.

4.1 Vulnerability Chart



4.2 Vulnerability Summary

S. No.	Affected Asset	Observation / Vulnerability Title	CVE / CWE	Severity	New or Repeat Observation
1	https://lockated-api.gophygital.work/task_managements/{id}.json	Insecure Direct Object Reference (IDOR) – Unauthorized Access to Task Details	CWE-639 – Authorization Bypass Through User-Controlled Key	**Critical**	**New**
2	https://web.gophygital.work/safety/report/msafe-detail-report	Insecure Direct Object Reference (IDOR) in MSafe Detail Report Download Functionality	CWE-639: Authorization Bypass Through User-Controlled Key	**Critical**	**New**

S. No.	Affected Asset	Observation / Vulnerability Title	CVE / CWE	Severity	New or Repeat Observation
3	https://live-api.gophygital.work/project_managements/323.json, https://live-api.gophygital.work/api/users/get_organizations_by_email?email=<any-email>	Insecure Direct Object Reference (IDOR) and User Enumeration Leading to Sensitive Information Disclosure	CWE-639 – Authorization Bypass Through User-Controlled Key, CWE-200 – Exposure of Sensitive Information to an Unauthorized Actor, CWE-284 – Improper Access Control	**Critical**	**New**
4	https://web.gophygital.work/maintenance/vendor/view/55820	Insecure Direct Object Reference (IDOR) in Vendor Details Functionality	CWE-639: Authorization Bypass Through User-Controlled Key	**Critical**	**New**

S. No.	Affected Asset	Observation / Vulnerability Title	CVE / CWE	Severity	New or Repeat Observation
5	https://web.gophygital.work/maintenance/service/details/17699	Insecure Direct Object Reference (IDOR) Leading to Unauthorized Access to Sensitive Records and File Attachments	CWE-639 – Authorization Bypass Through User-Controlled Key, CWE-284 – Improper Access Control	High	New
6	https://live-api.gophygital.work/cable?token=eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoxNTY0NjIsImVtYWlsIjoiYWpheS5waWh1bGthckBnb3BoeWdpdGFsLndvcmsiLCJleHAiOjE3ODUyMTg1ODQsImlzcyI6ImxvY2thdGVkLWFwaSIsImF1ZCI6ImxvY2thdGVkLWNsaWVudCJ9.PtR6kvKPebp3N65di11B67OIM1cllaz1HAQgekxU3ZI	JWT Bearer Token Exposed in URL Query Parameter	CWE-598 – Information Exposure Through Query Strings in GET Request, CWE-200 – Exposure of Sensitive Information to an Unauthorized Actor	High	New

S. No.	Affected Asset	Observation / Vulnerability Title	CVE / CWE	Severity	New or Repeat Observation
7	https://web.gophygital.work/	Insecure Direct Object Reference (IDOR) – Unauthorized Access and Modification of Project Details	CWE-639: Authorization Bypass Through User-Controlled Key	**High**	**New**
8	https://web.gophygital.work/	Insecure Direct Object Reference (IDOR) – Unauthorized Access and Modification of Maintenance Tickets	CWE-639: Authorization Bypass Through User-Controlled Key	**High**	**New**
9	https://web.gophygital.work/	Stored Cross-Site Scripting (Stored XSS) via Malicious PDF Upload	CWE-79: Improper Neutralization of Input During Web Page Generation (Cross-site Scripting)	**High**	**New**

S. No.	Affected Asset	Observation / Vulnerability Title	CVE / CWE	Severity	New or Repeat Observation
10	https://web.gophygital.work/	Stored Cross-Site Scripting (Stored XSS) via Malicious PDF Attachment Upload in Ticket Comments	CWE-79: Improper Neutralization of Input During Web Page Generation (Cross-Site Scripting)	High	New
11	https://web.gophygital.work/maintenance/inventory/details/<ID>	Insecure Direct Object Reference (IDOR) Allows Unauthorized Access to Inventory Details	CWE-639 – Authorization Bypass Through User-Controlled Key, CWE-284 – Improper Access Control	High	New
12	https://web.gophygital.work/login	Clickjacking Vulnerability on Login Page	CWE-1021: Improper Restriction of Rendered UI Layers or Frames	Medium	New

S. No.	Affected Asset	Observation / Vulnerability Title	CVE / CWE	Severity	New or Repeat Observation
13	https://web.gophygital.work/safety/incident/new-details/3181	Insecure Direct Object Reference (IDOR) in Incident Details Page	CWE-639: Authorization Bypass Through User-Controlled Key	Medium	New
14	https://web.gophygital.work/dashboard	Session Not Invalidated After Logout	CWE-613: Insufficient Session Expiration	Medium	New
15	https://web.gophygital.work/	Internal IP Address Disclosure	CWE-200: Exposure of Sensitive Information to an Unauthorized Actor	Medium	New
16	https://web.gophygital.work	Missing HTTP Security Headers	CWE-693 – Protection Mechanism Failure	Medium	New

S. No.	Affected Asset	Observation / Vulnerability Title	CVE / CWE	Severity	New or Repeat Observation
17	https://web.gophygital.work	Web Server Version Information Disclosure	CWE-200 – Exposure of Sensitive Information to an Unauthorized Actor	Medium	New
18	https://web.gophygital.work/	SSH Server Supports Weak Message Authentication Code (MAC) Algorithms (SHA-1)	CWE-327: Use of a Broken or Risky Cryptographic Algorithm	Medium	New
19	https://web.gophygital.work/	Replay Attack on Comment Submission API (Duplicate Comment Creation)	CWE-294: Authentication Bypass by Capture-Replay (Replay attack), CWE-799: Improper Control of Interaction Frequency (No rate limiting contributes to the impact)	Medium	New

S. No.	Affected Asset	Observation / Vulnerability Title	CVE / CWE	Severity	New or Repeat Observation
20	https://web.gophygital.work/	Sensitive Information Disclosure via EXIF Metadata in Uploaded Image Attachments	CWE-200: Exposure of Sensitive Information to an Unauthorized Actor	Medium	New
21	web.gophygital.work	Denial of Service via Multiple Overlapping HTTP Byte-Range Requests	CWE-400 – Uncontrolled Resource Consumption	Medium	New
22	https://web.gophygital.work/business-compass/profile	Insecure Storage of Session/Bearer Token in Browser Local Storage	CWE-922 – Insecure Storage of Sensitive Information	Medium	New
23	https://web.gophygital.work/maintenance/inventory, https://web.gophygital.work/maintenance/service	Improper Input Validation in Restricted Dropdown Fields	CWE-20: Improper Input Validation	Medium	New

S. No.	Affected Asset	Observation / Vulnerability Title	CVE / CWE	Severity	New or Repeat Observation
24	https://live-api.gophygital.work/api/users/sign_in.json	Session Reuse Leads To Authentication Bypass	CWE-613 – Insufficient Session Expiration	Medium	New
25	https://web.gophygital.work/safety/incident, https://web.gophygital.work/maintenance/ticket	Lack of Rate Limiting on Incident Creation Functionality	CWE-770: Allocation of Resources Without Limits or Throttling	Medium	New
26	https://web.gophygital.work/safety/incident	Improper Server-Side Validation of Character Length Restrictions	CWE-20: Improper Input Validation	Medium	New
27	https://web.gophygital.work/	Server Version Disclosure via HTTP Response Headers	CWE-200: Exposure of Sensitive Information to an Unauthorized Actor	Low	New

S. No.	Affected Asset	Observation / Vulnerability Title	CVE / CWE	Severity	New or Repeat Observation
28	https://web.gophygital.work/forgot-password, https://web.gophygital.work	Username Enumeration via Distinct Password Reset Error Messages	CWE-203 – Observable Discrepancy, CWE-204 – Observable Response Discrepancy	Low	New
29	https://web.gophygital.work/login	Account Enumeration via Authentication Response Messages	CWE-203 – Observable Discrepancy	Low	New

5. Detailed Observations

5.1 Insecure Direct Object Reference (IDOR) – Unauthorized Access to Task Details

Affected Asset: https://lockated-api.gophygital.work/task_managements/{id}.json

Affected Parameter: id (Path Parameter)

CWE-ID: CWE-639 – Authorization Bypass Through User-Controlled Key

Severity: **Critical**

Description:

The application is vulnerable to an Insecure Direct Object Reference (IDOR) due to missing server-side authorization checks on the id path parameter of the task management API. An authenticated user can modify the task ID in the request URL and retrieve details of tasks that belong to other users or projects. The server responds with HTTP 200 OK and returns sensitive task information without verifying whether the requesting user is authorized to access the requested resource. During testing, changing the task identifier from 43520 to 43420 successfully returned the details of another task, confirming that access control is enforced only by the client and not validated on the server.

Impact:

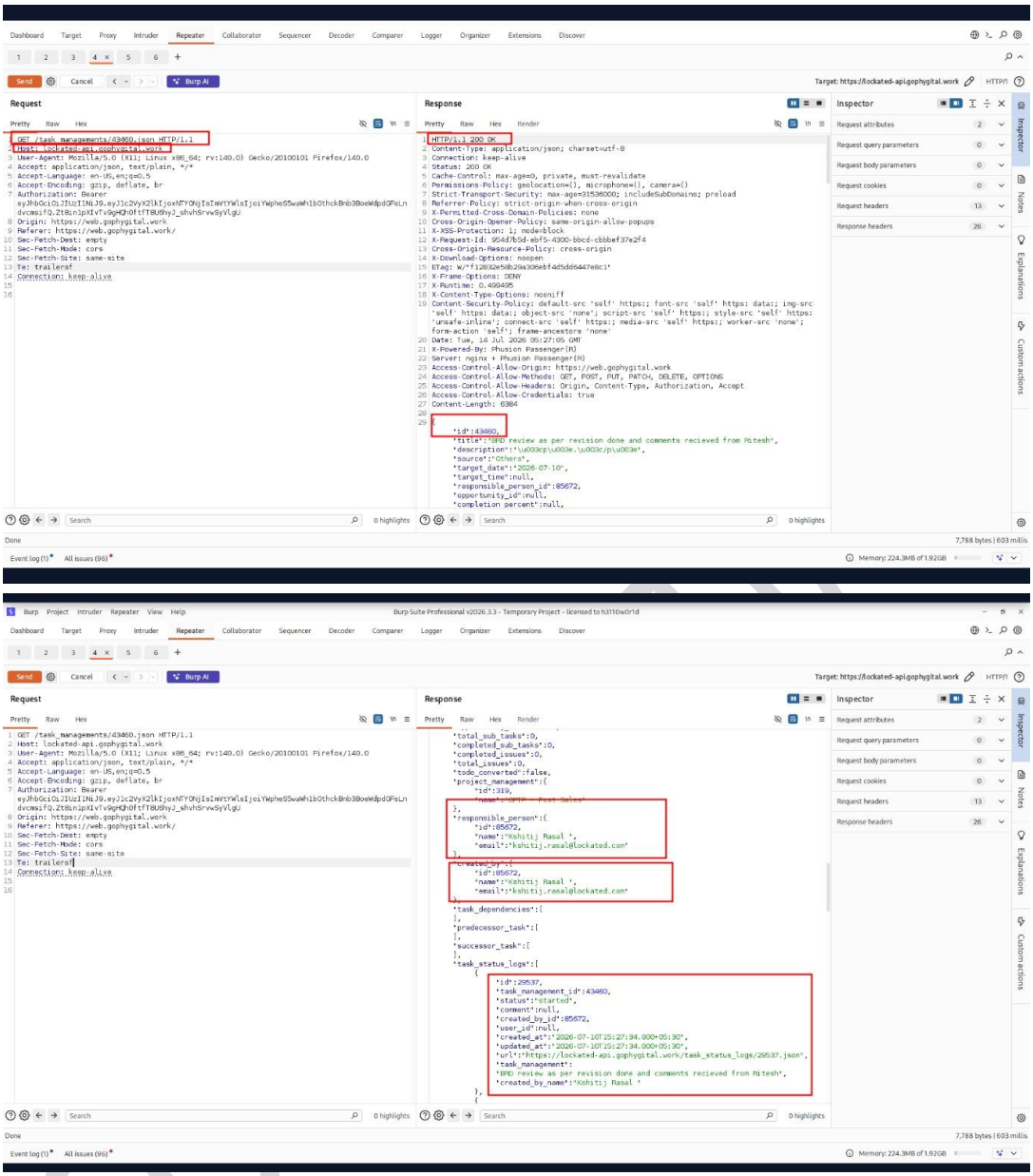
Unauthorized users can access tasks belonging to other users or projects. Disclosure of sensitive project information, including task descriptions, project names, milestones, employee names, and email addresses. Enables enumeration of task IDs and large-scale data harvesting. May expose confidential business information and internal workflow data.

References to Evidences and Proof of Concept:

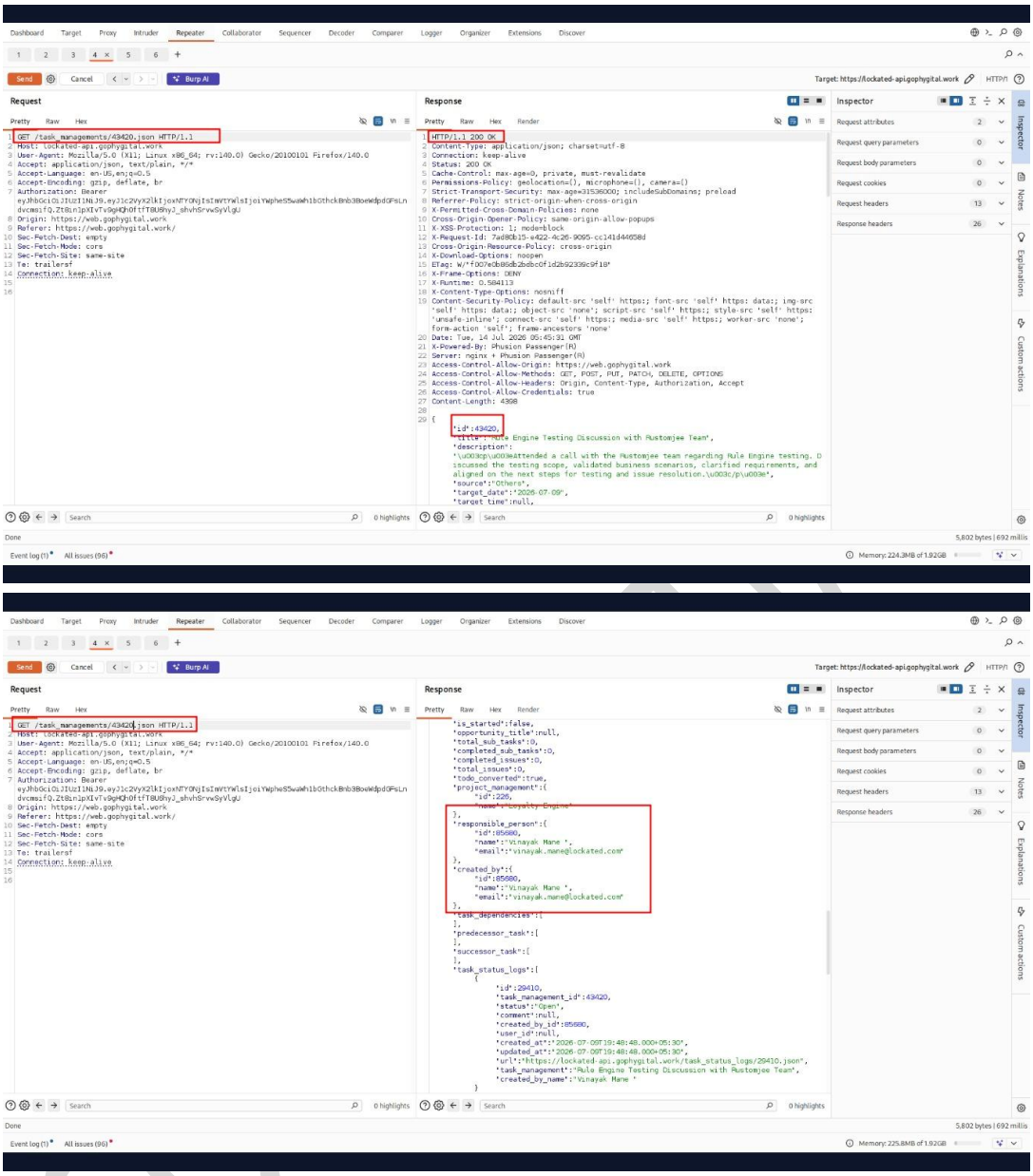
Step 1: open this URL https://web.gophygital.work/ and login with valid credintials

Step 2: Navigate through task tab

Step 3: Intercept this (GET /task_managements/43460.json) request using burp suit



Step 4: Modify the id path parameter from: 43460 to another valid task ID, for example:43420



Step 5: Forward the modified request.

Step 6: Observe that the server returns HTTP/1.1 200 OK along with the complete details of another user's task instead of returning 403 Forbidden or 404 Not Found, demonstrating an authorization bypass.

Recommendations:

Enforce server-side authorization checks for every request to /task_managements/{id}. Verify that the authenticated user has explicit permission to access the requested task before returning data. Return 403 Forbidden (or 404 Not Found) when access is unauthorized. Implement object-level authorization (BOLA) consistently across all API endpoints. Perform periodic access control testing to identify similar authorization issues across the application.

5.2 Insecure Direct Object Reference (IDOR) in MSafe Detail Report Download Functionality

Affected Asset: https://web.gophygital.work/safety/report/msafe-detail-report

Affected Parameter: company_id

CWE-ID: CWE-639: Authorization Bypass Through User-Controlled Key

Severity: **Critical**

Description:

The MSafe Detail Report download functionality is vulnerable to an Insecure Direct Object Reference (IDOR). During testing, it was observed that the application uses a user-controlled parameter (company_id) to determine which organization's report is generated and downloaded.

When accessing the report download feature, the application initially displayed a message indicating that no report was available for the current day. However, upon intercepting the request, it was identified that the request contained the parameter: *company_id=301*

By modifying the value of company_id to another valid identifier (e.g., 145), the application generated and returned an Excel report belonging to a different organization without performing proper authorization checks.

The downloaded report contained extensive sensitive information associated with employees and contractors from another organization.

This demonstrates a failure to enforce object-level authorization controls, allowing unauthorized access to highly sensitive business and personal data.

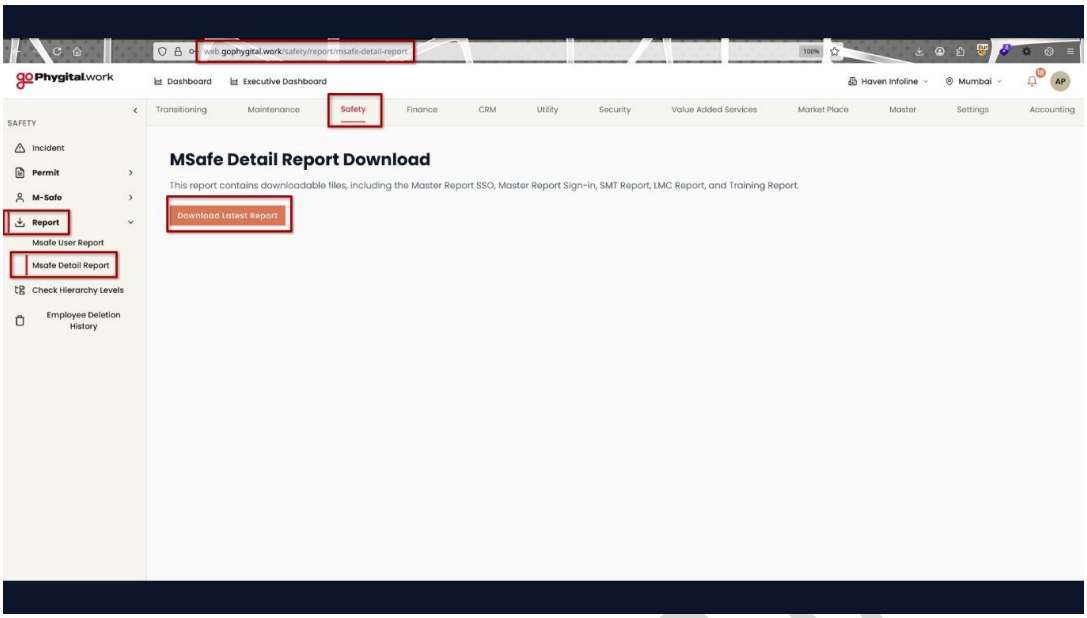
Impact:

- Unauthorized access to reports belonging to other organizations.
- Disclosure of Personally Identifiable Information (PII).
- Exposure of employee and contractor records.
- Exposure of vehicle and licensing information.
- Exposure of employment-related data.
- Violation of confidentiality requirements and data protection regulations.
- Potential identity theft, social engineering, and targeted attacks.
- Large-scale data harvesting through enumeration of company_id values.

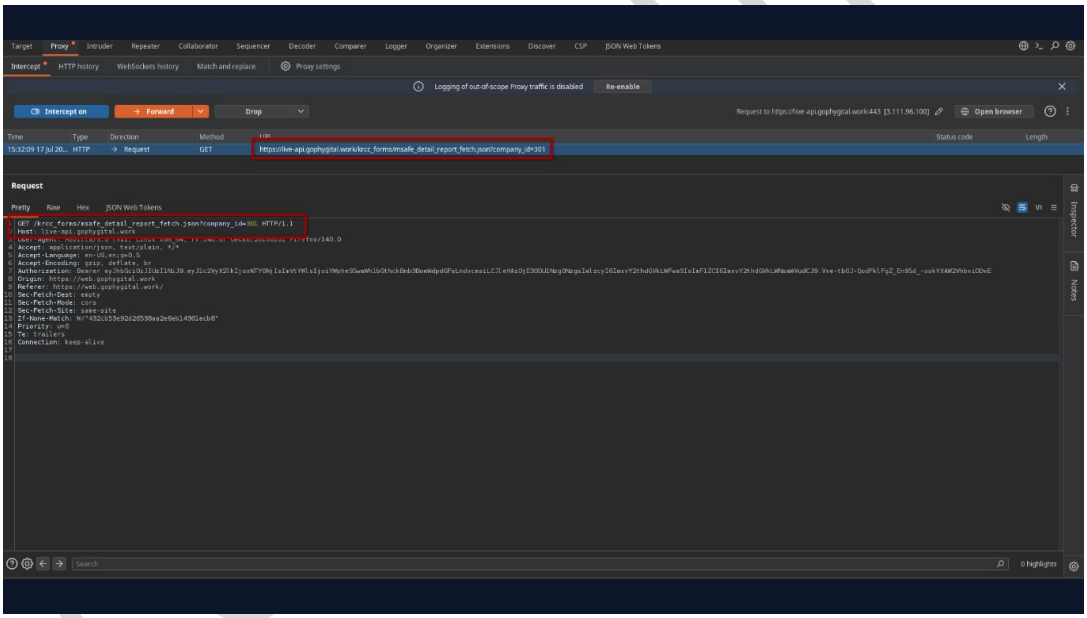
References to Evidences and Proof of Concept:

Step 1: Log in to the application with a valid account.

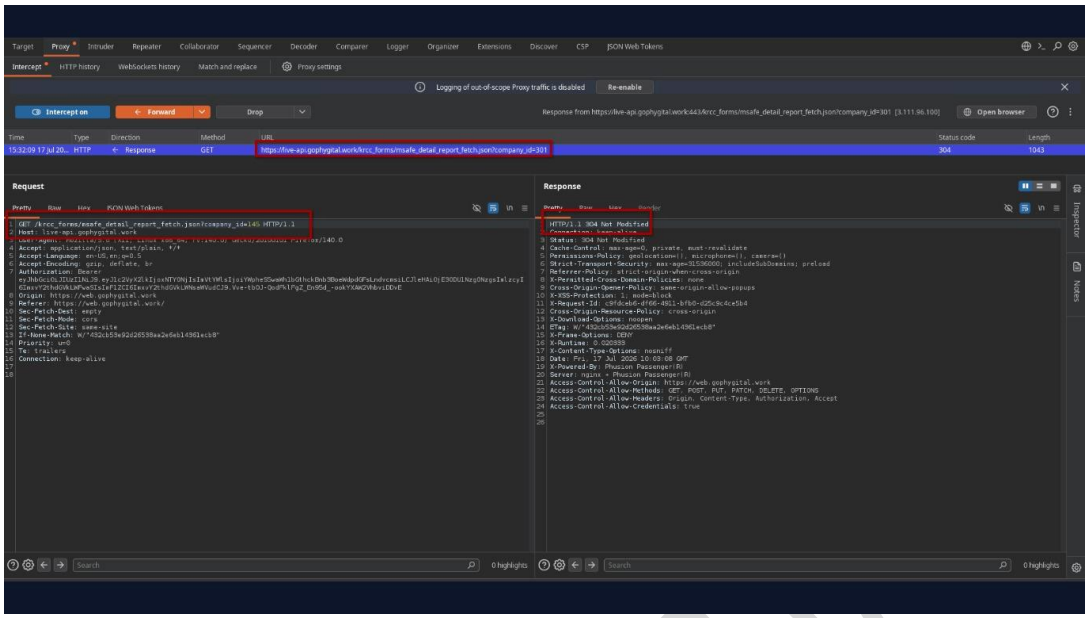
Step 2: Navigate to the MSafe Detail Report Download functionality.



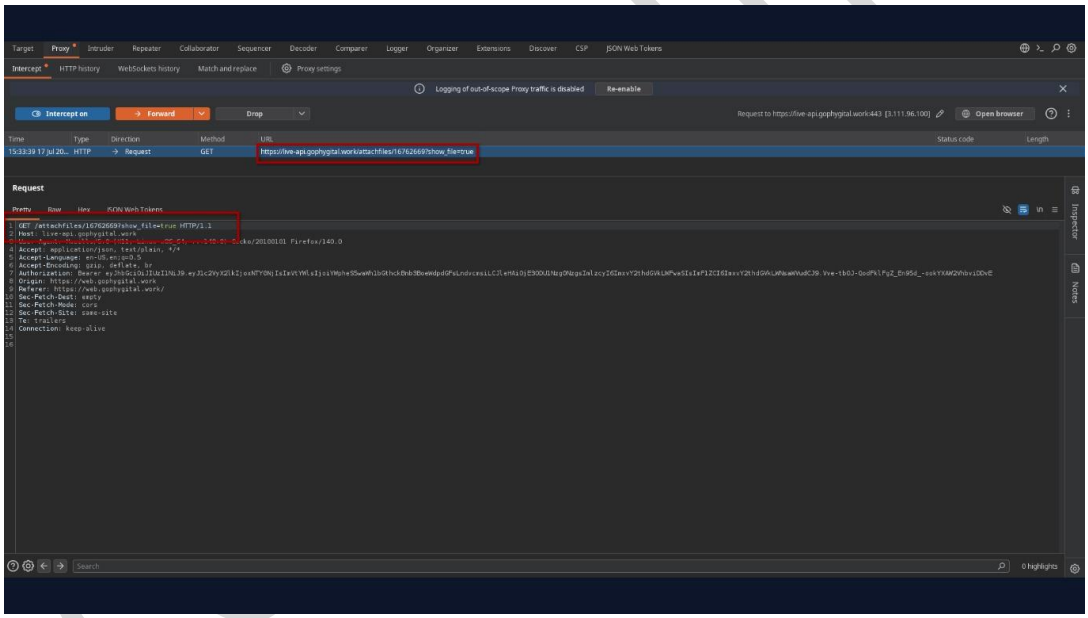
Step 3: Initiate a report download request and intercept it using Burp Suite.

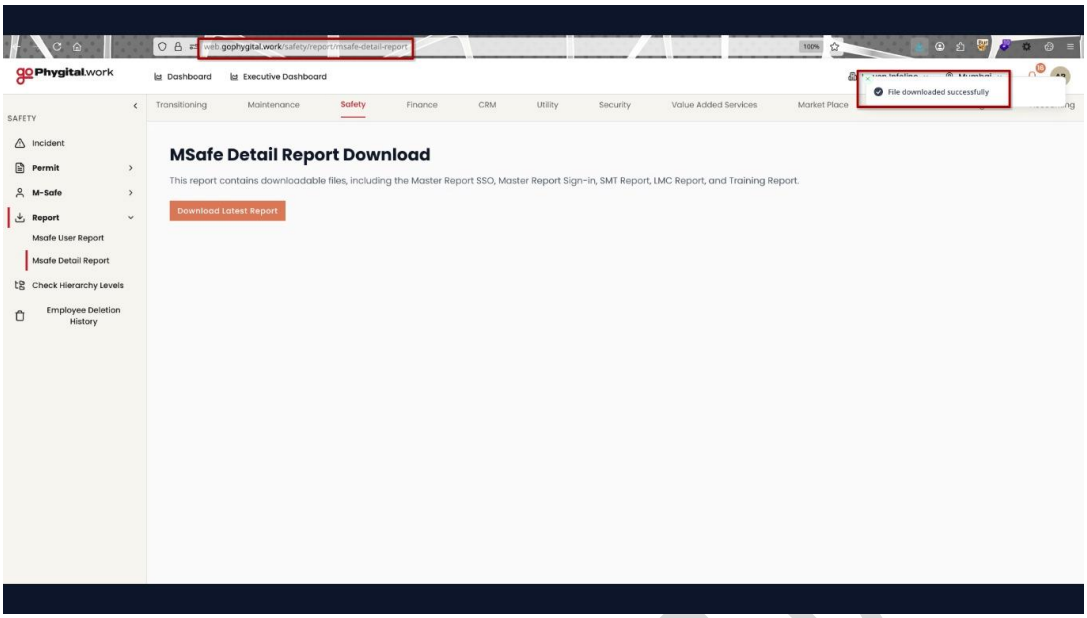


Step 4: Modify the parameter value and forward the request: company_id=145

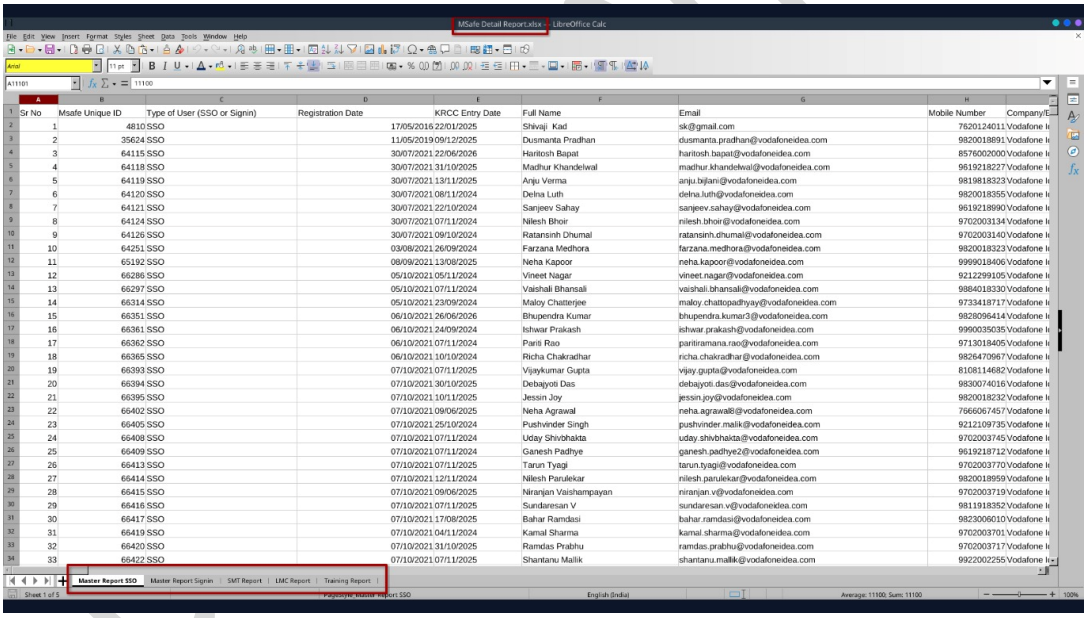


Step 5: Observe that the application returns an XLSX report belonging to another organization.





Step 6: Verify that the downloaded report contains data unrelated to the authenticated user's organization.



Recommendations:

- Implement strict server-side authorization checks for all report generation and download requests.
- Verify that the authenticated user is authorized to access the specified company_id.
- Do not rely on client-supplied identifiers for access control decisions.
- Enforce tenant-level access controls for multi-organization environments.
- Replace predictable identifiers with indirect references where appropriate.
- Log and monitor report access attempts involving unauthorized company identifiers.

- Conduct a comprehensive review of all export and reporting functionalities for similar authorization weaknesses.

5.3 Insecure Direct Object Reference (IDOR) and User Enumeration Leading to Sensitive Information Disclosure

Affected Asset:

- https://live-api.gophygital.work/project_managements/323.json
- https://live-api.gophygital.work/api/users/get_organizations_by_email?email=<any-email>

Affected Parameter: Maintenance --- Task

CWE-ID:

- CWE-639 – Authorization Bypass Through User-Controlled Key
- CWE-200 – Exposure of Sensitive Information to an Unauthorized Actor
- CWE-284 – Improper Access Control

Severity: **Critical**

Description:

The application is vulnerable to an Insecure Direct Object Reference (IDOR) on the ***/project_managements/{id}.json*** endpoint. An authenticated user can modify the numeric identifier in the request URL to access project records belonging to other users without proper authorization checks.

By automating requests with sequential IDs (e.g., using Burp Suite Intruder), an attacker can enumerate a large number of project records and obtain sensitive user information, including user names, email addresses, and assigned roles.

Furthermore, the disclosed email addresses can be used with another endpoint, /api/users/get_organizations_by_email?email=<email>, to retrieve additional information associated with the identified users. This allows an attacker to correlate data across multiple endpoints and significantly increases the amount of information exposed.

The lack of object-level authorization and unrestricted access to user-related APIs enables large-scale information disclosure and user enumeration.

Impact:

- Unauthorized users can enumerate project records belonging to other users by modifying the object ID.
- Exposure of sensitive user information, including email addresses and assigned roles.
- Disclosed email addresses can be used to retrieve additional user information through the /api/users/get_organizations_by_email endpoint.
- Attackers can identify privileged accounts (e.g., administrators or managers) and target them in phishing or social engineering attacks.
- Automated enumeration can lead to large-scale harvesting of user and organizational data.
- The vulnerability may result in unauthorized disclosure of personally identifiable information (PII) and potential compliance violations.

- Information obtained from this vulnerability can be combined with other vulnerabilities to facilitate further attacks against the application.

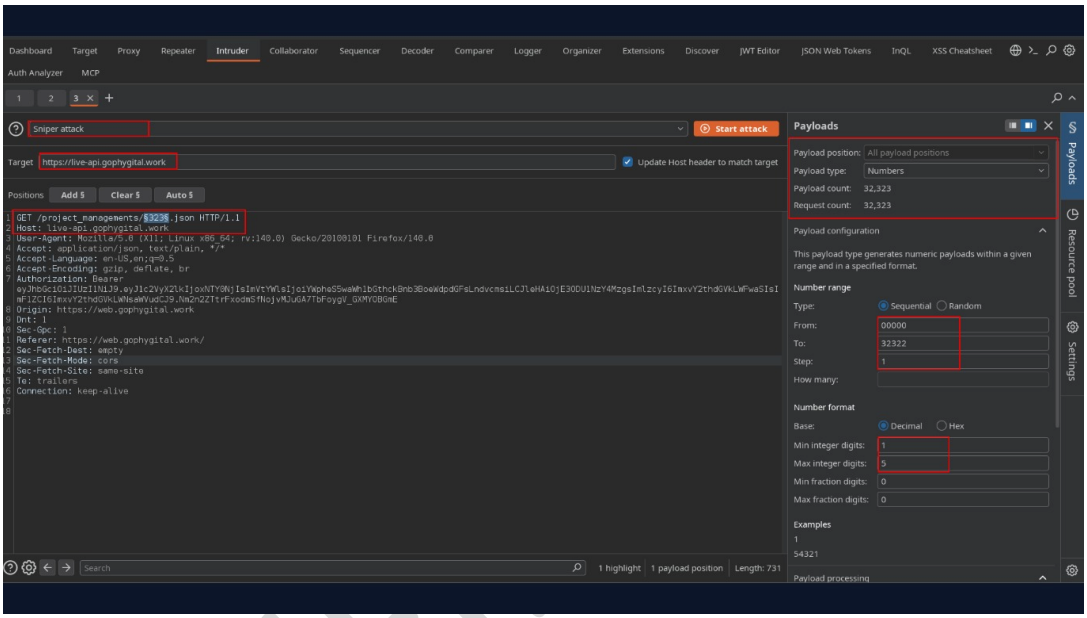
References to Evidences and Proof of Concept:

Step 1: Authenticate to the application using a valid account https://web.gophygital.work

Step 2: Browse to maintenance tab and explore the task tab in web application.

Step 3: Now go to burp suite HTTP history and search for this request https://live-api.gophygital.work/project_managements/<ID>.json

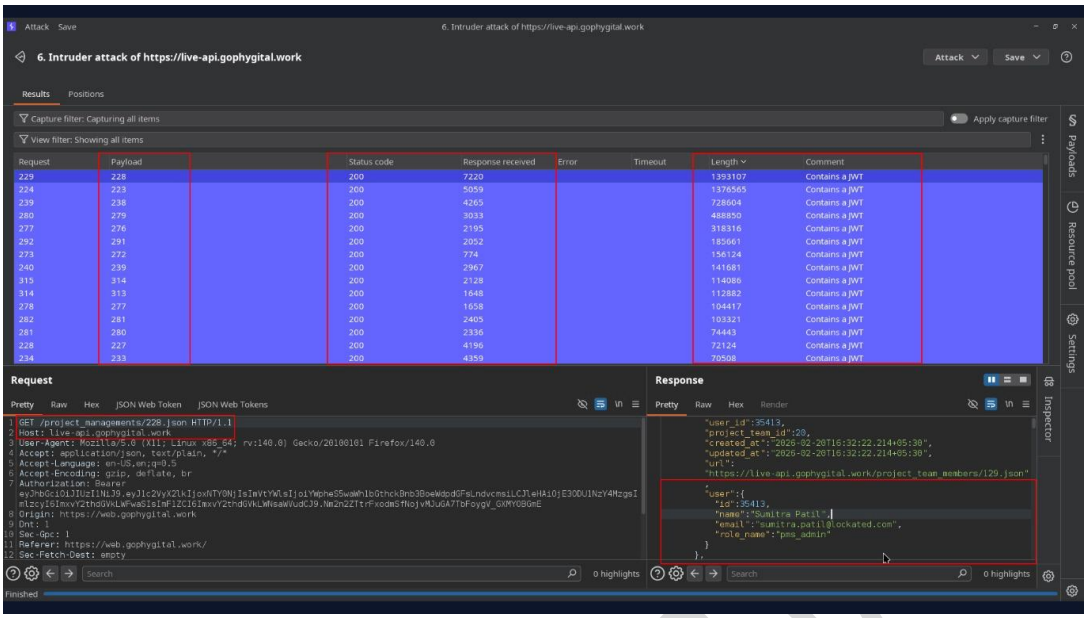
Step 4: Send the request to Burp Suite Intruder.



Step 5: Select the numeric project ID as the payload position.

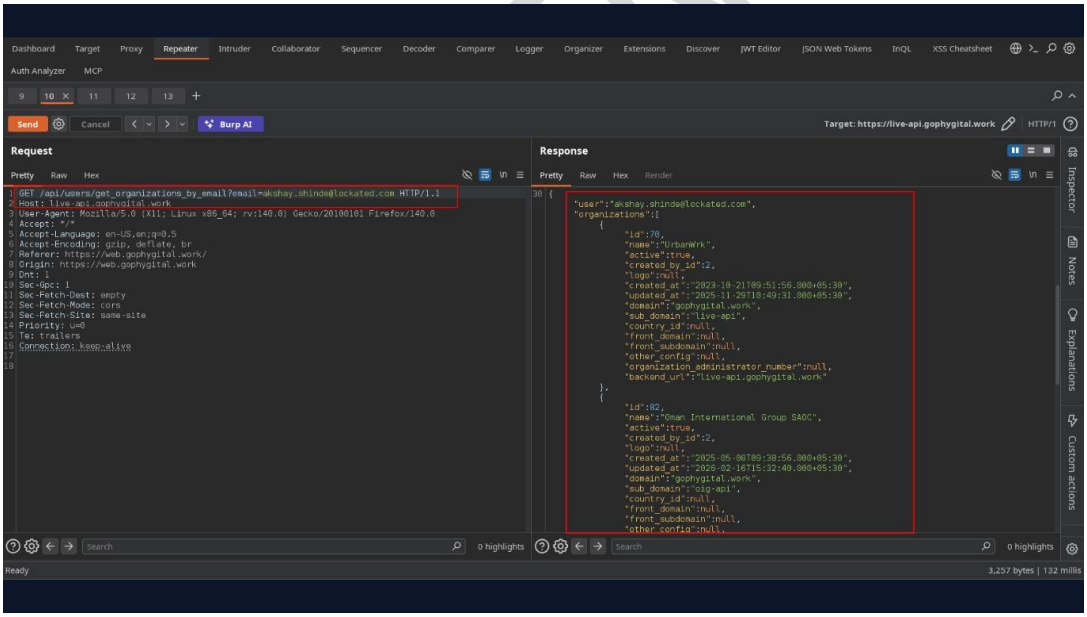
Step 6: Configure a sequential payload (e.g., 1–30000).

Step 7: Start the attack and observe that responses contain information for different users.



Step 8: Note the disclosed email addresses and user roles from the responses.

Step 9: Use one of the exposed email addresses in the following request: https://live-api.gophygital.work/api/users/get_organizations_by_email?email=<email>



Step 10: Observe that the application returns additional information associated with the specified email address without verifying that the requesting user is authorized to access it.

Recommendations:

- Prevent sequential enumeration by validating ownership before returning project data.
- Apply authorization checks consistently across all user-related API endpoints, including email lookup functionality.
- Avoid exposing unnecessary user attributes such as email addresses and roles unless strictly required.

- Implement rate limiting and anomaly detection to identify large-scale enumeration attempts.
- Return generic error responses (403 Forbidden or 404 Not Found) for unauthorized object access.
- Perform regular access control testing to identify and remediate Broken Object Level Authorization (BOLA/IDOR) vulnerabilities across the application.

5.4 Insecure Direct Object Reference (IDOR) in Vendor Details Functionality

Affected Asset: https://web.gophygital.work/maintenance/vendor/view/55820

Affected Parameter: /pms/suppliers/{id}.json

CWE-ID: CWE-639: Authorization Bypass Through User-Controlled Key

Severity: **Critical**

Description:

The application is vulnerable to an Insecure Direct Object Reference (IDOR) within the Vendor Details functionality. Users are intended to have access only to vendor records explicitly assigned to their account. However, the application fails to enforce proper server-side authorization checks when accessing vendor records.

During testing, it was observed that modifying the vendor identifier in the URL allowed access to vendor records that were not assigned to the authenticated user. By changing the value of the vendor_id parameter, arbitrary vendor records could be accessed without authorization.

The exposed vendor records contain highly sensitive business information, including contact details, banking information, financial data, audit information, attachments, and approval status data.

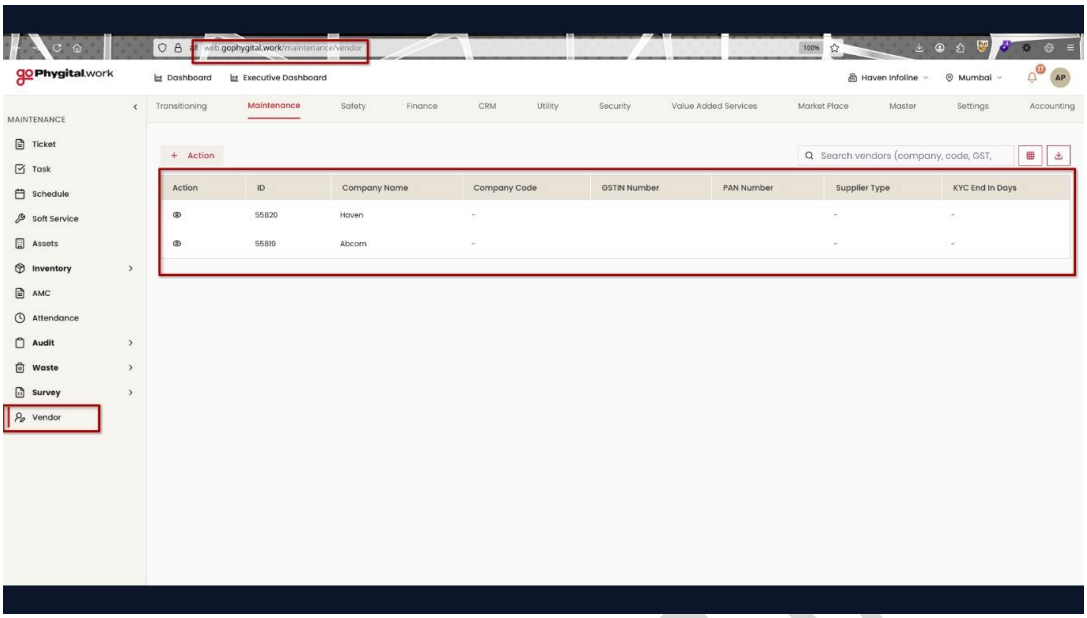
Impact:

- Unauthorized access to vendor records belonging to other organizations.
- Disclosure of sensitive vendor and business information.
- Exposure of banking and financial details.
- Access to vendor contact information and business records.
- Exposure of audit-related information.
- Unauthorized access to uploaded documents and attachments.
- Enumeration of vendor records through predictable identifiers.
- Potential financial fraud, business impersonation, and targeted social engineering attacks.
- Violation of confidentiality and access control requirements.

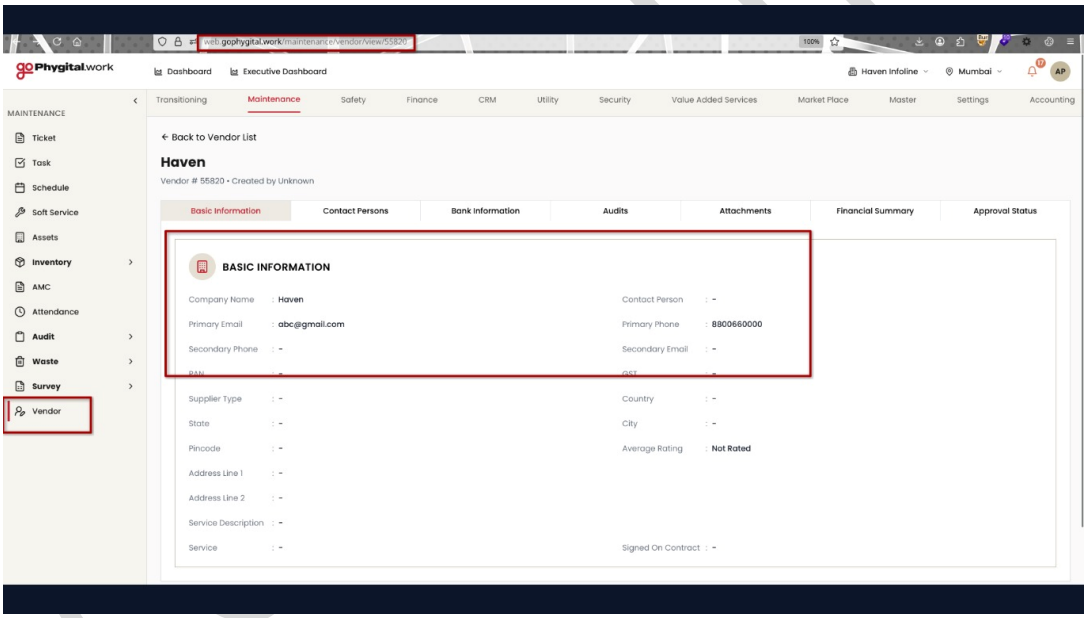
References to Evidences and Proof of Concept:

Step 1: Authenticate to the application using a valid user account.

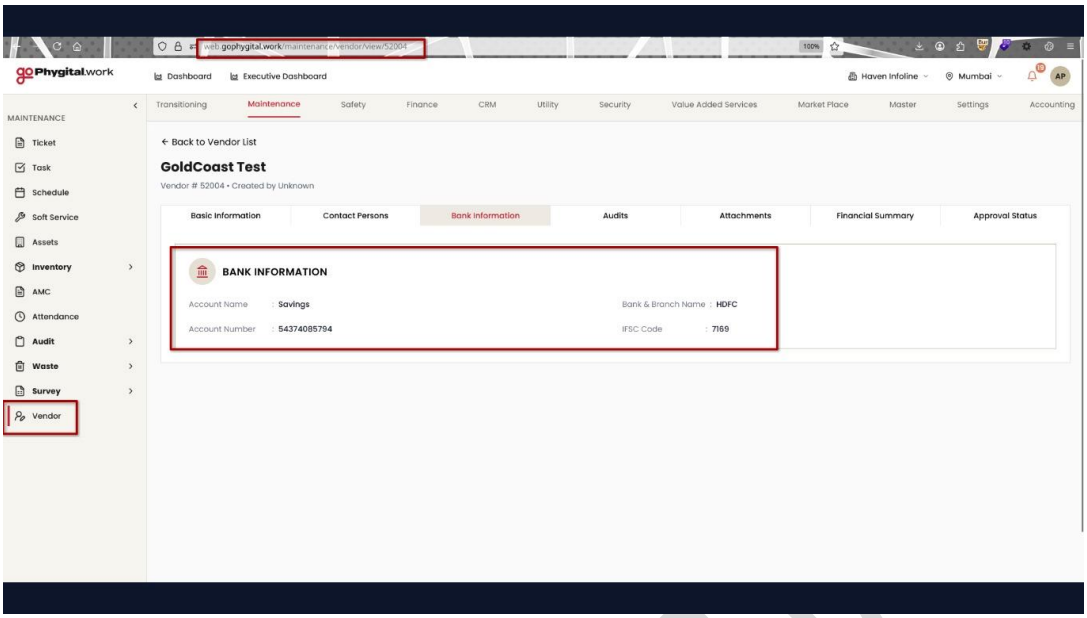
Step 2: Navigate to the Vendor Management section.



Step 3: Open one of the vendor records available to the authenticated account.



Step 4: Modify the vendor identifier in the URL.



Step 5: Observe that the application loads another vendor record without performing authorization validation.

Step 6: Verify that sensitive information belonging to a different vendor is disclosed.

Step 7: Repeat the process with additional vendor identifiers and confirm that multiple vendor records can be accessed through parameter manipulation.

Recommendations:

- Implement robust server-side authorization checks for every vendor access request.
- Verify ownership and access permissions before returning vendor data.
- Enforce tenant-level segregation of vendor records.
- Apply Role-Based Access Control (RBAC) to vendor management functions.
- Avoid relying on client-controlled identifiers for authorization decisions.
- Consider using non-predictable identifiers (UUIDs) where feasible.
- Log and monitor unauthorized access attempts.
- Conduct a comprehensive access-control review across all vendor-related endpoints and APIs.

5.5 Insecure Direct Object Reference (IDOR) Leading to Unauthorized Access to Sensitive Records and File Attachments

Affected Asset: https://web.gophygital.work/maintenance/service/details/17699

Affected Parameter: Soft Service

CWE-ID:

- CWE-639 – Authorization Bypass Through User-Controlled Key
- CWE-284 – Improper Access Control

Severity: High

Description:

The application is vulnerable to an Insecure Direct Object Reference (IDOR) vulnerability in the endpoint: https://web.gophygital.work/maintenance/service/details/17699 The endpoint uses a numeric identifier to retrieve service records. During testing, it was observed that modifying the identifier (e.g., 16679 to another valid ID) allows an authenticated user to access records belonging to other users without any authorization checks. The server responds with sensitive information associated with the requested record regardless of ownership. Additionally, if a record contains uploaded attachments, those files can also be accessed and downloaded by simply requesting another valid identifier. This indicates that the application performs object lookup based solely on the user-supplied identifier and fails to verify whether the requesting user is authorized to access the requested resource.

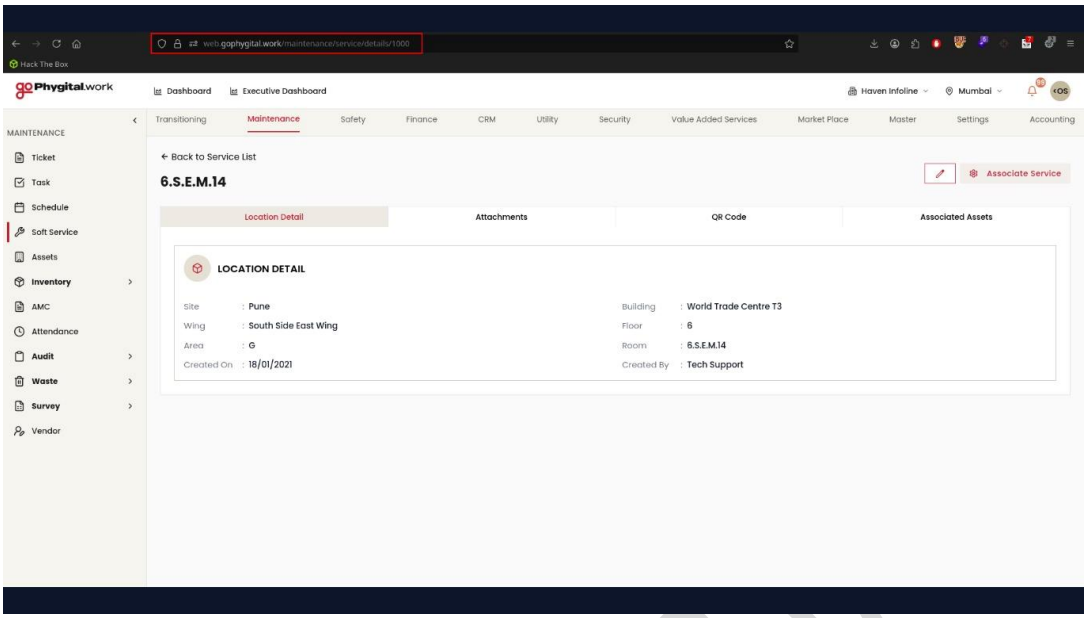
Impact:

- Access sensitive information belonging to other users.
- Enumerate valid record identifiers and harvest large amounts of data.
- Download confidential documents and attachments uploaded by other users.
- Compromise the confidentiality and privacy of user information. Obtain personal, financial, medical, or business-related information depending on the application's purpose.
- Facilitate further attacks using the disclosed information, such as identity theft, phishing, or social engineering. Result in regulatory and compliance violations due to unauthorized disclosure of sensitive data.

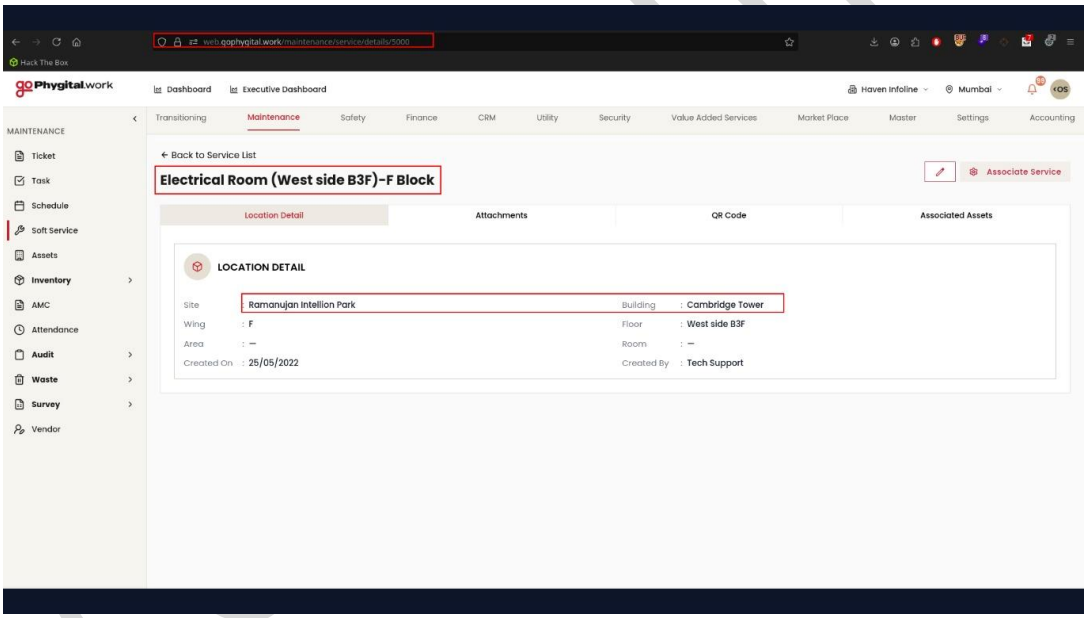
References to Evidences and Proof of Concept:

Step 1: Log in to the application using a valid user account.

Step 2: Navigate to a service record and observe the URL in the browser, for example: https://web.gophygital.work/maintenance/service/details/17699

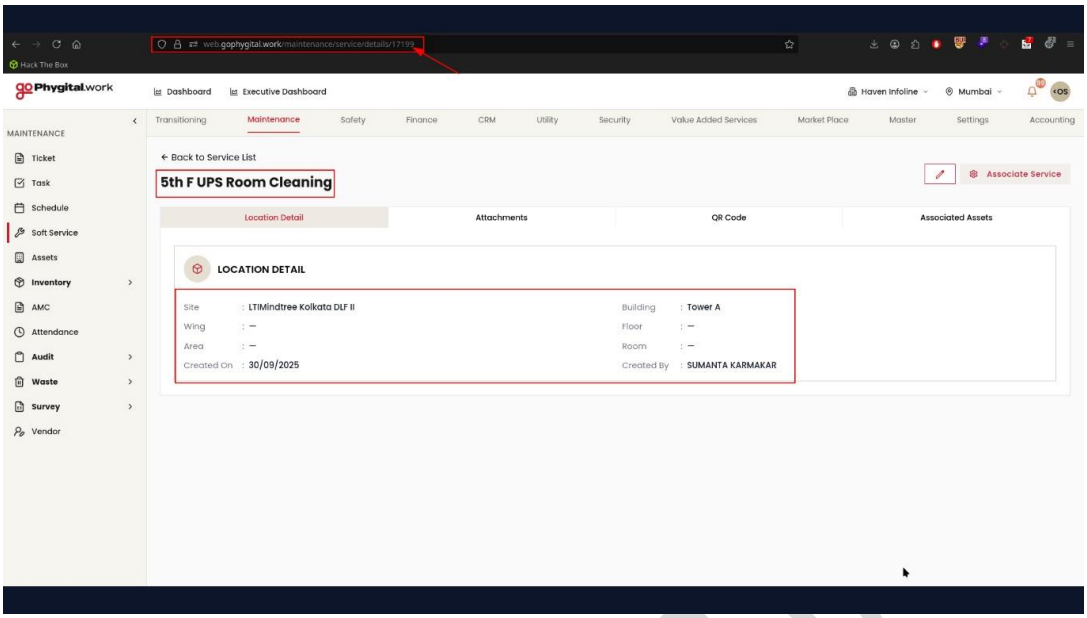


Step 3: Modify the numeric identifier in the URL to another valid service ID.



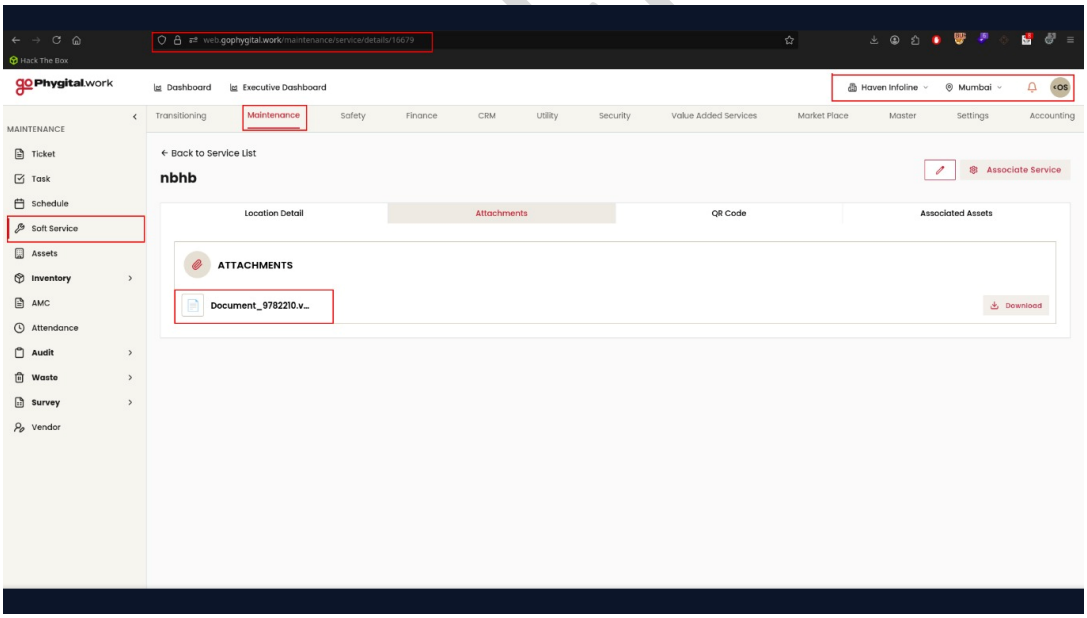
Step 4: Observe that the application returns the details of another user's service record without verifying whether the authenticated user is authorized to access the requested resource.

Step 5: Review the response and verify that sensitive information belonging to another user is disclosed.



Step 6: If the record contains uploaded attachments, access or download the attachment from the returned record.

Step 7: Observe that the attachment is accessible and can be downloaded without proper authorization.



Step 8: Repeat the process with additional valid identifiers to confirm that records and associated attachments belonging to multiple users can be accessed.

Recommendations:

- Implement server-side authorization checks for every request accessing user-specific resources.
- Verify that the authenticated user is authorized to access the requested object before returning any data or attachments.

- Never rely solely on client-supplied object identifiers for access control. Enforce ownership or role-based access control (RBAC/ABAC) for all records and associated files.
- Apply the same authorization checks to attachment download endpoints to prevent unauthorized file access.
- Consider using indirect, non-predictable object references (such as UUIDs) to make identifier enumeration more difficult. Note that this should complement—not replace—proper authorization.
- Log and monitor unauthorized access attempts and repeated enumeration of object identifiers. Conduct periodic access control testing to ensure all endpoints consistently enforce authorization.

5.6 JWT Bearer Token Exposed in URL Query Parameter

Affected Asset: https://live-api.gophygital.work/cable?token=eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoxNTY0NjIsImVtYWlsIjoiYWpheS5waWh1bGthckBnb3BoeWdpdGFsLndvcmsiLCJleHAiOjE3ODUyMTg1ODQsImlzcyI6ImxvY2thdGVkLWFwaSIsImF1ZCI6ImxvY2thdGVkLWNsaWVudCJ9.PtR6kvKPebp3N65di11B67OIM1cllaz1HAQgekxU3ZI

Affected Parameter: Bearer Token

CWE-ID:

- CWE-598 – Information Exposure Through Query Strings in GET Request
- CWE-200 – Exposure of Sensitive Information to an Unauthorized Actor

Severity: High

Description:

During security testing, it was observed that the application transmits a JSON Web Token (JWT) as a query string parameter in a GET request. Passing authentication tokens in the URL exposes sensitive credentials because URLs may be stored in browser history, web server logs, reverse proxy logs, CDN logs, monitoring systems, and analytics platforms. Additionally, the URL may be disclosed through the HTTP Referer header when users navigate to external websites. If an attacker gains access to the exposed JWT before it expires, they may be able to replay the token and impersonate the legitimate user, resulting in unauthorized access to protected resources.

Impact:

Exposure of JWT tokens in browser history. Leakage of authentication tokens through server, proxy, CDN, and application logs. Potential disclosure via the HTTP Referer header. Increased risk of session hijacking if the token is intercepted. Unauthorized access to user accounts and sensitive information. Violation of secure authentication and session management best practices.

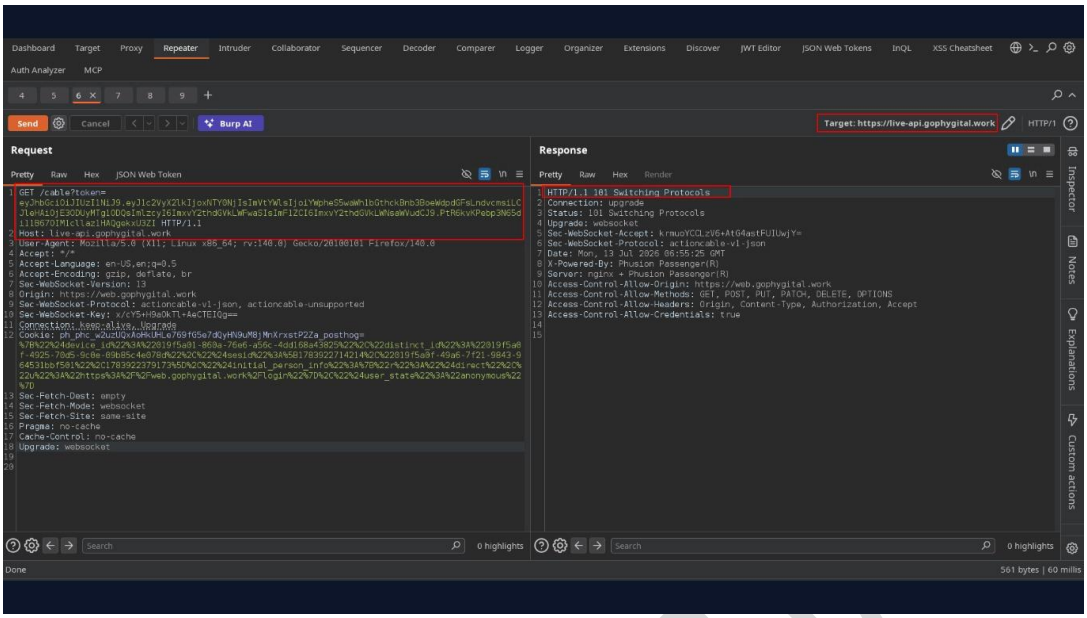
References to Evidences and Proof of Concept:

Step 1: Authenticate to the application using a valid user account.

Step 2: Intercept the application's traffic using Burp Suite.

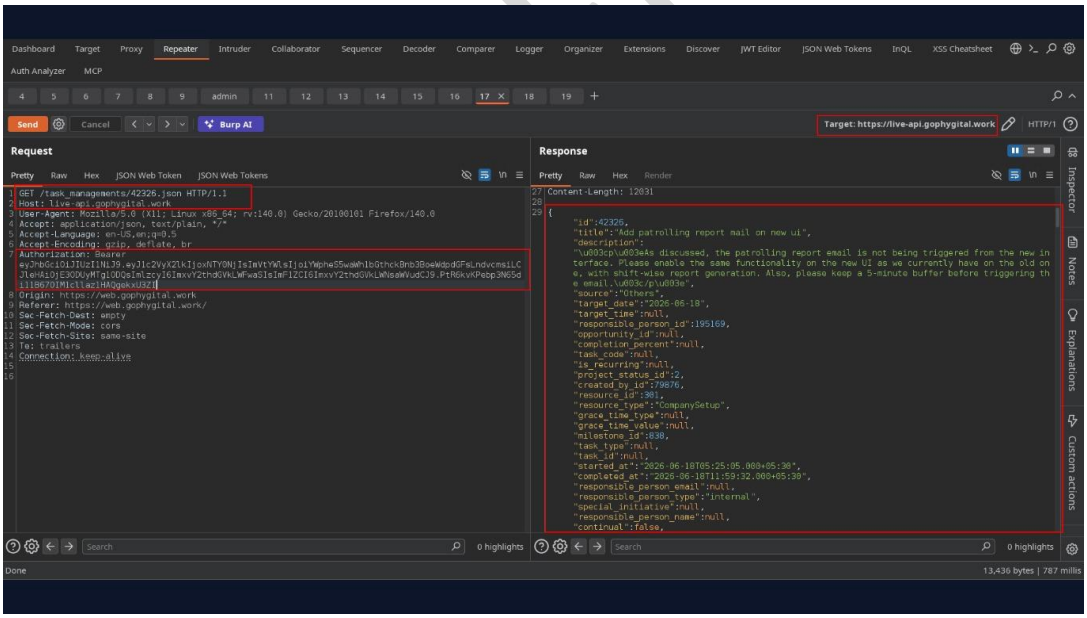
Step 3: Navigate to any functionality that establishes the connection.

Step 4: Verify that the JWT is included as the value of the token query parameter.



Step 5: Observe that the authentication token is transmitted within the URL instead of the Authorization header.

Step 6: Note that the token may be exposed through browser history, application logs, reverse proxies, and the HTTP Referer header.



Recommendations:

Do not transmit JWTs or any authentication credentials in URL query parameters. Send access tokens exclusively in the Authorization: Bearer HTTP header. If applicable, use HttpOnly, Secure, and SameSite cookies for session management. Remove or redact sensitive query parameters from server and application logs. Configure applications to prevent authentication tokens from appearing in URLs. Implement short-lived access tokens with refresh token rotation and support immediate token revocation. Review existing logs for exposed tokens and invalidate any tokens that may have been disclosed.

5.7 Insecure Direct Object Reference (IDOR) – Unauthorized Access and Modification of Project Details

Affected Asset: https://web.gophygital.work/

Affected Parameter: /vas/projects/details/{project_id}

CWE-ID: CWE-639: Authorization Bypass Through User-Controlled Key

Severity: High

Description:

The application is vulnerable to an **Insecure Direct Object Reference (IDOR)** due to insufficient server-side authorization checks on project resources. Authenticated users can modify the numeric project identifier in the URL and access project details belonging to other users or teams without proper authorization.

During testing, changing the project ID from **323** to **324** allowed access to another project's information. Additionally, the application allowed the authenticated user to access the **Edit Project** functionality for the unauthorized project, indicating that authorization checks were not properly enforced for project modification operations.

This vulnerability allows authenticated users to enumerate project IDs and gain unauthorized access to sensitive project information and potentially modify projects that they do not own.

Impact:

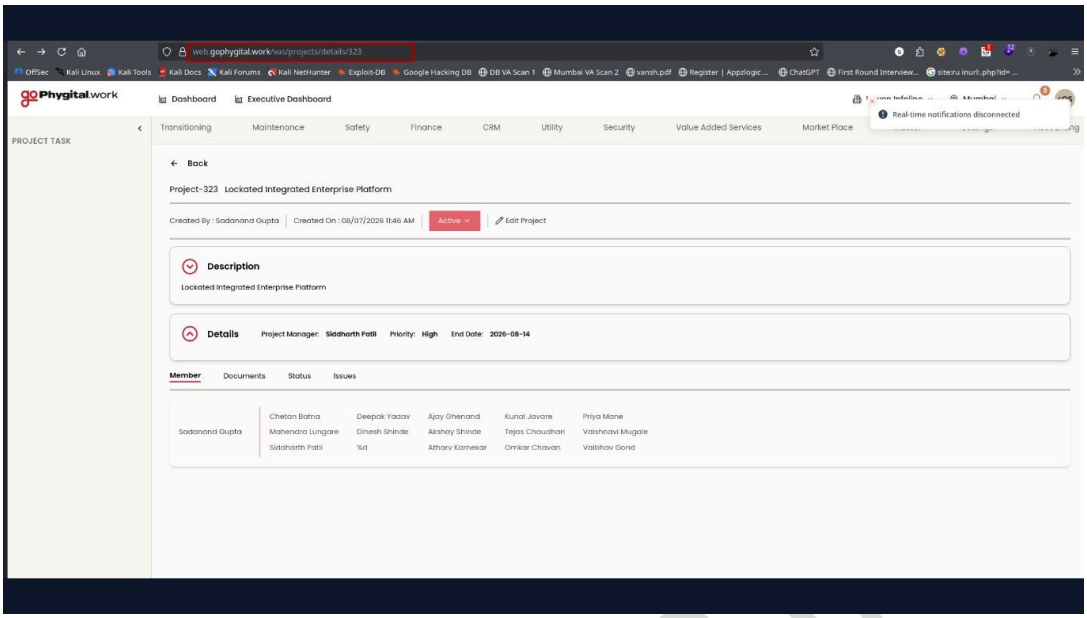
An attacker with a valid user account could:

- View confidential information belonging to other projects.
- Access project metadata such as project managers, members, priorities, milestones, tasks, and issues.
- Modify unauthorized project details through the **Edit Project** functionality.
- Enumerate project IDs to discover additional projects.
- Compromise the confidentiality and integrity of project data.

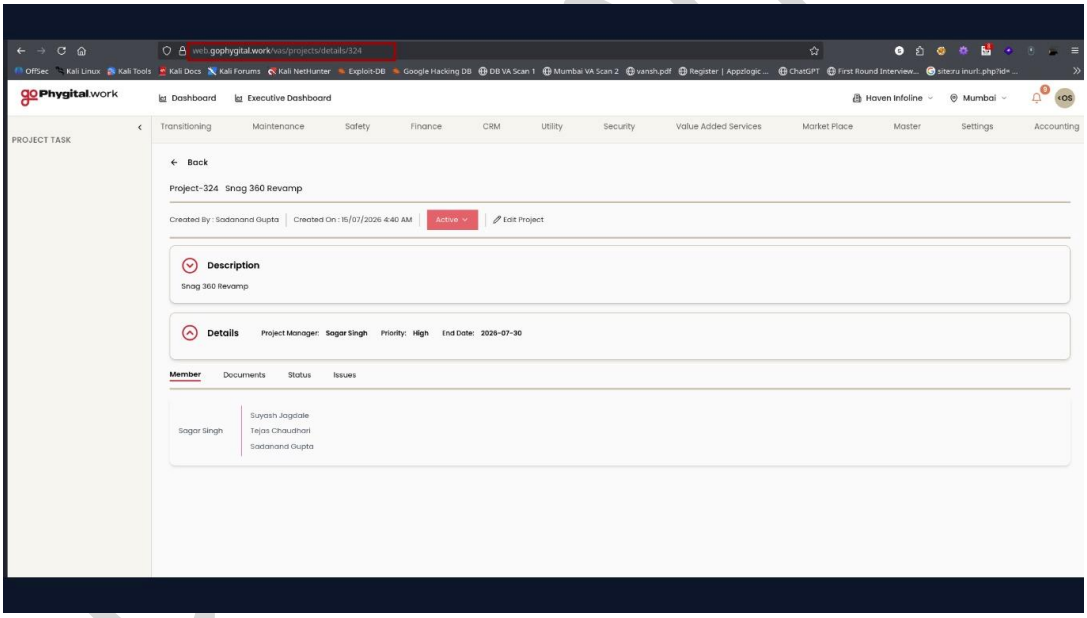
References to Evidences and Proof of Concept:

Step 1: Authenticate to the application using a valid user account.

Step 2: Navigate to: https://web.gophygital.work/vas/projects/details/323



Step 3: Modify the Project ID Change the project identifier in the URL: https://web.gophygital.work/vas/projects/details/324



Step 4: Observe the Response: The application returns the details of Project 324, including: • Project Name • Description • Project Manager • Members • Tasks • Milestones • Issues • Project Status

Step 5: Verify Unauthorized Modification: Edit Project

Step 6: Observe that the application permits editing of the unauthorized project instead of denying access with 403 Forbidden or another authorization error.

Recommendations:

- Implement **server-side authorization checks** for every request accessing or modifying project resources.

- Verify that the authenticated user is authorized to view or edit the requested project before returning data or performing any action.
- Do not rely on client-side controls or hidden UI elements to enforce permissions.
- Use role-based access control (RBAC) or attribute-based access control (ABAC) to restrict access to project resources.
- Log and monitor unauthorized object access attempts.
- Perform security testing for IDOR vulnerabilities across all endpoints that use user-controlled object identifiers.

5.8 Insecure Direct Object Reference (IDOR) – Unauthorized Access and Modification of Maintenance Tickets

Affected Asset: https://web.gophygital.work/

Affected Parameter: /maintenance/ticket/details/{ticket_id}

CWE-ID: CWE-639: Authorization Bypass Through User-Controlled Key

Severity: High

Description:

The application is vulnerable to an **Insecure Direct Object Reference (IDOR)** due to insufficient server-side authorization checks on maintenance ticket resources.

An authenticated user can modify the numeric ticket identifier in the URL to access maintenance tickets belonging to other users without proper authorization.

During testing, changing the ticket ID from **831634** to **831635** exposed another user's maintenance ticket, including confidential ticket information such as:

- Ticket description
- Ticket creator
- Assigned engineer
- Internal comments
- Customer comments
- Ticket history and audit logs
- Root cause analysis
- Corrective actions
- Additional notes
- Vendor information

Furthermore, the authenticated user was able to use the available **Edit** and **Comment** functionality on unauthorized tickets, allowing modification of records belonging to other users.

This demonstrates that the application performs authorization checks only on the client side (or not at all) and fails to verify ownership or permissions on the server before granting access to ticket resources.

Impact:

An authenticated attacker can:

- Access maintenance tickets belonging to other users.
- View confidential operational information.
- View internal comments and support logs.
- Access root cause analysis and corrective actions.
- Modify unauthorized tickets.

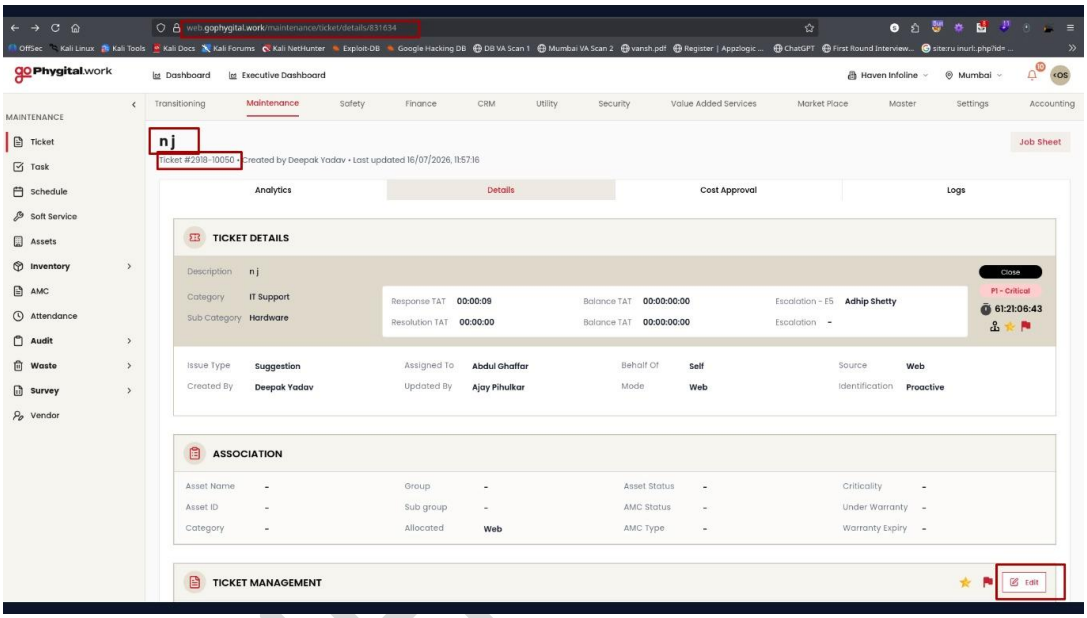
- Add internal or customer comments to other users' tickets.
- Tamper with ticket history and audit records.
- Enumerate ticket IDs to access additional tickets across the application.

Successful exploitation compromises both the **confidentiality** and **integrity** of the maintenance ticket management system.

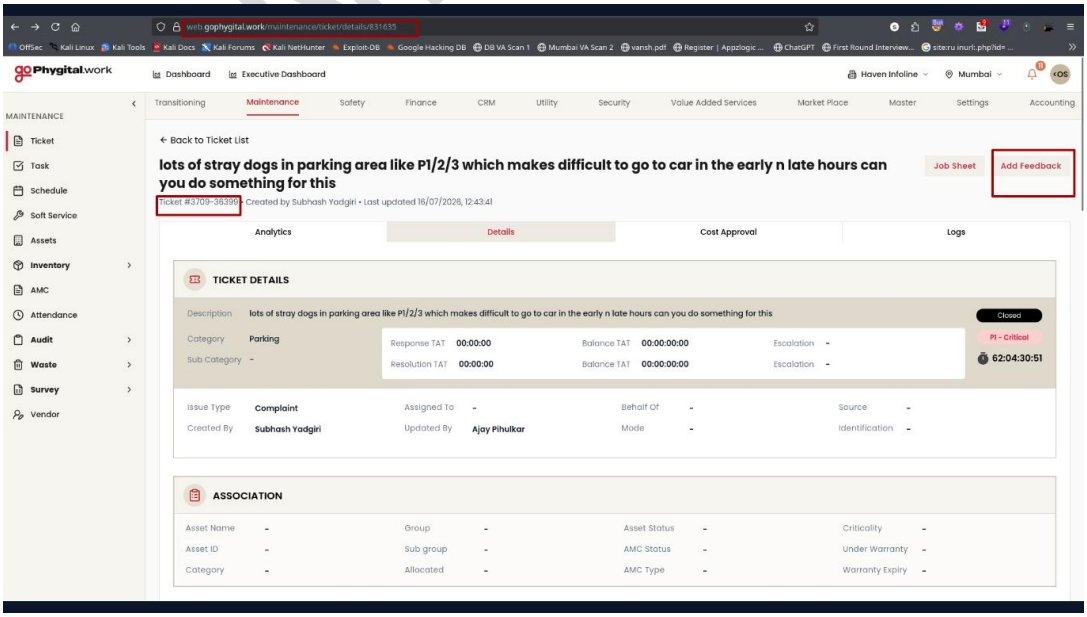
References to Evidences and Proof of Concept:

Step 1: Log in using a valid user account.

Step 2: Navigate to: https://web.gophygital.work/maintenance/ticket/details/831634



Step 3: Change the ticket identifier in the browser address bar: https://web.gophygital.work/maintenance/ticket/details/831635



Step 4: The application returns another user's maintenance ticket, exposing: • Ticket Details • Description • Created By • Assigned To • Ticket Status • Vendor Details • Internal Notes • Root Cause Analysis • Corrective Actions • Review Information • Comments • Audit Logs

Step 5: Using the available functionality, submit a comment or edit the ticket. Observe that the application accepts the request successfully and updates the ticket even though it belongs to another user. This confirms that authorization is not enforced on server-side operations.

Recommendations:

- Implement **server-side authorization checks** for every request involving ticket resources.
- Verify that the authenticated user is authorized to **view**, **comment on**, or **modify** the requested ticket before processing the request.
- Do not rely on ticket IDs supplied by the client as proof of authorization.
- Enforce Role-Based Access Control (RBAC) or Attribute-Based Access Control (ABAC) consistently across all maintenance ticket endpoints.
- Return **HTTP 403 Forbidden** when users attempt to access or modify tickets outside their authorization scope.
- Use indirect object references (e.g., UUIDs) where appropriate to make identifier enumeration more difficult.
- Log and monitor unauthorized access attempts to ticket resources.

5.9 Stored Cross-Site Scripting (Stored XSS) via Malicious PDF Upload

Affected Asset: https://web.gophygital.work/

Affected Parameter:

- /maintenance/service/edit/17700
- /safety/incident/add

CWE-ID: CWE-79: Improper Neutralization of Input During Web Page Generation (Cross-site Scripting)

Severity: High

Description:

The application is vulnerable to **Stored Cross-Site Scripting (Stored XSS)** through the service attachment upload functionality.

During testing, a PDF file containing embedded JavaScript (xss.pdf) was uploaded through the **Edit Service** page. The application accepted and stored the file without sanitization or content validation.

After upload, the file was accessible through the application's download endpoint. Opening the uploaded PDF via the application URL caused the embedded JavaScript payload to execute in the browser, confirming that the application serves active content without appropriate protections.

This allows malicious files to be stored on the server and executed when accessed by other authenticated users.

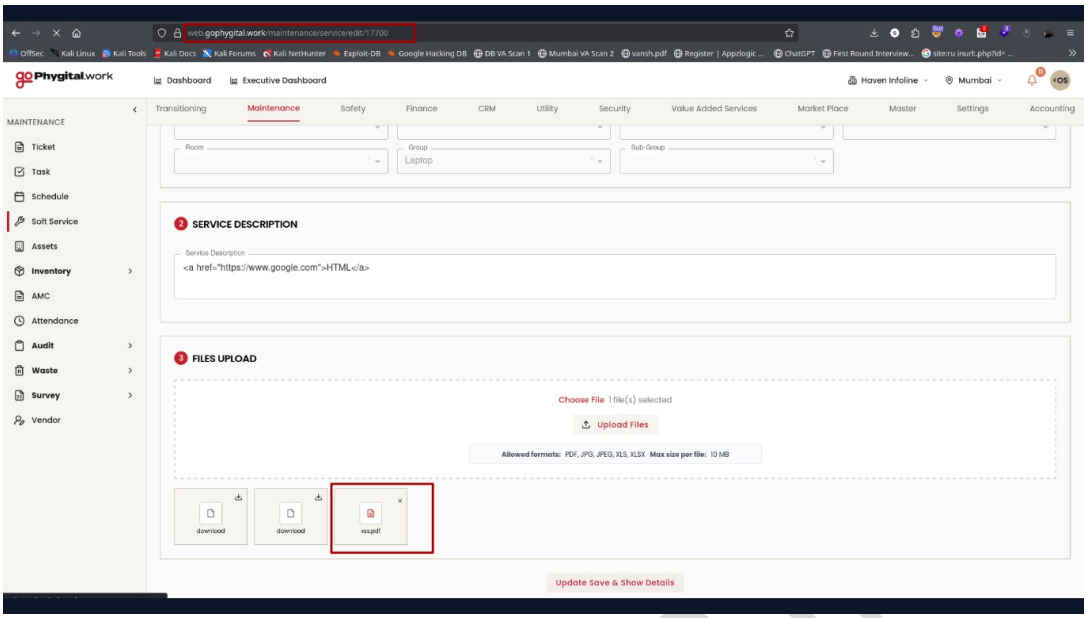
Impact:

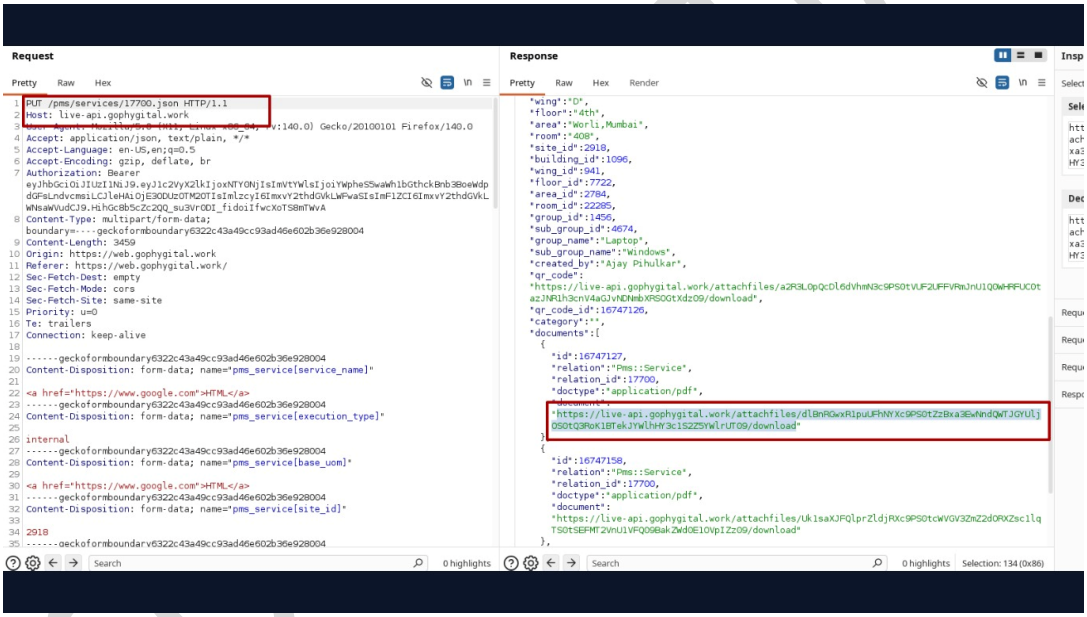
A successful attacker could:

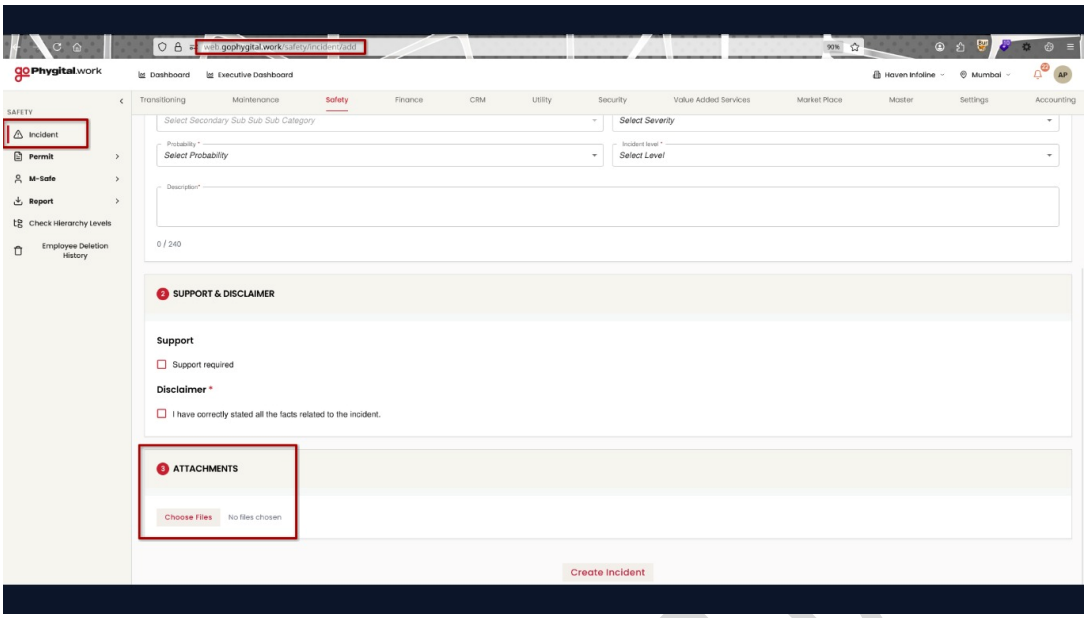
- Execute arbitrary JavaScript in a victim's browser.
- Steal session tokens (where not protected by HttpOnly).
- Perform actions on behalf of authenticated users.
- Redirect users to malicious websites.
- Modify displayed content.
- Conduct phishing attacks against application users.
- Compromise administrator accounts if they view the malicious attachment.

References to Evidences and Proof of Concept:

Step 1: Navigate to the Service Edit page. Open: https://web.gophygital.work/maintenance/service/edit/17700


web.gophygital.work/maintenance/service/edit/17700
Phygital.work
Dashboard
Executive Dashboard
Haven Infoline
Mumbai
Transitioning
Maintenance
Safety
Finance
CRM
Utility
Security
Value Added Services
Market Place
Master
Settings
Accounting
MAINTENANCE
Ticket
Task
Schedule
Soft Service
Assets
Inventory
AMC
Attendance
Audit
Waste
Survey
Vendor
Room
Group
Laptop
Sub-Group
2 SERVICE DESCRIPTION
Service Description
HTML
3 FILES UPLOAD
Choose File 1 file(s) selected
Upload Files
Allowed formats: PDF, JPG, JPEG, XLS, XLSX Max size per file: 10 MB
download
download
xss.pdf
Update Save & Show Details

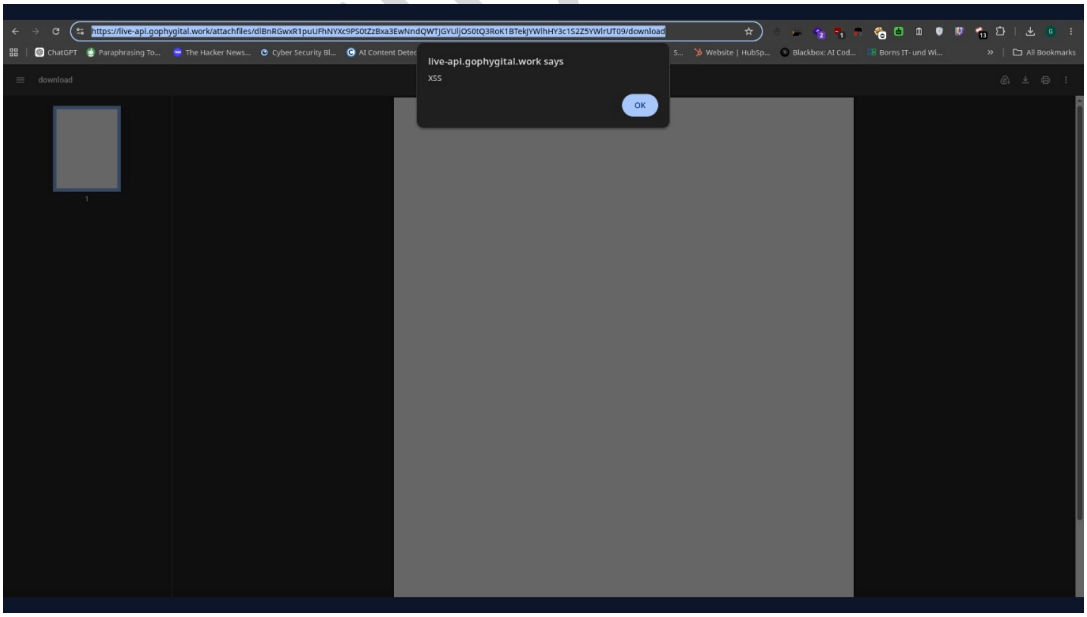

Request
Pretty
Raw
Hex
PUT /pms/services/17700.json HTTP/1.1
Host: live-api.gophygital.work
Accept: application/json, text/plain, */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: multipart/form-data;
Origin: https://web.gophygital.work
Referer: https://web.gophygital.work/
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-site
Priority: u=0
Te: trailers
Connection: keep-alive
Content-Disposition: form-data; name="pms_service[service_name]"
HTML
Content-Disposition: form-data; name="pms_service[execution_type]"
internal
Content-Disposition: form-data; name="pms_service[base_uom]"
HTML
Content-Disposition: form-data; name="pms_service[site_id]"
2918
Response
Pretty
Raw
Hex
Render
"wing":"D",
"floor":"4th",
"area":"Worli,Mumbai",
"room":"408",
"site_id":2918,
"building_id":1096,
"wing_id":941,
"floor_id":7722,
"area_id":2784,
"room_id":22285,
"group_id":1456,
"sub_group_id":4674,
"group_name":"Laptop",
"sub_group_name":"Windows",
"created_by":"Ajay Pihulkar",
"qr_code":
"category":"",
"documents":[
"id":16747127,
"relation":"Pms::Service",
"relation_id":17700,
"doctype":"application/pdf",
"https://live-api.gophygital.work/attachfiles/dlBnRGwxR1puUFhNYXc9PS0tZzBxa3EwNndQWTJGYUljOS0tQ3RoK1BTekJYWlhHY3c1S2Z5YWlrUT09/download"
"id":16747158,
"relation":"Pms::Service",
"relation_id":17700,
"doctype":"application/pdf",
"https://live-api.gophygital.work/attachfiles/Uk1saXJFQlprZldjRXc9PS0tcWVGV3ZmZ2dORXZsc1lqTS0tSEFMT2VnU1VFQ09BakZWdOE1OVpIZz09/download"
Search
0 highlights
Selection: 134 (0x86)

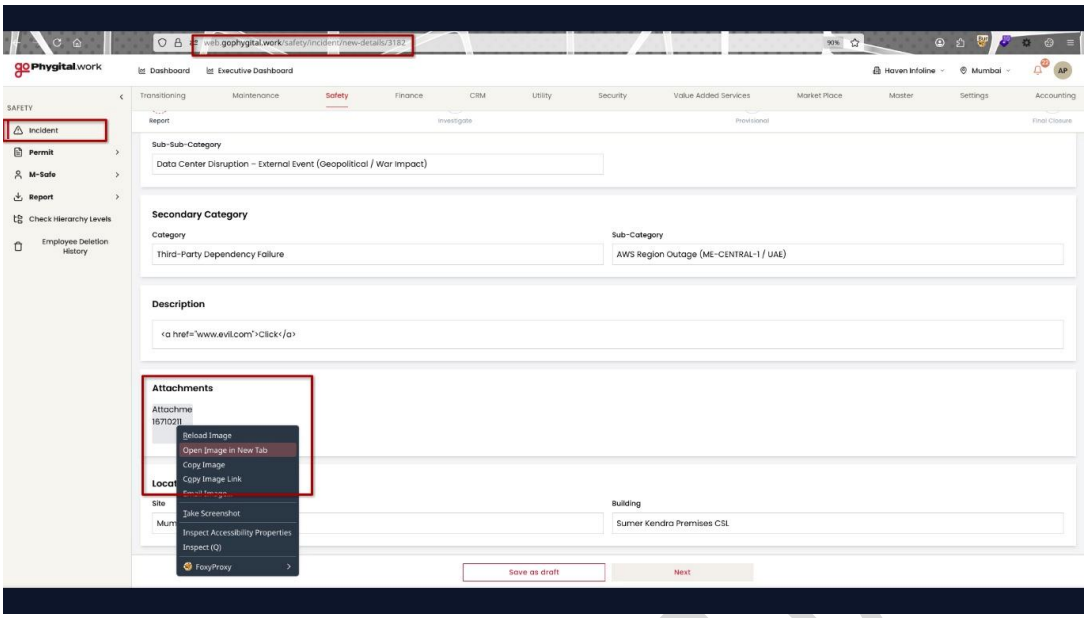


Step 2: Upload a PDF containing an embedded JavaScript payload (e.g., xss.pdf) using the Files Upload functionality.

Step 3: Submit the form and allow the file to be uploaded successfully.

Step 4: Intercept the response or inspect the uploaded attachment to obtain its download URL. Example: https://live-api.gophygital.work/attachfiles/dlBnRGwxR1puUFhNYXc9PS0tZzBxa3EwNndQWTJGYUljOS0tQ3RoK1BTekJYWlhHY3c1S2Z5YWlrUT09/download





Step 5: Open the attachment using the application-generated URL.

Step 6: The embedded JavaScript executes automatically when the file is rendered by the browser through the application, demonstrating a Stored Cross-Site Scripting (Stored XSS) vulnerability.

Recommendations:

- Disallow JavaScript and other active content in uploaded PDF documents.
- Sanitize uploaded files before making them available to users.
  - Serve uploaded files with appropriate security headers, including:
  - Content-Disposition: attachment
  - X-Content-Type-Options: nosniff
- Avoid inline rendering of untrusted documents; force downloads where appropriate.
- Validate file types based on content (magic bytes), not just file extensions.
- Consider processing uploaded PDFs through a sanitization pipeline to remove embedded scripts and active objects.
- Implement a strong Content Security Policy (CSP) to reduce the impact of XSS.

5.10 Stored Cross-Site Scripting (Stored XSS) via Malicious PDF Attachment Upload in Ticket Comments

Affected Asset: https://web.gophygital.work/

Affected Parameter: /maintenance/ticket/details/{ticket_id}

CWE-ID: CWE-79: Improper Neutralization of Input During Web Page Generation (Cross-Site Scripting)

Severity: High

Description:

The application allows users to upload attachments through the ticket comment functionality. During testing, a malicious PDF file (xss.pdf) containing an XSS payload was uploaded as a comment attachment.

After the upload, the application generated a publicly accessible attachment URL. When the uploaded PDF was accessed through the application-generated URL, the embedded JavaScript payload executed successfully, confirming a **Stored Cross-Site Scripting (Stored XSS)** vulnerability.

The application stores and serves user-supplied files without properly sanitizing active content or enforcing secure file handling controls. As a result, malicious content can be stored on the server and executed when other users access the attachment.

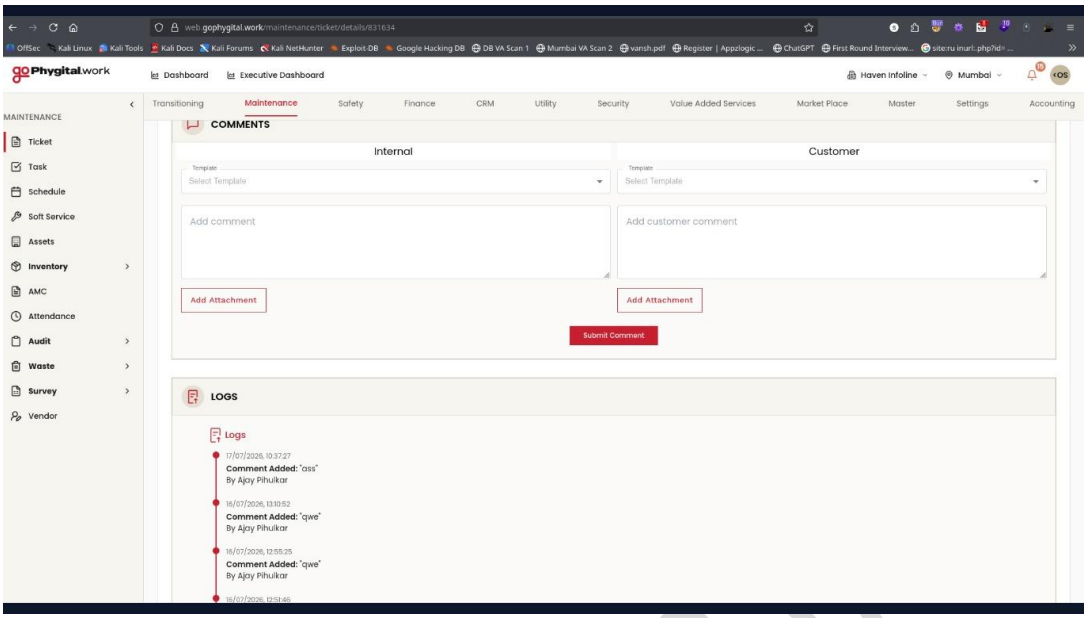
Impact:

An attacker could:

- Execute arbitrary JavaScript in the browser of users viewing the attachment.
- Hijack authenticated user sessions.
- Perform actions on behalf of victims.
- Modify page content displayed to users.
- Redirect users to phishing or malicious websites.
- Target privileged users such as support staff or administrators.
- Compromise the confidentiality and integrity of the application.

References to Evidences and Proof of Concept:

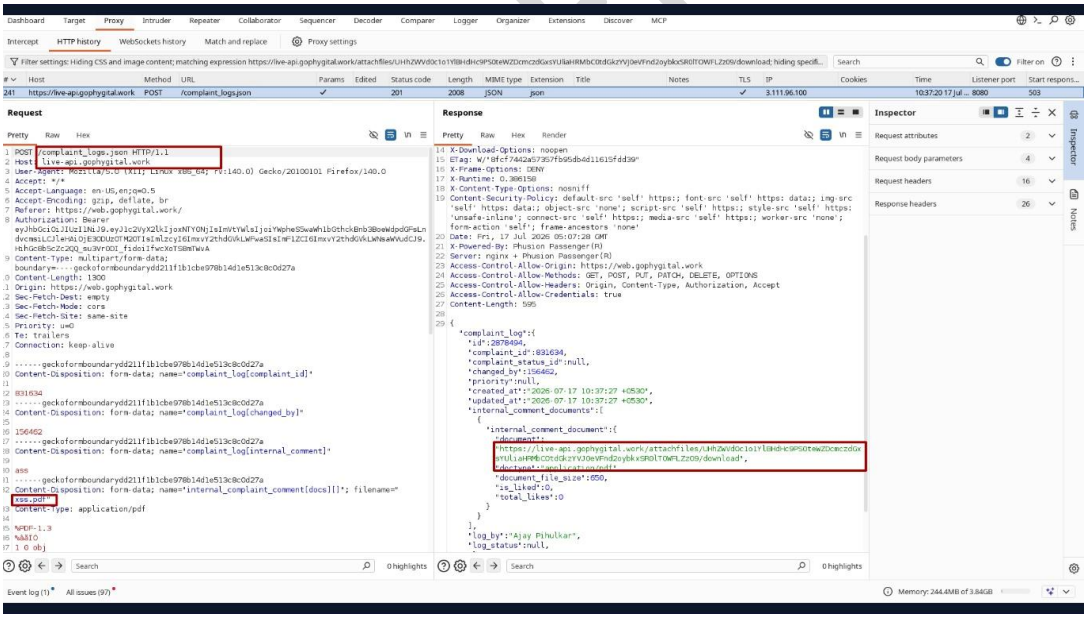
Step 1: Navigate to: https://web.gophygital.work/maintenance/ticket/details/831634



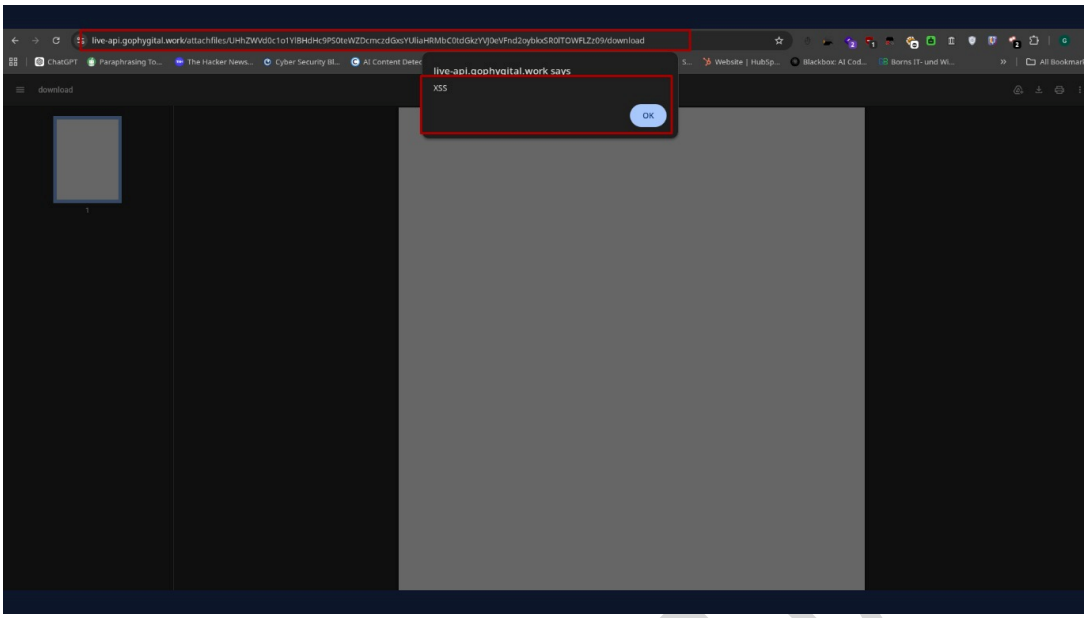
Step 2: Use the comment attachment feature and upload a PDF file (xss.pdf) containing an embedded JavaScript payload.

Step 3: Add the attachment and save the comment.

Step 4: Using Burp Suite, identify the generated attachment URL returned by the application.



Step 5: Open the attachment URL in a browser: https://live-api.gophygital.work/attachfiles/UHhZWVd0c1o1YlBHdHc9PS0teWZDcmczdGxsYUliaHRMbC0tdGkzYVJ0eVFnd2oybkxSR0lTOWFLZz09/download



Step 6: The embedded payload executes when the file is rendered, confirming that malicious content is stored and served without adequate sanitization.

Recommendations:

- Disallow JavaScript and active content within uploaded PDF files.
- Sanitize uploaded documents before storage.
- Force uploaded files to be downloaded rather than rendered inline by setting: Content-Disposition: attachment
- Implement strict file validation using MIME type and file content inspection.
- Use a secure document processing solution to remove embedded scripts and active objects.
- Configure appropriate security headers such as: X-Content-Type-Options: nosniff Content-Security-Policy
- Restrict the rendering of untrusted user-uploaded content within the application context.

5.11 Insecure Direct Object Reference (IDOR) Allows Unauthorized Access to Inventory Details

Affected Asset: https://web.gophygital.work/maintenance/inventory/details/<ID>

Affected Parameter: Inventory Master

CWE-ID:

- CWE-639 – Authorization Bypass Through User-Controlled Key
- CWE-284 – Improper Access Control

Severity: High

Description:

The application is vulnerable to an Insecure Direct Object Reference (IDOR) in the inventory details endpoint. The endpoint identifies inventory records using a sequential numeric identifier in the URL *(e.g., /maintenance/inventory/details/{id})*. By modifying this identifier to another valid value, an authenticated user can access inventory records belonging to other users or organizations without proper authorization checks.

The application fails to validate whether the requesting user is authorized to access the requested inventory record, resulting in unauthorized disclosure of sensitive information.

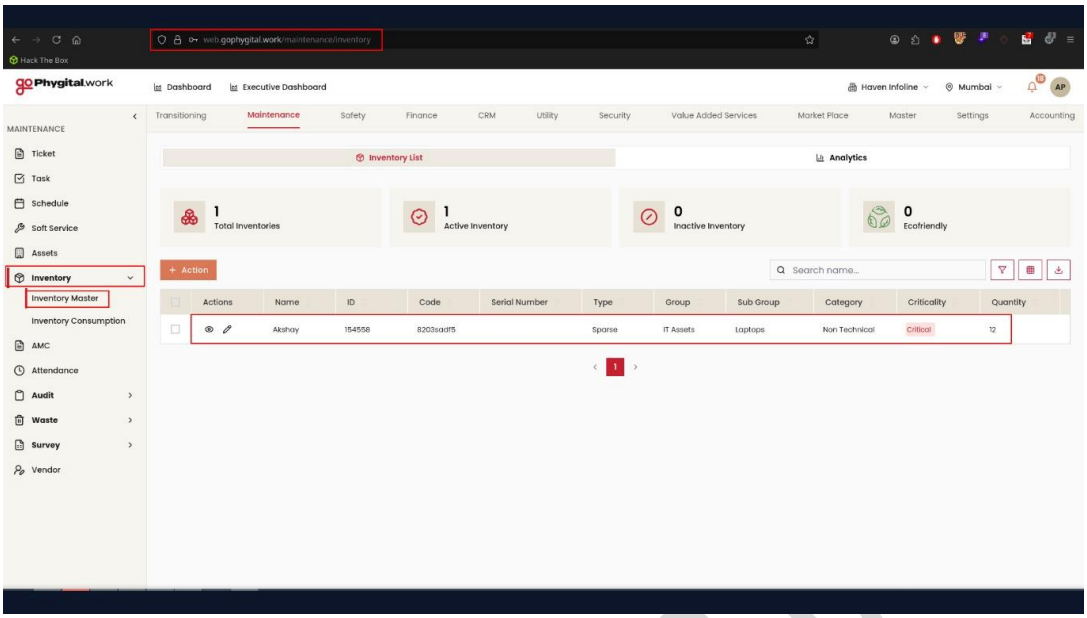
Impact:

- Unauthorized users can access inventory records belonging to other users, departments, or organizations.
- Exposure of sensitive inventory information, including item details, asset information, stock levels, pricing, supplier information, and internal identifiers.
- Attackers can enumerate valid inventory IDs and systematically collect large volumes of business-sensitive data.
- Confidential business information may be disclosed, resulting in loss of data confidentiality.
- The exposed information can be used to facilitate further attacks, such as targeted phishing, social engineering, or privilege escalation.
- Knowledge of inventory assets and infrastructure may assist attackers in planning more sophisticated attacks against the organization.
- If additional functions (edit, delete, approve, or download) use the same authorization logic, the vulnerability could lead to unauthorized modification or deletion of inventory records.

References to Evidences and Proof of Concept:

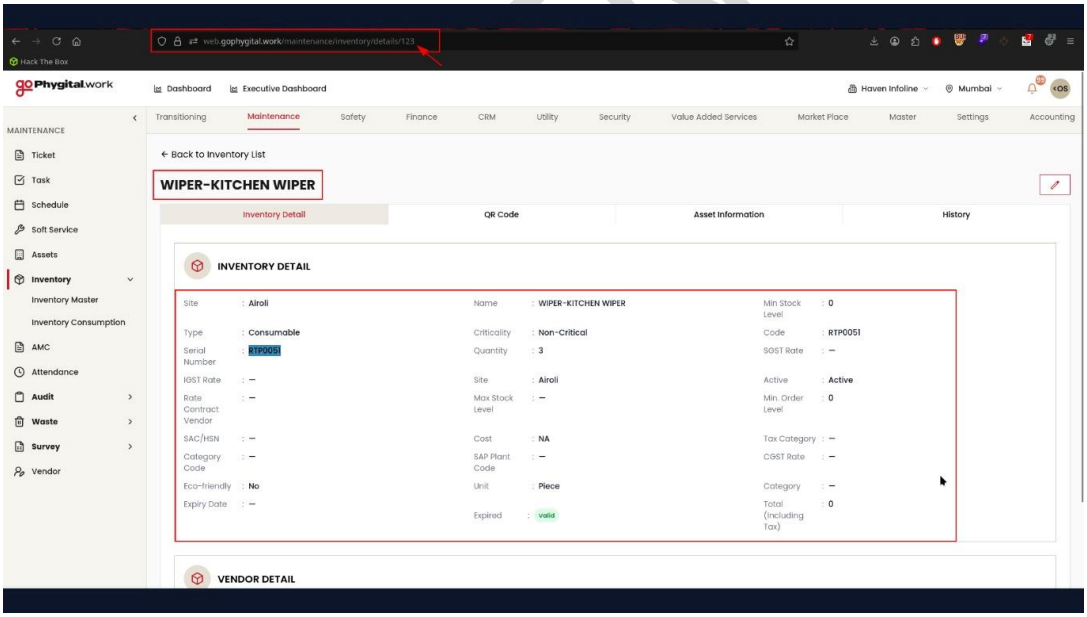
Step 1: Log in to the application using a valid low-privileged user account.

Step 2: Navigate to any inventory details page, click on add button and fill all the details.

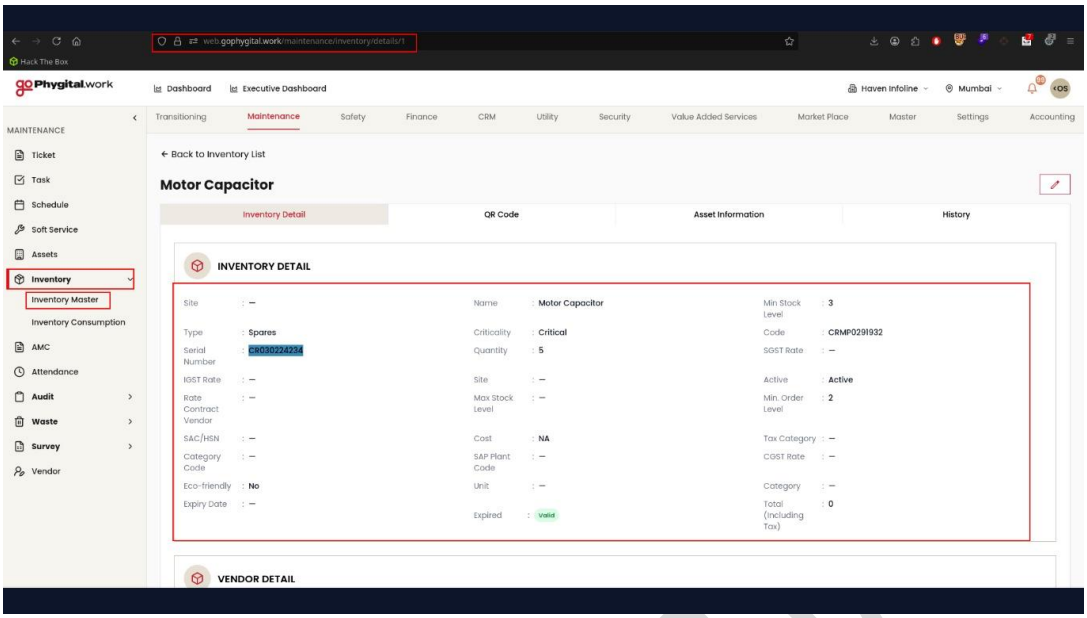


Step 3: Now click on view request and observe the parameter in URL: https://web.gophygital.work/maintenance/inventory/details/<ID>

Step 4: Modify the numeric identifier to another valid inventory ID.



Step 5: Observe that the application returns another user's inventory details instead of denying access with a 403 Forbidden or 404 Not Found response.



Step 6: Here we are able to view another inventories details.

Recommendations:

- Enforce server-side authorization checks for every request to ensure users can access only resources they own or are explicitly permitted to view.
- Do not rely on client-side controls or hidden parameters for authorization.
- Validate object ownership before returning any inventory information.
- Return 403 Forbidden or 404 Not Found when an unauthorized resource is requested.
- Implement centralized access control mechanisms to consistently enforce authorization across all endpoints.
- Perform regular access control testing to identify and remediate similar IDOR vulnerabilities throughout the application.

5.12 Clickjacking Vulnerability on Login Page

Affected Asset: https://web.gophygital.work/login

Affected Parameter: Web Application Login Interface

CWE-ID: CWE-1021: Improper Restriction of Rendered UI Layers or Frames

Severity: Medium

Description:

The application's login page can be embedded within an external website using an HTML because appropriate anti-clickjacking protections are not implemented. During testing, it was observed that the login page was successfully loaded inside a malicious webpage frame. User interactions with the framed login page remained functional, allowing authentication requests to be processed successfully. A victim could therefore be tricked into interacting with the embedded page without realizing they are performing actions within the legitimate application.

Impact:

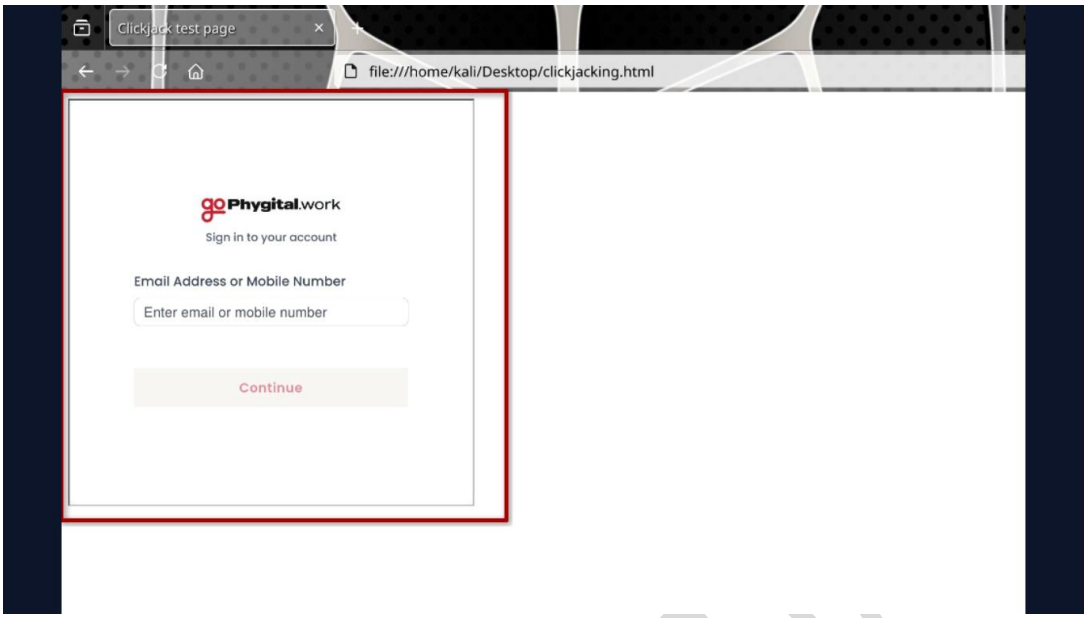
Attackers can embed the application within a malicious website. Users may be deceived into entering valid credentials into a framed application. Sensitive actions can potentially be performed without the user's awareness. Increased risk of UI redressing attacks and unauthorized actions through social engineering. May facilitate phishing and credential theft scenarios.

References to Evidences and Proof of Concept:

Step 1: Navigate to the application login page.

Step 2: Create an HTML page containing the following code: <html> <head> <title>Clickjack test page</title> </head> <body> <iframe src="https://web.gophygital.work/login" width="500" height="500"></iframe> </body> </html>

Step 3: Open the HTML file in a browser and observe that the login page loads successfully within the iframe.



Step 4: Enter valid credentials into the framed login page and submit the login request.

Step 5: Observe that authentication is completed successfully and the user gains access to the application, confirming that framing restrictions are not enforced.

Recommendations:

Implement the X-Frame-Options response header with one of the following values: ◦ DENY ◦ SAMEORIGIN Implement a Content Security Policy (CSP) using the frame-ancestors directive. Ensure all sensitive pages, including login and authenticated pages, are protected against framing. Conduct security testing to verify that the application cannot be loaded within unauthorized frames or iframes. Consider implementing additional UI redressing protections where appropriate.

5.13 Insecure Direct Object Reference (IDOR) in Incident Details Page

Affected Asset: https://web.gophygital.work/safety/incident/new-details/3181

Affected Parameter: IncidentID

CWE-ID: CWE-639: Authorization Bypass Through User-Controlled Key

Severity: Medium

Description:

The application does not properly enforce authorization controls when accessing incident records through the Incident Details page. During testing, it was observed that modifying the numeric incident identifier within the URL allowed access to incident records other than those intended for the authenticated user. The application returned incident details without validating whether the requesting user had permission to access the specified record. This behavior enables unauthorized users to enumerate and access internal incident records by manipulating the object identifier in the URL.

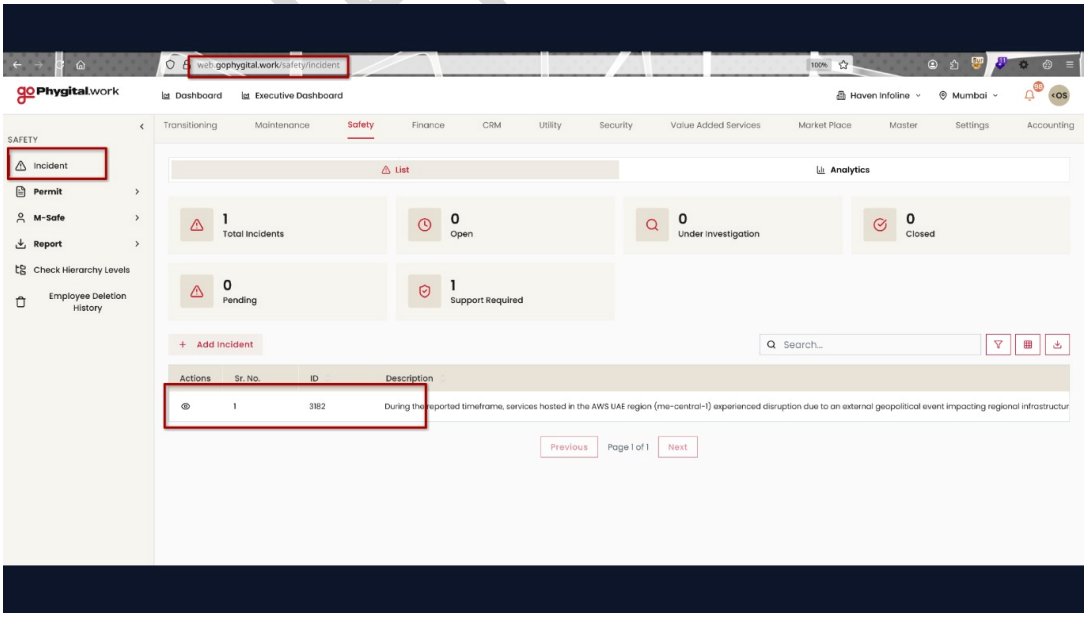
Impact:

Unauthorized access to internal incident records. Exposure of employee names and operational information. Disclosure of site and location details. Enumeration of valid incident identifiers. Potential leakage of internal organizational information. Violation of access control requirements.

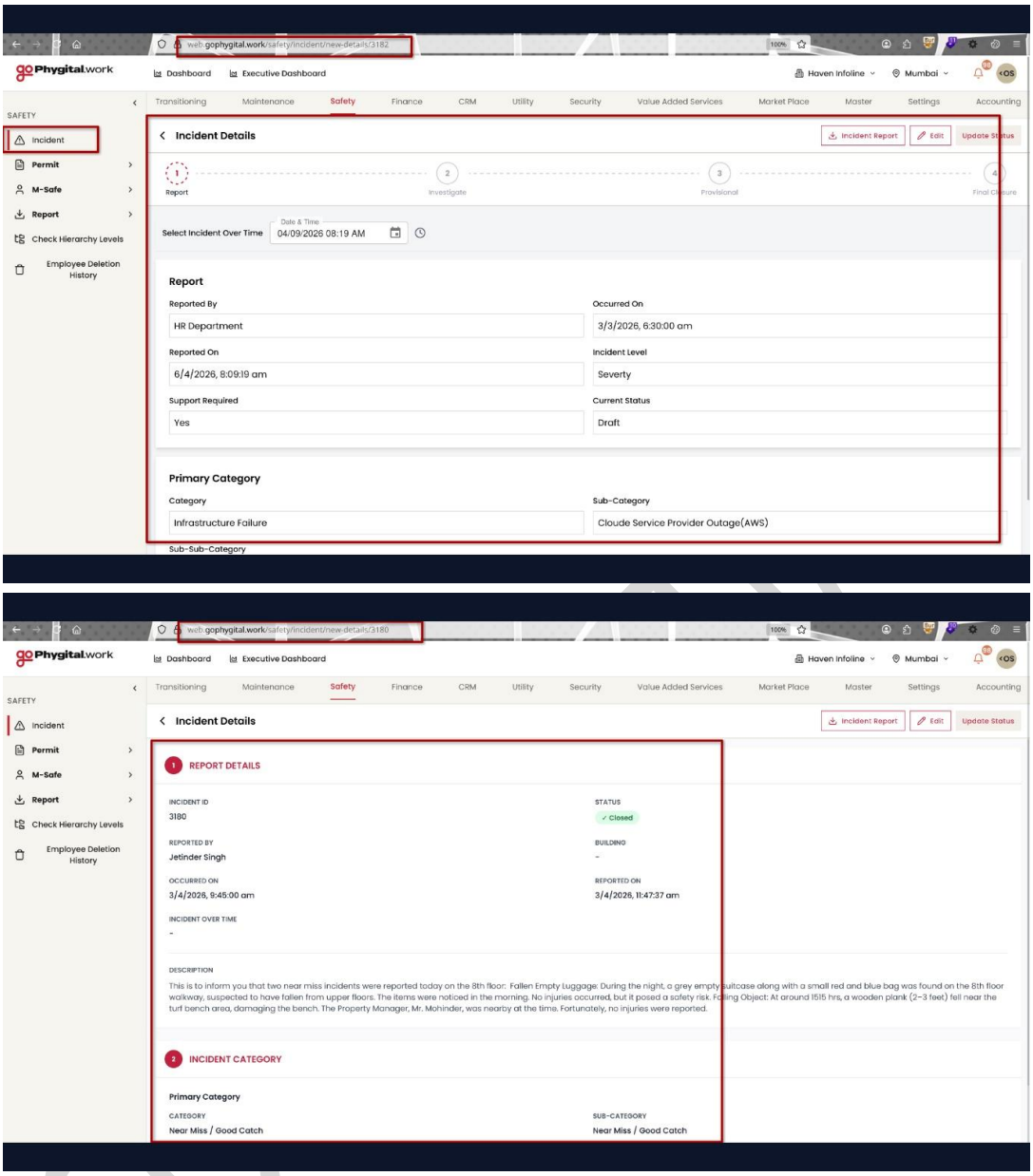
References to Evidences and Proof of Concept:

Step 1: Authenticate to the application with a valid user account.

Step 2: Browse to the incident record.



Step 3: Modify the Incident ID within the URL.



Step 4: Observe that the application displays details belonging to a different incident report.

Step 5: Repeat the process with additional identifiers and verify that multiple incident records can be accessed without authorization checks.

Recommendations:

Implement server-side authorization validation for every incident record request. Ensure users can access only records they own or are explicitly permitted to view. Apply role-based access control (RBAC) for incident management functions. Avoid exposing predictable sequential identifiers where possible. Log and monitor unauthorized access attempts. Conduct a full access-control review across similar endpoints.

5.14 Session Not Invalidated After Logout

Affected Asset: https://web.gophygital.work/dashboard

Affected Parameter: JWT Access Token

CWE-ID: CWE-613: Insufficient Session Expiration

Severity: Medium

Description:

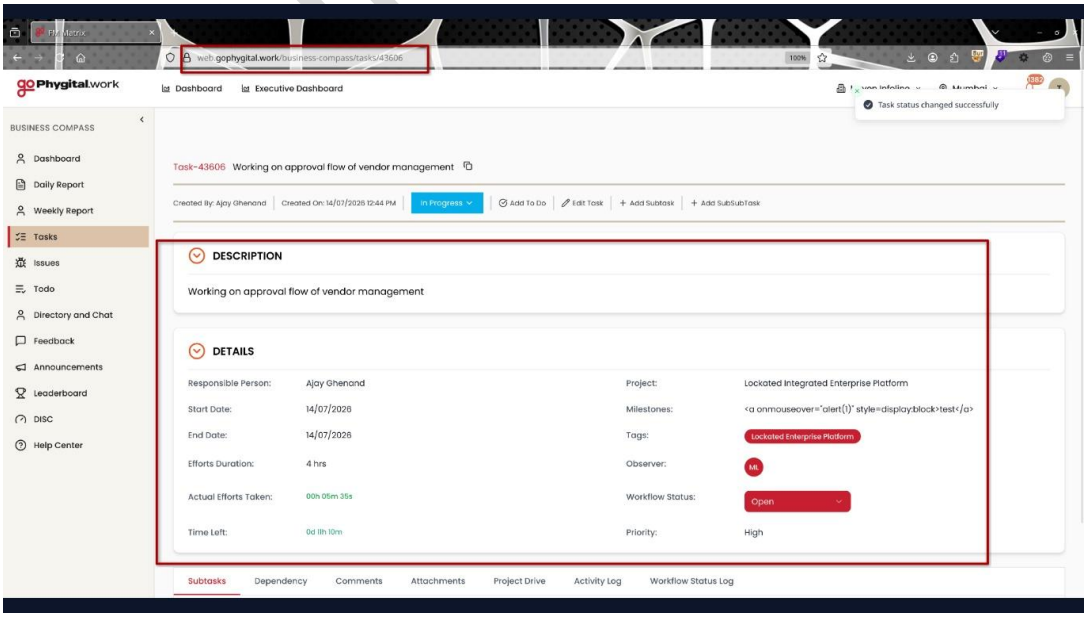
The application does not properly invalidate active user sessions upon logout. During testing, it was observed that a previously issued JWT access token remained valid and continued to provide access to authenticated resources even after the user had successfully logged out. The logout functionality appears to remove the session from the client-side interface; however, the server continues to accept the previously issued authentication token until its expiration time is reached. As a result, any party possessing a valid token can continue accessing protected resources despite the user's logout action. This behavior weakens session management controls and increases the risk of unauthorized access if authentication tokens are disclosed, intercepted, or obtained by an attacker.

Impact:

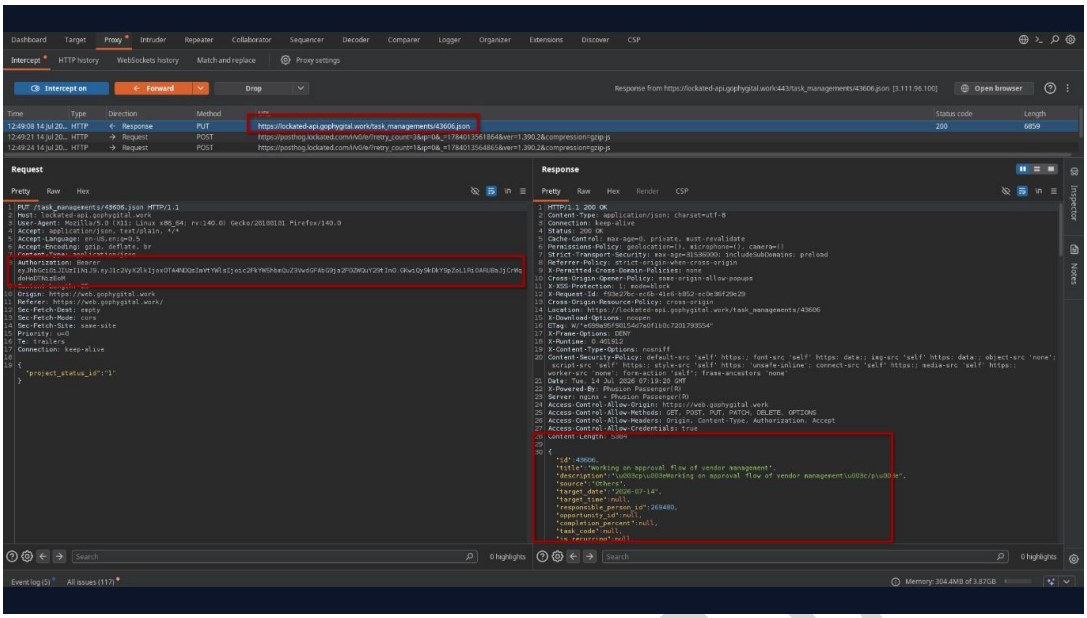
Users remain authenticated after logging out. Stolen or leaked tokens can continue to be used until expiration. Increased risk of session hijacking and unauthorized account access. Logout functionality does not effectively terminate active sessions. Potential exposure of sensitive application data and functionality.

References to Evidences and Proof of Concept:

Step 1: Log in to the application using valid user credentials.

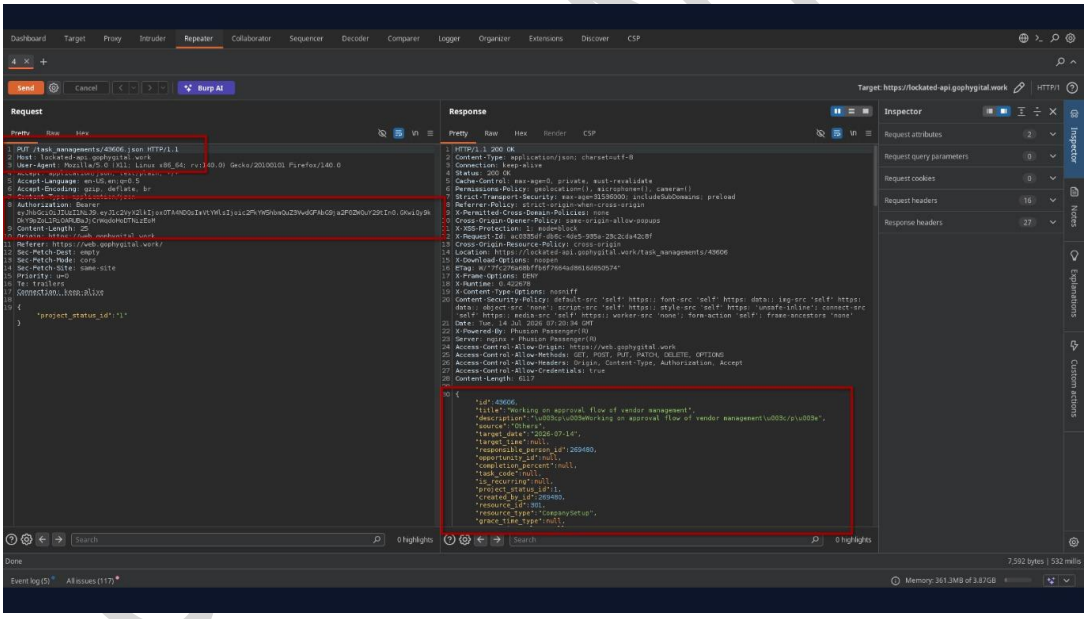


Step 2: Capture the issued JWT access token from an authenticated request.



Step 3: Log out of the application using the provided logout functionality.

Step 4: Replay an authenticated request using the previously captured JWT access token.



Step 5: Observe that the application returns a valid response (200 OK) and grants access to authenticated resources despite the user having logged out.

Step 6: Verify that the token remains valid until the expiration time (exp) embedded within the JWT is reached.

Recommendations:

Invalidate active access tokens upon logout. Implement server-side token revocation or token blacklisting mechanisms. Use short-lived access tokens and secure refresh token rotation. Ensure logout terminates all active authenticated sessions associated with the token. Monitor and log the reuse of revoked or expired tokens. Avoid relying solely on client-side token deletion for session termination.

5.15 Internal IP Address Disclosure

Affected Asset: https://web.gophygital.work/

Affected Parameter: HTTP Response

CWE-ID: CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

Severity: Medium

Description:

The target application discloses the internal private IP address (172.31.16.75) within its HTTP response, exposing information about the internal network infrastructure. Further verification confirmed that the disclosed IP belongs to a live internal host exposing a DNS service (dnsmasq). This information can assist an attacker in reconnaissance and mapping the internal environment for subsequent attacks.

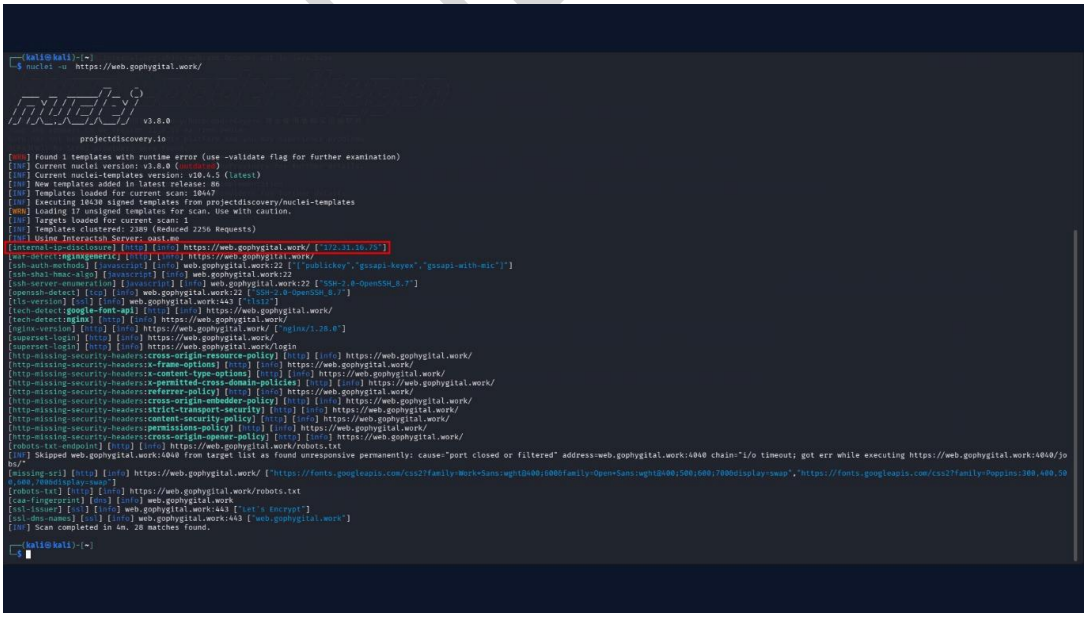
Impact:

Exposure of internal network addressing. Assists attackers in internal infrastructure reconnaissance. Reveals the presence of internal services and network topology. Increases the likelihood of successful targeted attacks after initial compromise.

References to Evidences and Proof of Concept:

Step 1: Access the application (https://web.gophygital.work) and inspect the HTTP response using Nuclei.

Step 2: Observe that the application discloses the internal IP address 172.31.16.75 in the response.



Step 3: Verify the disclosed IP by scanning it using Nmap and observe that it is an active internal host exposing TCP port 53 running the dnsmasq service.

Recommendations:

Remove internal IP addresses from all HTTP responses, headers, and error messages. Configure reverse proxies and web servers to prevent leakage of internal infrastructure information. Replace private IP addresses with public hostnames where necessary. Perform a review of the application and infrastructure to identify and remediate similar information disclosure issues.

5.16 Missing HTTP Security Headers

Affected Asset: https://web.gophygital.work

Affected Parameter: HTTP Response Headers

CWE-ID: CWE-693 – Protection Mechanism Failure

Severity: Medium

Description:

The target application does not include several recommended HTTP security headers in its HTTP responses. These headers provide browser-side security protections against attacks such as Cross-Site Scripting (XSS), Clickjacking, MIME-type sniffing, information leakage, and unauthorized cross-origin resource sharing. Manual verification confirmed that the following security headers are absent: Content-Security-Policy (CSP) X-Content-Type-Options Strict-Transport-Security (HSTS) X-Frame-Options Referrer-Policy Permissions-Policy Cross-Origin-Embedder-Policy (COEP) Cross-Origin-Opener-Policy (COOP) Cross-Origin-Resource-Policy (CORP) X-Permitted-Cross-Domain-Policies

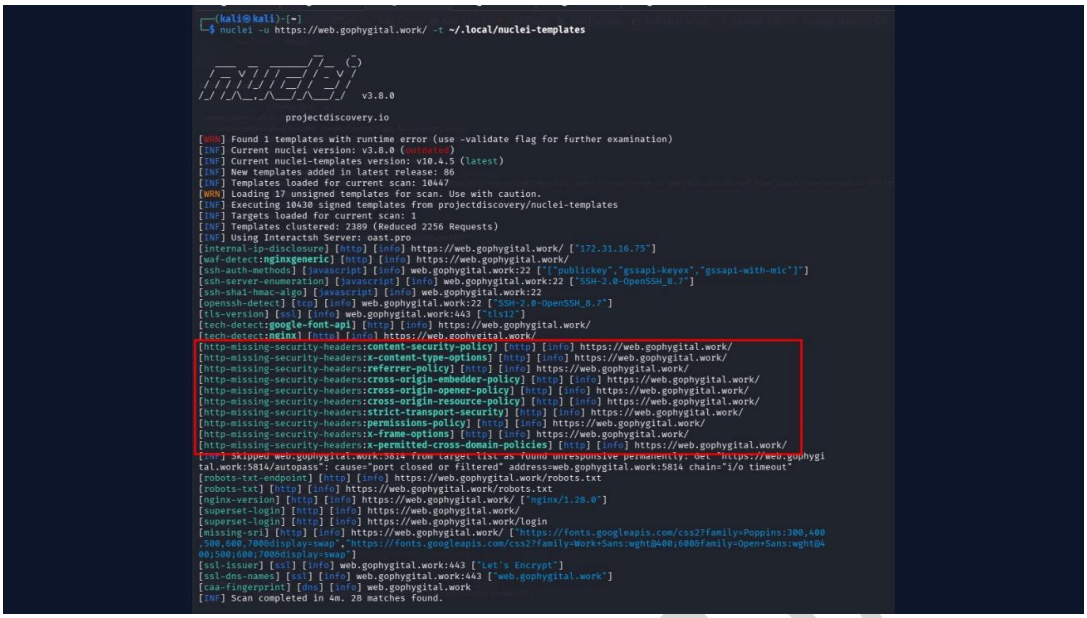
Impact:

Increased exposure to browser-based attacks such as Cross-Site Scripting (XSS) and Clickjacking. Browsers may perform MIME-type sniffing, potentially leading to unintended content execution. Referrer information may be disclosed to third-party websites. Lack of HSTS increases the risk of protocol downgrade attacks. Missing cross-origin security headers reduce browser-enforced isolation protections.

References to Evidences and Proof of Concept:

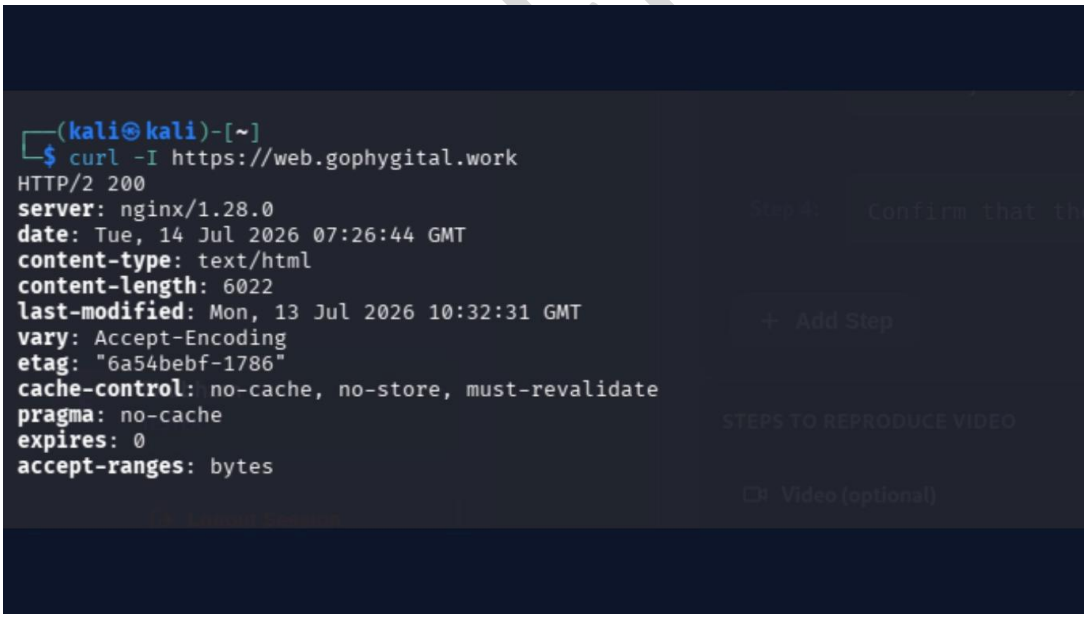
Step 1: nuclei -u https://web.gophygital.work/ -t ~/.local/nuclei-templates

Step 2: Observe that Nuclei reports the absence of the following HTTP security headers: Content-Security-Policy X-Content-Type-Options Referrer-Policy Cross-Origin-Embedder-Policy Cross-Origin-Opener-Policy Cross-Origin-Resource-Policy Strict-Transport-Security Permissions-Policy X-Frame-Options X-Permitted-Cross-Domain-Policies



Step 3: Manually verify the finding by inspecting the HTTP response headers: curl -k -I https://web.gophygital.work

Step 4: Confirm that the HTTP response does not include the reported security headers, validating the automated scan results.



Recommendations:

Configure the web server to include all recommended HTTP security headers. Implement a restrictive Content-Security-Policy (CSP) appropriate for the application. Enable Strict-Transport-Security (HSTS) with an appropriate max-age and includeSubDomains directive. Set X-Content-Type-Options: nosniff to prevent MIME-type sniffing. Configure X-Frame-Options as DENY or SAMEORIGIN to mitigate Clickjacking. Configure Referrer-Policy to limit the amount of referrer information shared with external websites. Implement Permissions-Policy to restrict access to browser features. Configure Cross-Origin-Embedder-Policy (COEP), Cross-Origin-

Opener-Policy (COOP), and Cross-Origin-Resource-Policy (CORP) based on application requirements. Set X-Permitted-Cross-Domain-Policies: none unless cross-domain policies are explicitly required.

5.17 Web Server Version Information Disclosure

Affected Asset: https://web.gophygital.work

Affected Parameter: HTTP Response Header (Server)

CWE-ID: CWE-200 – Exposure of Sensitive Information to an Unauthorized Actor

Severity: Medium

Description:

The target application discloses the web server software and its exact version through the Server HTTP response header. During testing, the server returned nginx/1.28.0, revealing the underlying web server and version information. Exposing the exact server version provides attackers with information that can be used during reconnaissance to identify known vulnerabilities, exploit publicly disclosed CVEs, or tailor attacks against the specific server version.

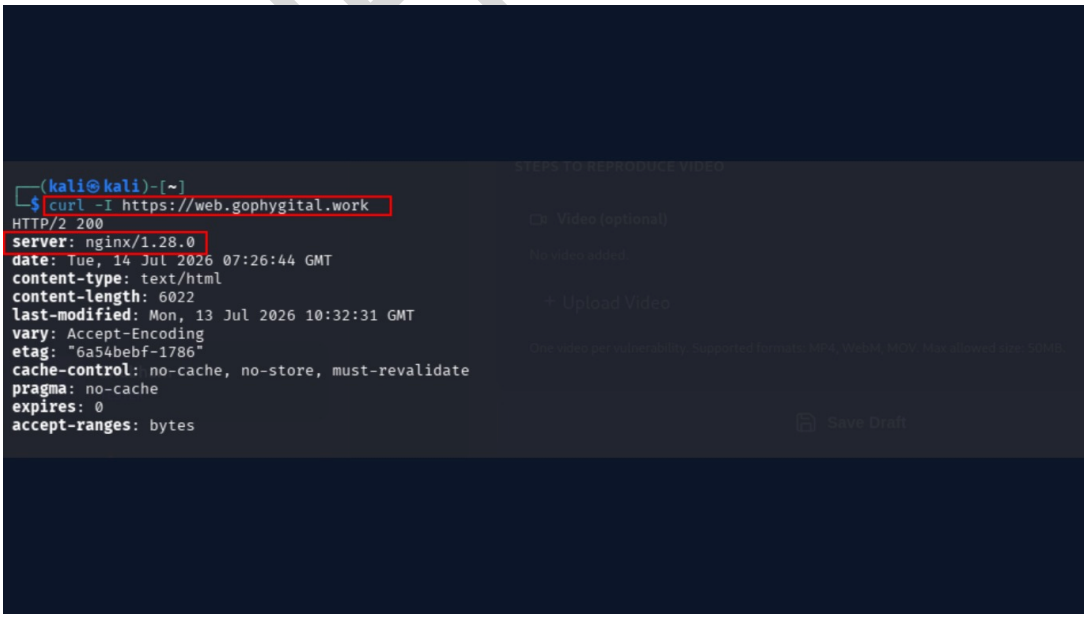
Impact:

Reveals the underlying web server software and version. Assists attackers during reconnaissance and fingerprinting. May facilitate identification of publicly known vulnerabilities affecting the disclosed server version. Increases the effectiveness of targeted attacks against the server.

References to Evidences and Proof of Concept:

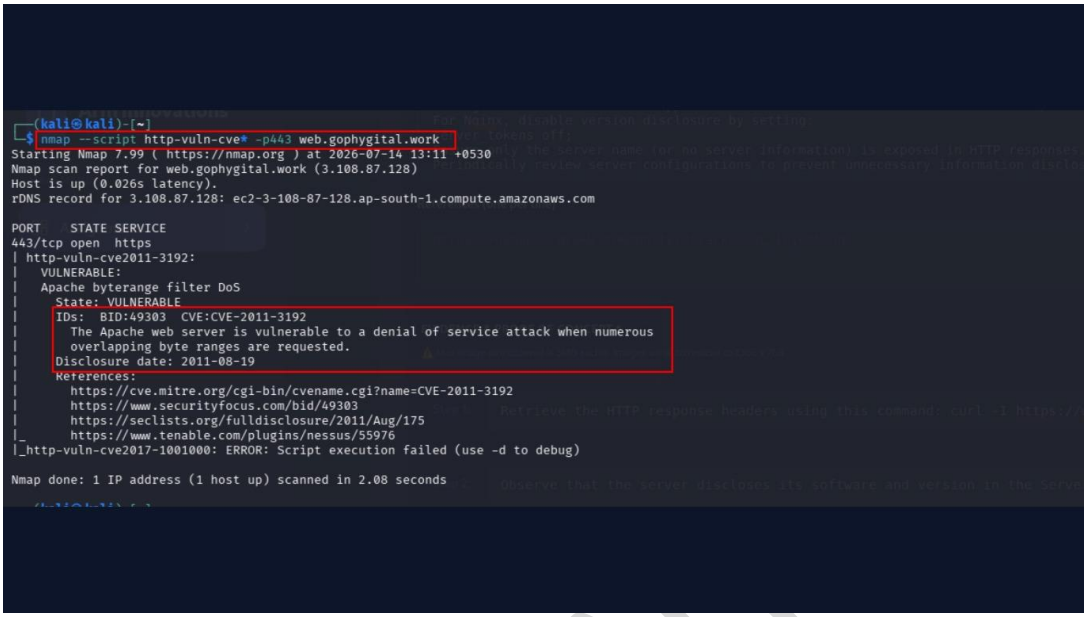
Step 1: Retrieve the HTTP response headers using this command: curl -I https://web.gophygital.work

Step 2: Observe that the server discloses its software and version in the Server header. HTTP/2 200 OK Server: nginx/1.28.0



Step 3: Perform additional server fingerprinting using Nmap. nmap --script http-vuln-cve* -p443 web.gophygital.work

Step 4: Observe that the automated NSE script flags a potential vulnerability associated with server fingerprinting.



Recommendations:

Configure the web server to suppress the version information in the Server response header. For Nginx, disable version disclosure by setting: server_tokens off; Ensure only the server name (or no server information) is exposed in HTTP responses. Periodically review server configurations to prevent unnecessary information disclosure.

5.18 SSH Server Supports Weak Message Authentication Code (MAC) Algorithms (SHA-1)

Affected Asset: https://web.gophygital.work/

Affected Parameter: N/A

CWE-ID: CWE-327: Use of a Broken or Risky Cryptographic Algorithm

Severity: Medium

Description:

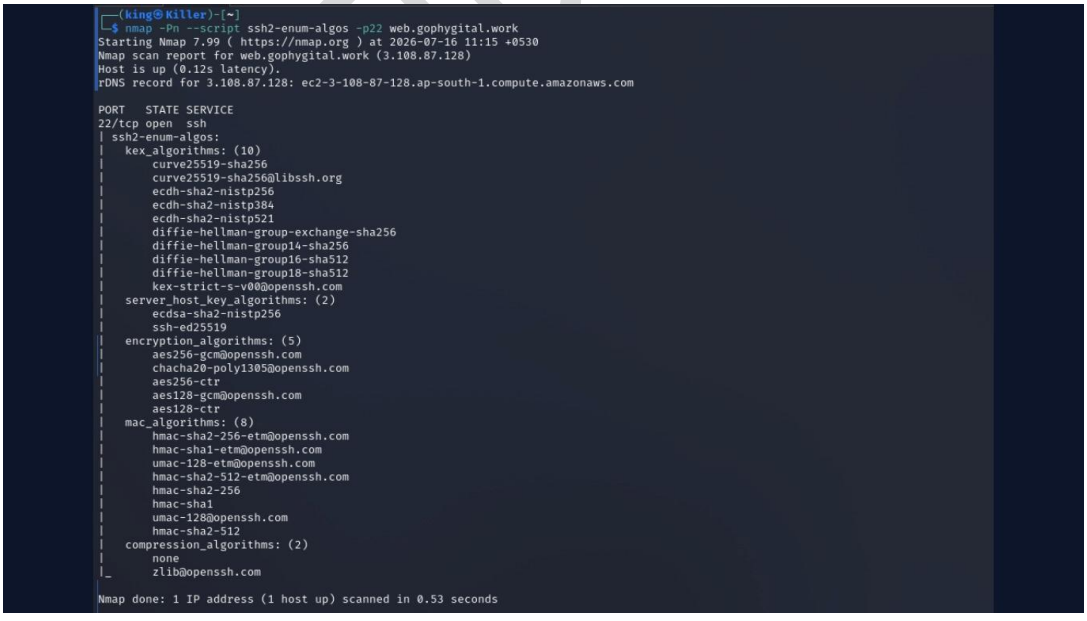
The SSH service supports deprecated SHA-1 based Message Authentication Code (MAC) algorithms (hmac-sha1 and hmac-sha1-etm@openssh.com). SHA-1 is considered cryptographically weak and has been deprecated due to practical collision attacks. Modern security standards recommend using SHA-2-based MAC algorithms exclusively.

Impact:

An attacker may exploit weaknesses in deprecated cryptographic algorithms under specific attack scenarios, reducing the overall security of SSH communications. While exploitation is difficult in practice, the use of obsolete algorithms weakens the server's cryptographic posture and does not comply with current security best practices.

References to Evidences and Proof of Concept:

Step 1: Enumerate supported SSH algorithms: nmap -Pn --script ssh2-enum-algos -p22 web.gophygital.work



Step 2: Verify using ssh-audit: ssh-audit web.gophygital.work

Step 3: Observe the following supported MAC algorithms: hmac-sha1, hmac-sha1-etm@openssh.com

Recommendations:

- Disable SHA-1 based MAC algorithms (hmac-sha1 and hmac-sha1-etm@openssh.com).
  - Configure the SSH server to use only modern SHA-2 or stronger MAC algorithms such as:
  - hmac-sha2-256-etm@openssh.com
  - hmac-sha2-512-etm@openssh.com
  - umac-128-etm@openssh.com
- Periodically review SSH cryptographic settings against current security recommendations.

5.19 Replay Attack on Comment Submission API (Duplicate Comment Creation)

Affected Asset: https://web.gophygital.work/

Affected Parameter: POST /complaint_logs.json

CWE-ID:

- CWE-294: Authentication Bypass by Capture-Replay (Replay attack)
- CWE-799: Improper Control of Interaction Frequency (No rate limiting contributes to the impact)

Severity: Medium

Description:

The comment submission endpoint accepts replayed authenticated requests without implementing anti-replay protections. After submitting a valid comment, the identical HTTP request can be captured and resent multiple times, resulting in duplicate comments being added to the same ticket.

The application does not validate whether the request has already been processed and does not require a unique request identifier (nonce) or one-time token. Additionally, the endpoint does not implement request rate limiting, allowing repeated submissions in rapid succession.

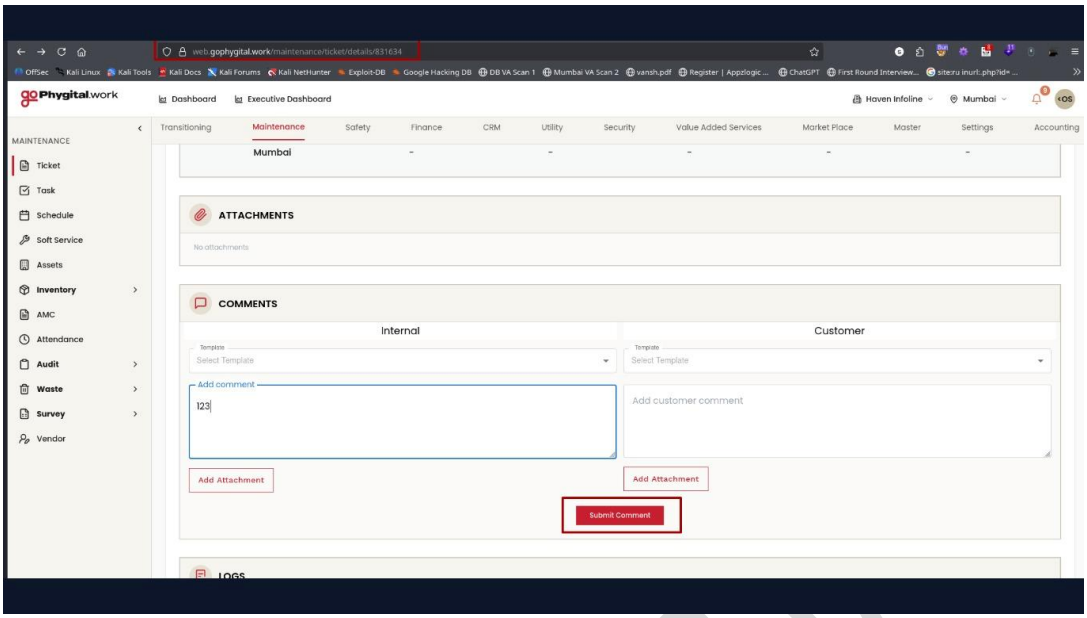
Impact:

- Attackers can flood tickets with duplicate comments.
- Ticket history and audit logs can be manipulated or polluted.
- Excessive duplicate requests may impact application performance and storage.
- Administrators and support staff may experience operational disruption due to log flooding.

References to Evidences and Proof of Concept:

Step 1: Authenticate to the application using a valid user account.

Step 2: Navigate to: https://web.gophygital.work/maintenance/ticket/details/831634

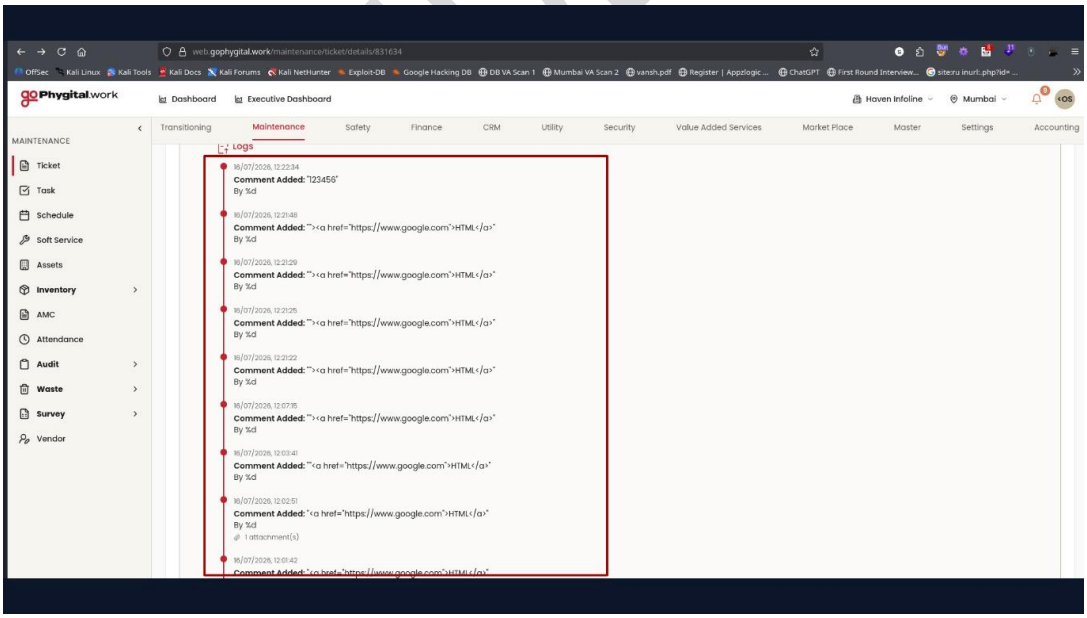


Step 3: Submit a comment

Step 4: Intercept the request using Burp Suite or another proxy: POST /complaint_logs.json HTTP/1.1 ... complaint_log[complaint_id]=831634 complaint_log[internal_comment]=123456

Step 5: Forward the exact same HTTP request multiple times without modifying the payload.

Step 6: Each replayed request is accepted successfully, and a new duplicate comment is added to the ticket.



Recommendations:

- Implement anti-replay protection using one-time request tokens or unique request identifiers (nonces).
- Validate duplicate submissions on the server side before creating a new comment.
- Implement rate limiting on comment creation endpoints.

- Log and detect excessive repeated requests from the same user or IP address.
- Return an appropriate error response when an identical request is replayed.

5.20 Sensitive Information Disclosure via EXIF Metadata in Uploaded Image Attachments

Affected Asset: https://web.gophygital.work/

Affected Parameter: Ticket Comment Attachment Upload

CWE-ID: CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

Severity: Medium

Description:

The application allows users to upload image attachments without removing embedded EXIF metadata. After uploading an image as a ticket attachment, the downloaded image retained its original metadata.

The downloaded image was analyzed using Jimpl EXIF Metadata Viewer, which confirmed that the image still contained sensitive EXIF information, including:

- GPS coordinates (Latitude and Longitude)
- Camera make and model
- Original date and time
- Camera configuration (ISO, aperture, shutter speed, focal length)
- Image processing software

This demonstrates that the application stores and serves uploaded images without sanitizing embedded metadata.

Impact:

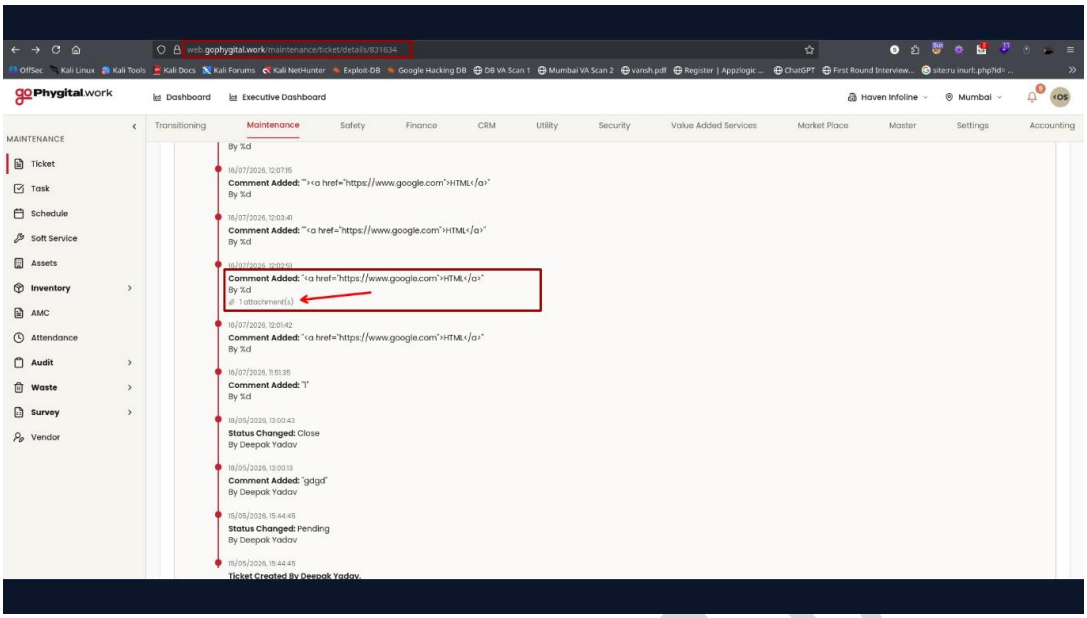
An attacker with access to uploaded images may obtain sensitive information, including:

- Physical location where the image was captured.
- Camera and device information.
- Date and time the image was taken.
- Additional metadata useful for profiling users or organizational assets.

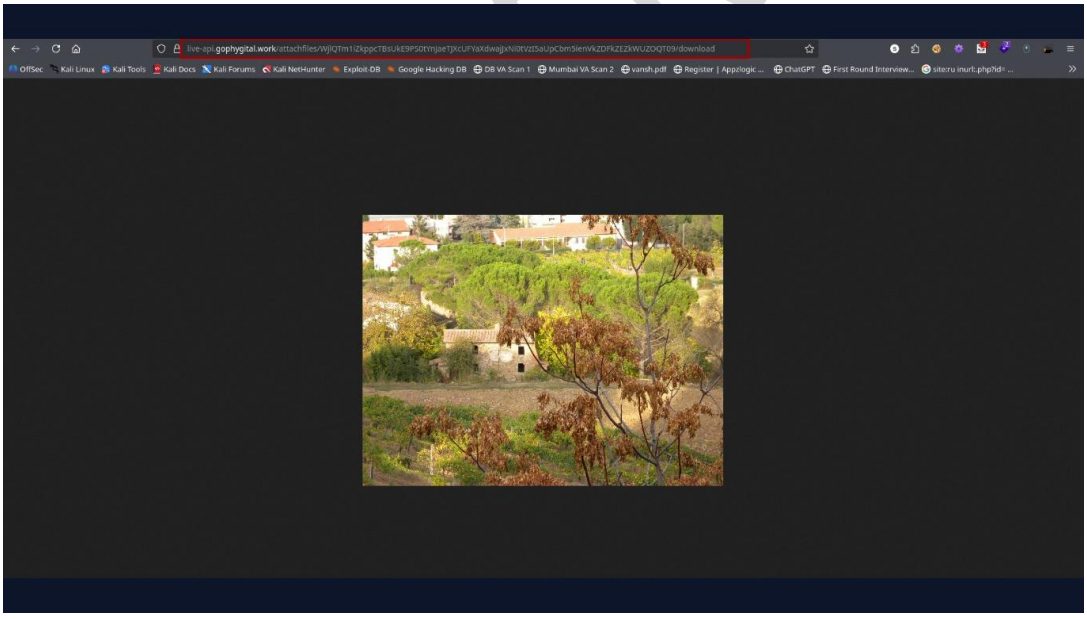
If employees upload photographs taken within corporate premises or sensitive locations, GPS coordinates and device metadata may unintentionally disclose confidential operational information.

References to Evidences and Proof of Concept:

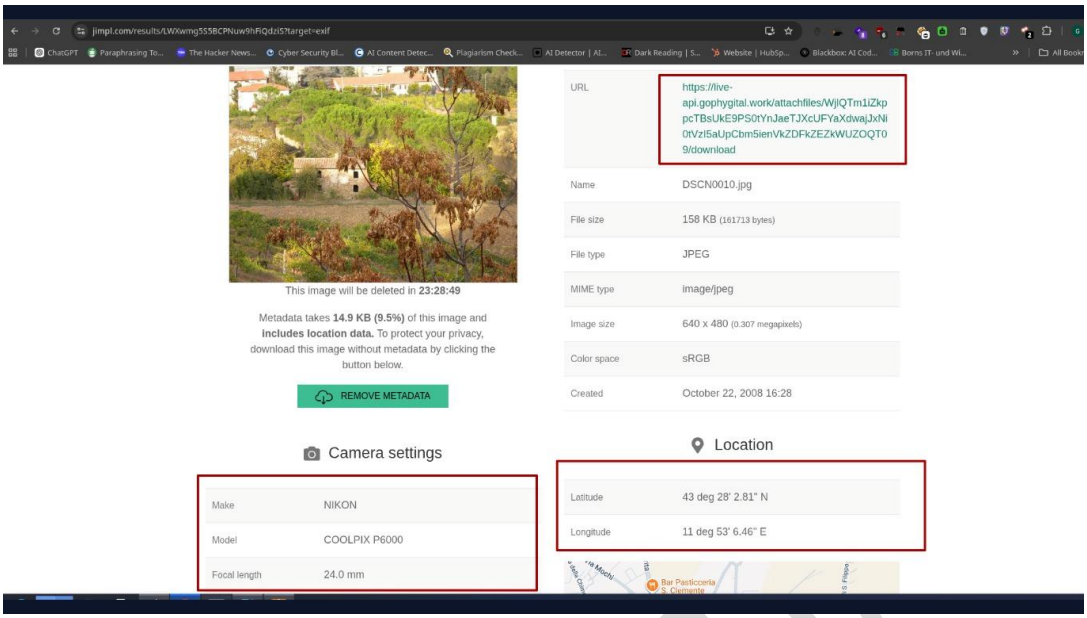
Step 1: Log in to the application and upload a JPEG image as an attachment in a ticket comment.



Step 2: Open the uploaded attachment using the generated download URL. Example: https://live-api.gophygital.work/attachfiles/WjlQTm1iZkppcTBsUkE9PS0tYnJaeTJXcUFYaXdwajJxNi0tVzI5aUpCbm5ienVkZDFkZEZkWUZOQT09/download



Step 3: Open the downloaded image using Jimpl EXIF Metadata Viewer by providing the image URL or uploading the downloaded file.



Step 4: The analysis revealed embedded EXIF metadata, including: • GPS Latitude and Longitude • Camera Make: Nikon • Camera Model: COOLPIX P6000 • Date/Time Original • Camera Settings • Software Information This confirms that EXIF metadata is preserved after upload and remains accessible to users downloading the image.

Recommendations:

- Strip all EXIF metadata from uploaded images before storing or serving them.
- Remove GPS coordinates, camera details, timestamps, and other non-essential metadata.
- Re-encode images during upload to ensure metadata is removed.
- Validate uploaded media through a secure image-processing pipeline.
- Inform users that uploaded images may contain embedded metadata that could expose sensitive information.

5.21 Denial of Service via Multiple Overlapping HTTP Byte-Range Requests

Affected Asset: web.gophygital.work

Affected Parameter: N/A

CWE-ID: CWE-400 – Uncontrolled Resource Consumption

Severity: Medium

Description:

The web server is vulnerable to the Apache Byte-Range Denial of Service (DoS) vulnerability (CVE-2011-3192). An attacker can send specially crafted HTTP requests containing multiple overlapping byte ranges in the Range header, causing excessive CPU and memory consumption. Successful exploitation may degrade server performance or render the web service unavailable to legitimate users.

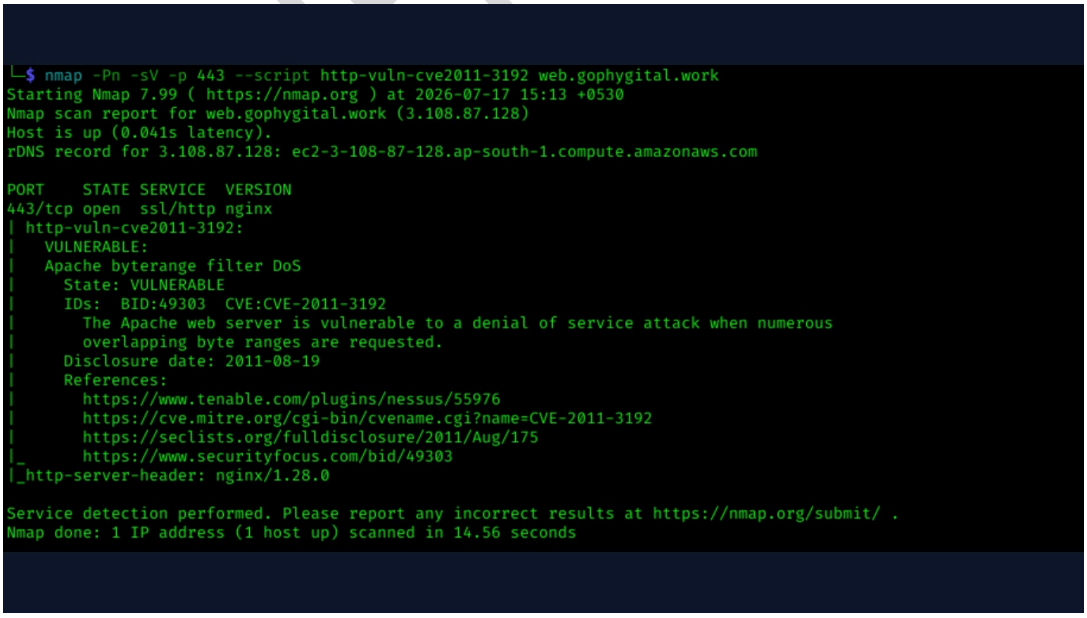
Impact:

- Denial of Service (DoS) by exhausting server resources.
- Degraded application performance and increased response times.
- Temporary unavailability of the web application for legitimate users.
- Potential disruption of business operations.

References to Evidences and Proof of Concept:

Step 1: Execute the following Nmap NSE script to check for the Apache Byte-Range DoS.

Step 2: nmap -Pn -sV -p 443 --script http-vuln-cve2011-3192 vyapaarsense.com



Step 3: Observe that the scan output identifies the target as VULNERABLE to CVE-2011-3192 (Apache Byte-Range DoS).

Recommendations:

- Upgrade the web server to a version that is not affected by CVE-2011-3192.
- Apply the latest security patches provided by the web server vendor.
- Configure the web server to restrict or disable excessive byte-range requests where appropriate.
- Deploy a Web Application Firewall (WAF) or reverse proxy to detect and block malicious Range header requests.
- Continuously monitor server logs for abnormal requests targeting the Range header and investigate suspicious activity.

References:

- N/A

5.22 Insecure Storage of Session/Bearer Token in Browser Local Storage

Affected Asset: https://web.gophygital.work/business-compass/profile

Affected Parameter: local storage

CWE-ID: CWE-922 – Insecure Storage of Sensitive Information

Severity: Medium

Description:

The application stores the session/bearer token in the browser's **Local Storage** after successful authentication. Since Local Storage is accessible to any JavaScript executing within the application's origin, authentication tokens may be exposed if the application is affected by a Cross-Site Scripting (XSS) vulnerability or if a malicious browser extension executes within the user's browser context.

Unlike **HttpOnly** cookies, data stored in Local Storage cannot be protected from client-side JavaScript. Consequently, an attacker who obtains the token can potentially impersonate the victim until the token expires or is revoked.

Impact:

- Theft of authentication/session tokens through client-side JavaScript.
- Unauthorized access to the victim's account by replaying the stolen bearer token.
- Session hijacking without requiring the user's credentials.
- Increased impact of any existing or future Cross-Site Scripting (XSS) vulnerability.
- Potential compromise of sensitive user information and protected resources accessible through authenticated APIs.
- Increased risk from malicious browser extensions or compromised third-party JavaScript executing within the application's origin.

References to Evidences and Proof of Concept:

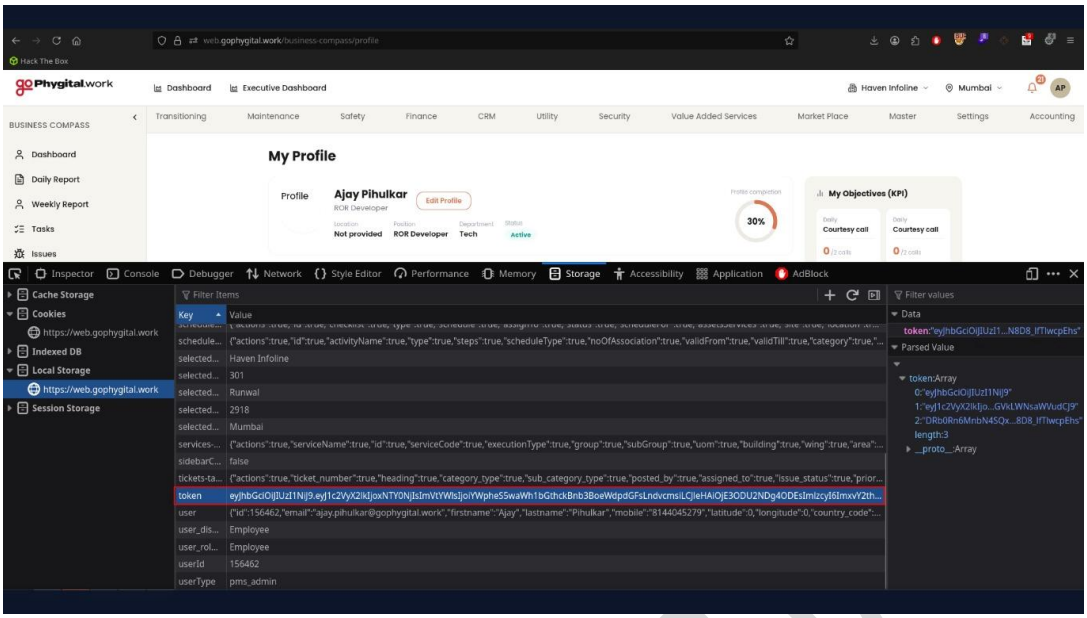
Step 1: Log in to the application using a valid user account.

Step 2: Open the browser's Developer Tools.

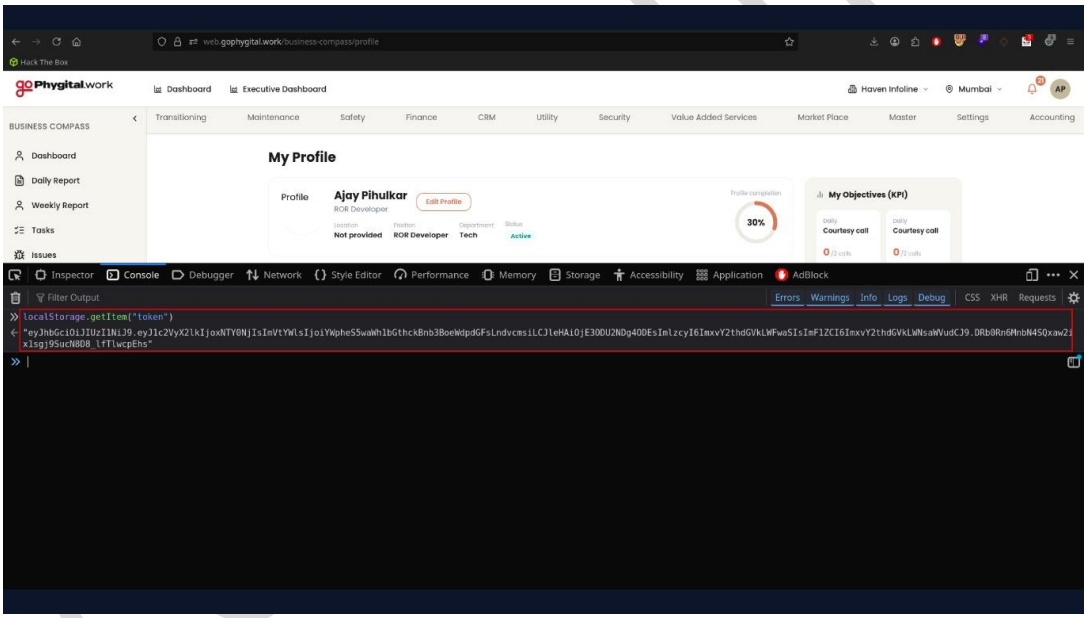
Step 3: Navigate to the Application (or Storage) tab.

Step 4: Select Local Storage for the application's domain.

Step 5: Observe that the session/bearer token is stored in Local Storage.



Step 6: Open the browser console and execute: localStorage.getItem("<token_key>")



Step 7: Verify that the authentication token is returned and can be copied for reuse in authenticated requests.

Recommendations:

- Avoid storing authentication tokens in **Local Storage** or **Session Storage**.
- Store session identifiers in **Secure**, **HttpOnly**, and **SameSite** cookies to prevent access through client-side JavaScript.
- Implement a strong **Content Security Policy (CSP)** to reduce the risk of XSS.
- Minimize the lifetime of access tokens and implement token rotation where applicable.
- Revoke tokens immediately upon logout and invalidate compromised or expired tokens on the server.

- Regularly test the application for XSS vulnerabilities, as successful XSS can lead to authentication token theft.

5.23 Improper Input Validation in Restricted Dropdown Fields

Affected Asset:

- https://web.gophygital.work/maintenance/inventory
- https://web.gophygital.work/maintenance/service

Affected Parameter:

- unit
- category
- spa_plant_code
- pms_service[execution_type]

CWE-ID: CWE-20: Improper Input Validation

Severity: Medium

Description:

The application relies on client-side controls to restrict user input for fields that are intended to accept only predefined values, such as dropdown selections, reference/master data fields, lookup fields, and other controlled input parameters.

During testing, it was observed that by intercepting and modifying HTTP requests, arbitrary values could be supplied in place of the predefined options. The application accepted, processed, and stored these unauthorized values without performing adequate server-side validation.

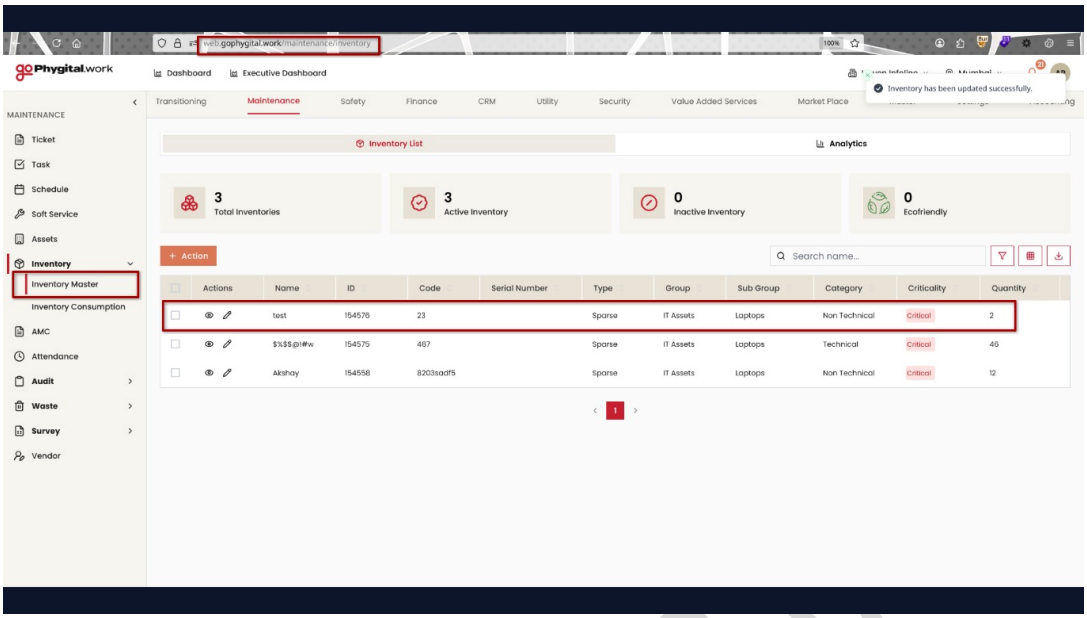
As a result, users can bypass intended business rules and inject custom values into fields that should only contain approved entries from a predefined list. This issue was observed across multiple functionalities and endpoints, indicating a systemic lack of server-side validation for controlled input fields.

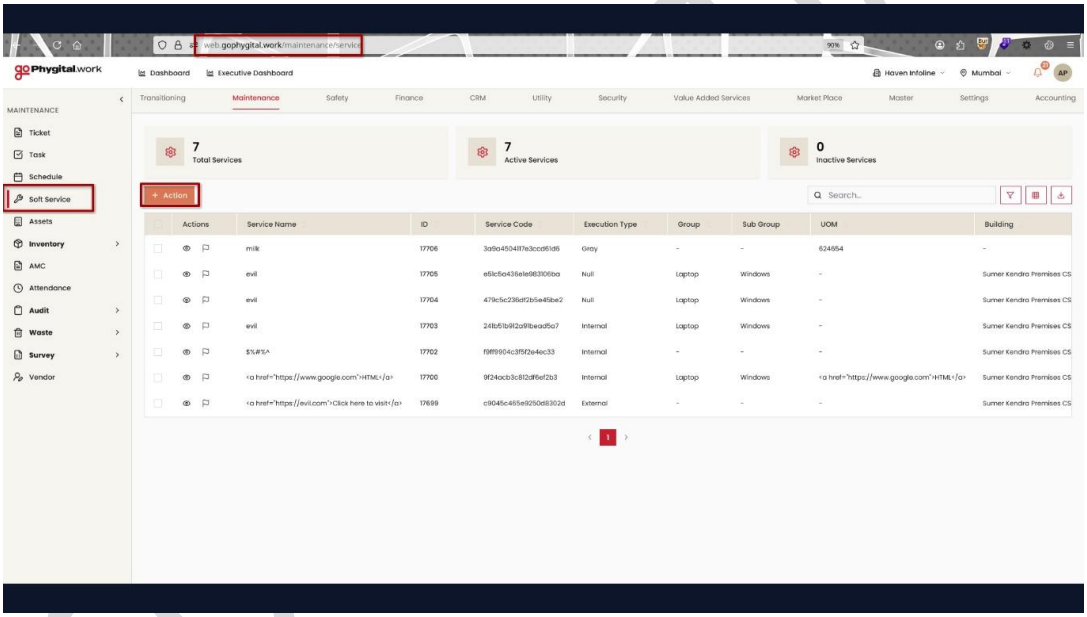
Impact:

- Compromise of data integrity across the application.
- Insertion of unauthorized or invalid values into master and transactional records.
- Inconsistent data across business processes and reports.
- Corruption of reference/master data relationships.
- Potential disruption of downstream workflows, integrations, and automated processes.
- Increased risk of reporting inaccuracies and operational issues.
- Circumvention of business rules and application-enforced constraints.
- Reduced reliability of application data used for decision-making and compliance purposes.

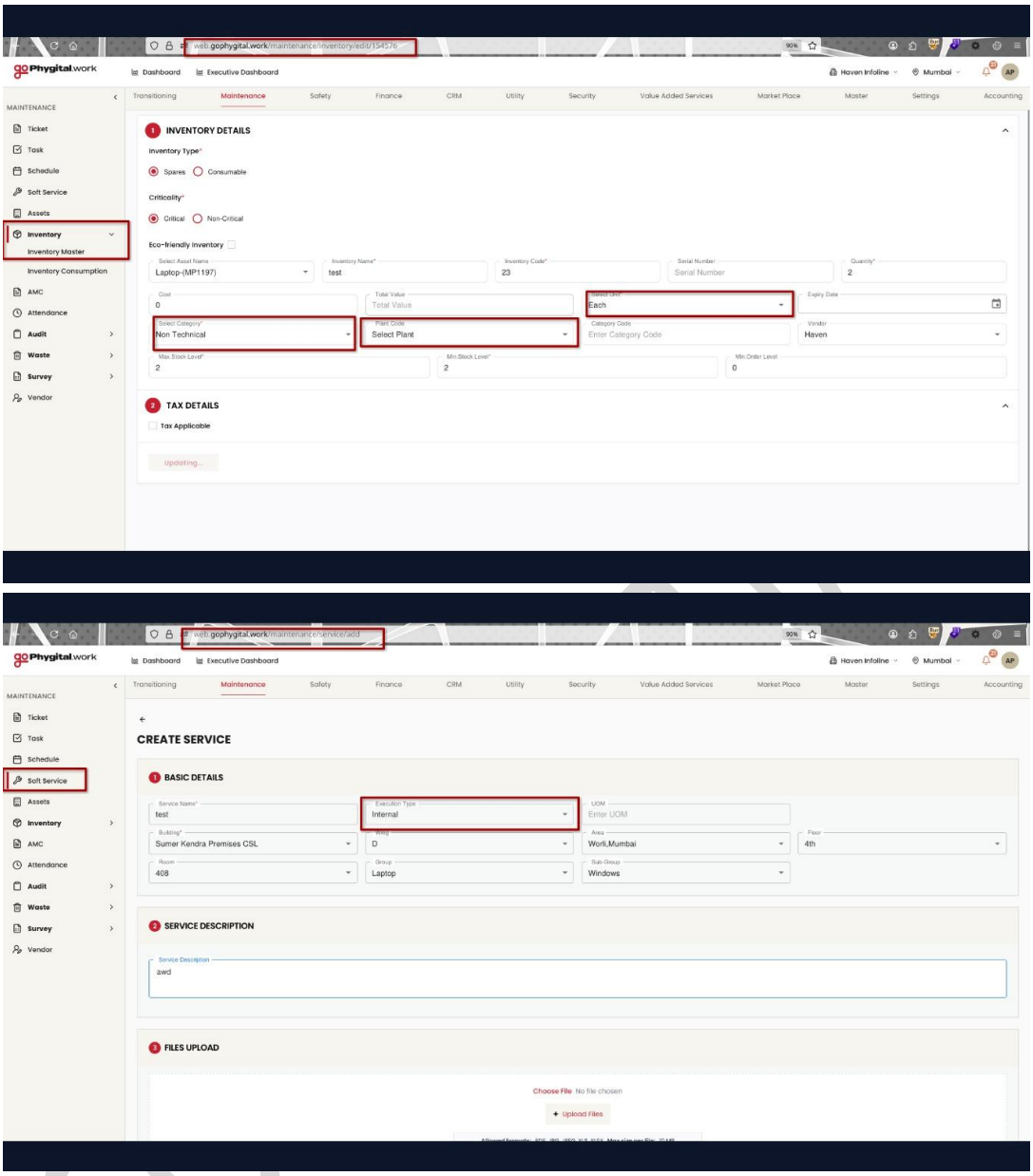
References to Evidences and Proof of Concept:

Step 1: Navigate to the edit functionality.



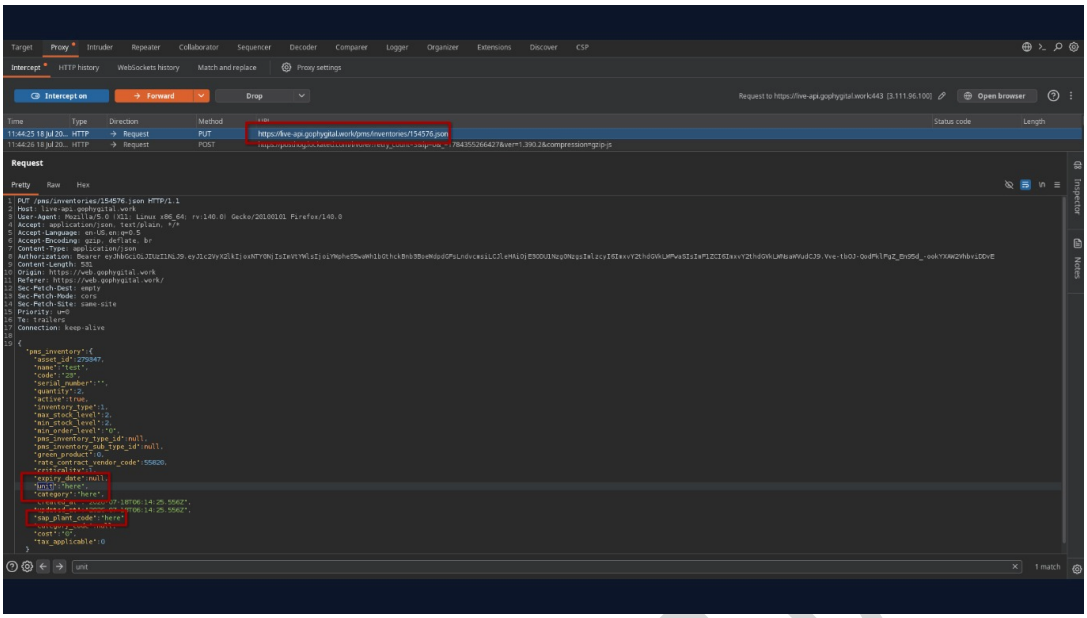


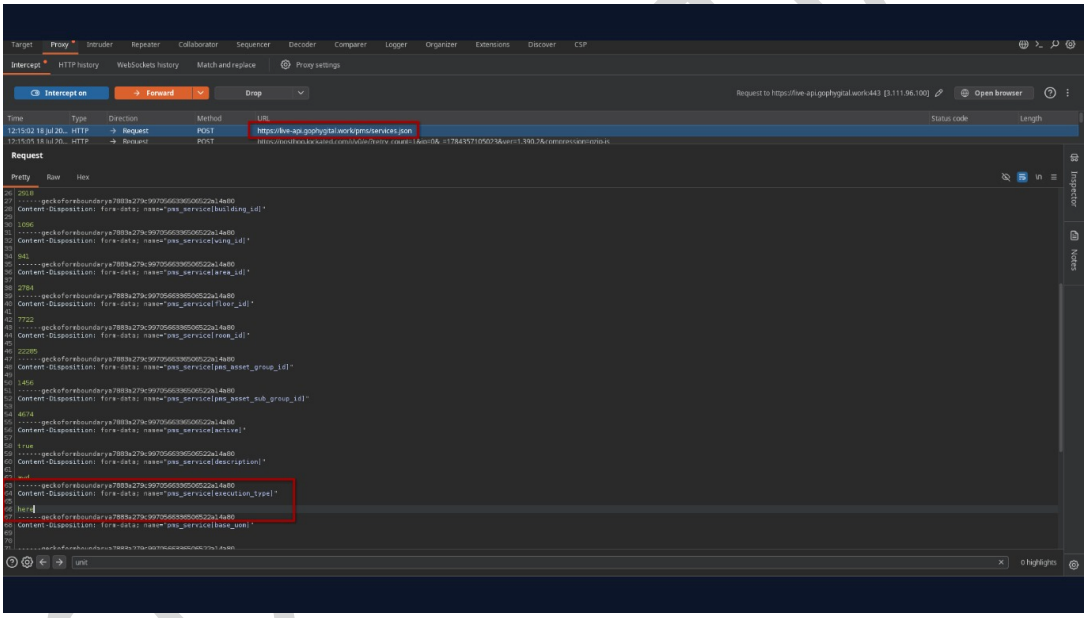
Step 2: Observe that the affected parameters are presented as dropdown selections.



Step 3: Select any valid value and intercept the Create/Update request using Burp Suite.

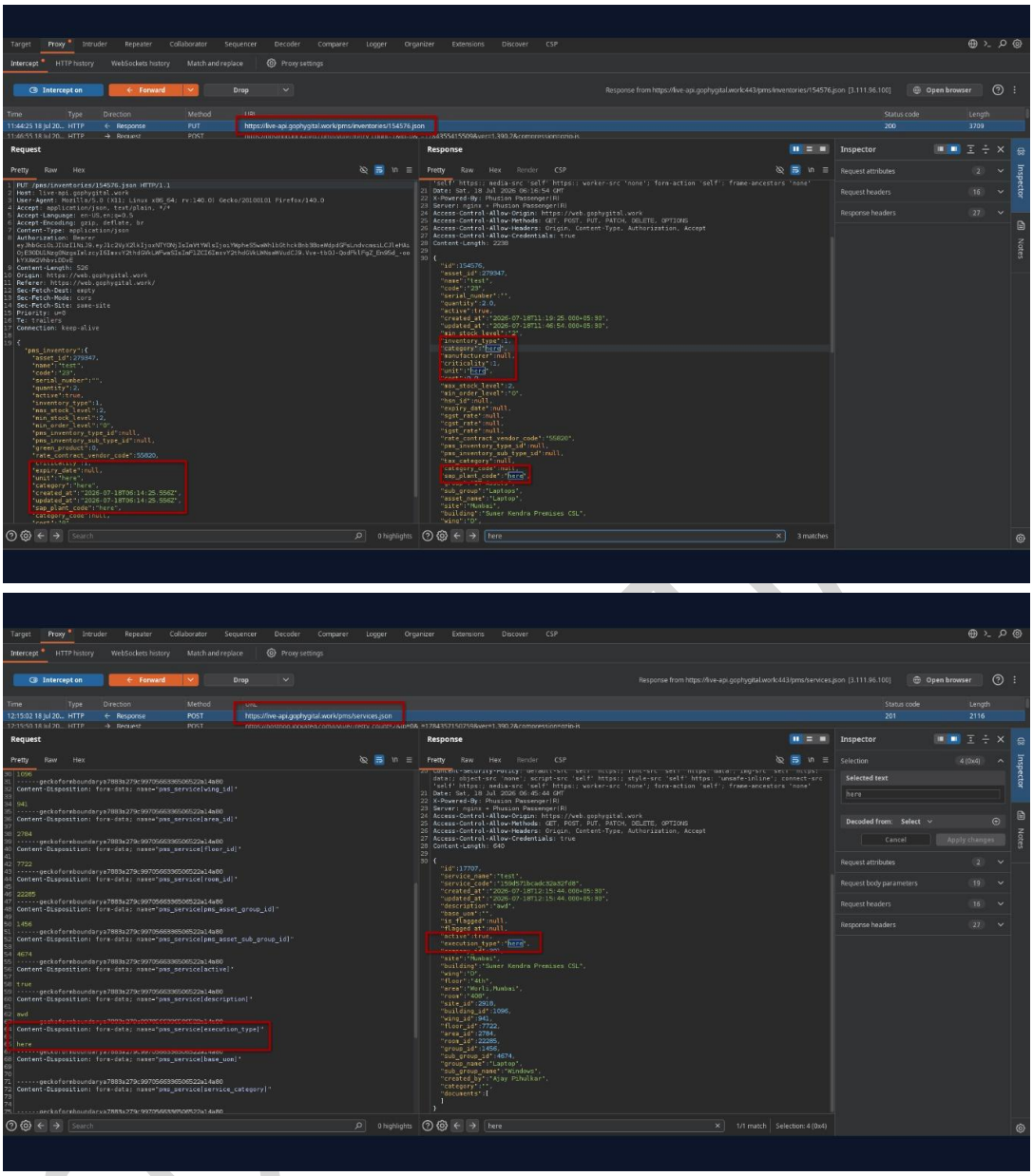
Step 4: Modify the request before forwarding it.



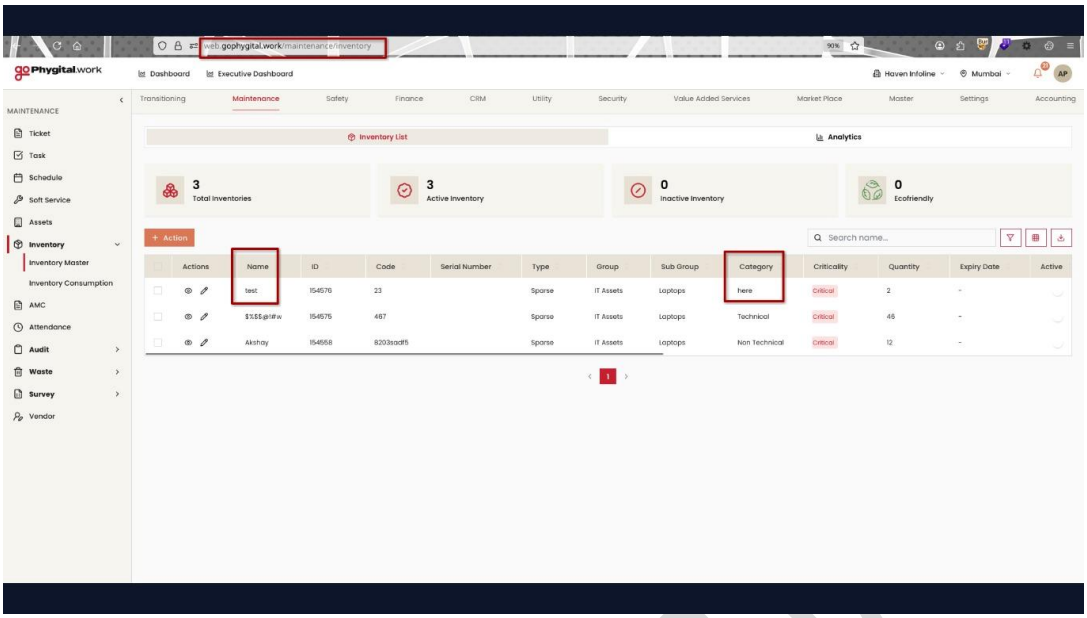


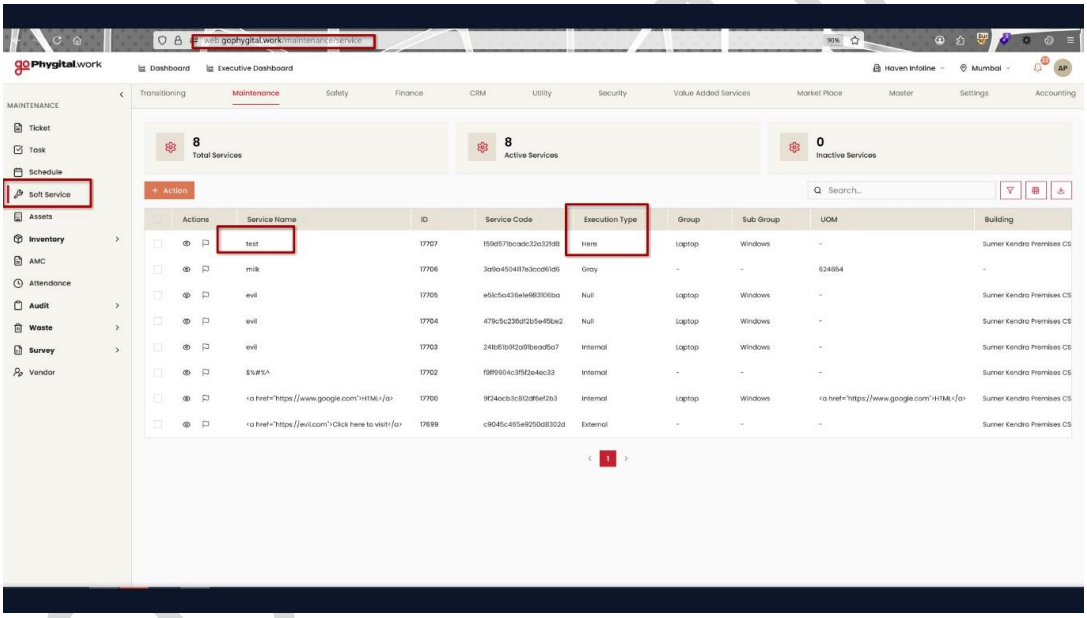
Step 5: Forward the modified request.

Step 6: Observe that the application returns a successful response and stores the modified values.



Step 7: Navigate back to the record and verify that the custom values are displayed and persisted within the application.





Recommendations:

- Implement strict server-side validation for all user-supplied input, including fields presented as dropdowns, lookup values, and controlled selections.
- Validate submitted values against an approved whitelist or reference dataset maintained on the server.
- Reject requests containing values that are not explicitly authorized by the application.
- Do not rely solely on client-side controls, HTML form restrictions, or frontend validation mechanisms.
- Apply consistent validation controls across all create, update, import, and API endpoints.
- Implement centralized input validation routines to ensure uniform enforcement of business rules.

- Periodically review application endpoints to identify and remediate similar validation bypass issues.
- Log and monitor validation failures to detect potential abuse or tampering attempts.

5.24 Session Reuse Leads To Authentication Bypass

Affected Asset: https://live-api.gophygital.work/api/users/sign_in.json

Affected Parameter: Login panel

CWE-ID: CWE-613 – Insufficient Session Expiration

Severity: Medium

Description:

During testing, it was observed that after an account was locked due to multiple failed authentication attempts, the application displayed the message **"Contact Administrator"**, indicating that further authentication attempts were blocked.

However, a previously captured successful authentication response containing a valid bearer/session token remained usable. By replaying or injecting the earlier authentication response, the client continued to use the old token and was able to access authenticated functionality even though the account was in a locked state.

This indicates that the application does not invalidate existing sessions or authentication tokens when an account is locked. As a result, account lockout only prevents new logins while existing authenticated sessions remain active.

Impact:

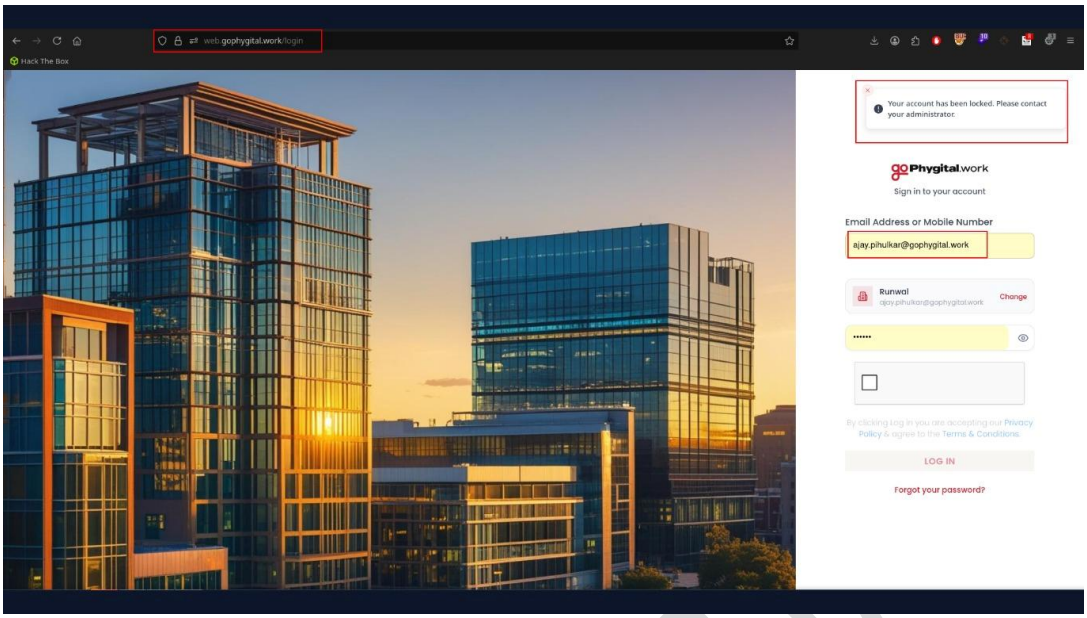
- Bypass of the account lockout mechanism.
- Previously issued bearer/session tokens remain valid after account lockout.
- Attackers with a stolen or previously compromised token can continue accessing the application.
- Account lockout fails to terminate active authenticated sessions.
- Protected resources remain accessible despite the account being disabled or locked.
- Reduces the effectiveness of brute-force protection and incident response measures.
- May result in unauthorized access to sensitive information and privileged functionality.

References to Evidences and Proof of Concept:

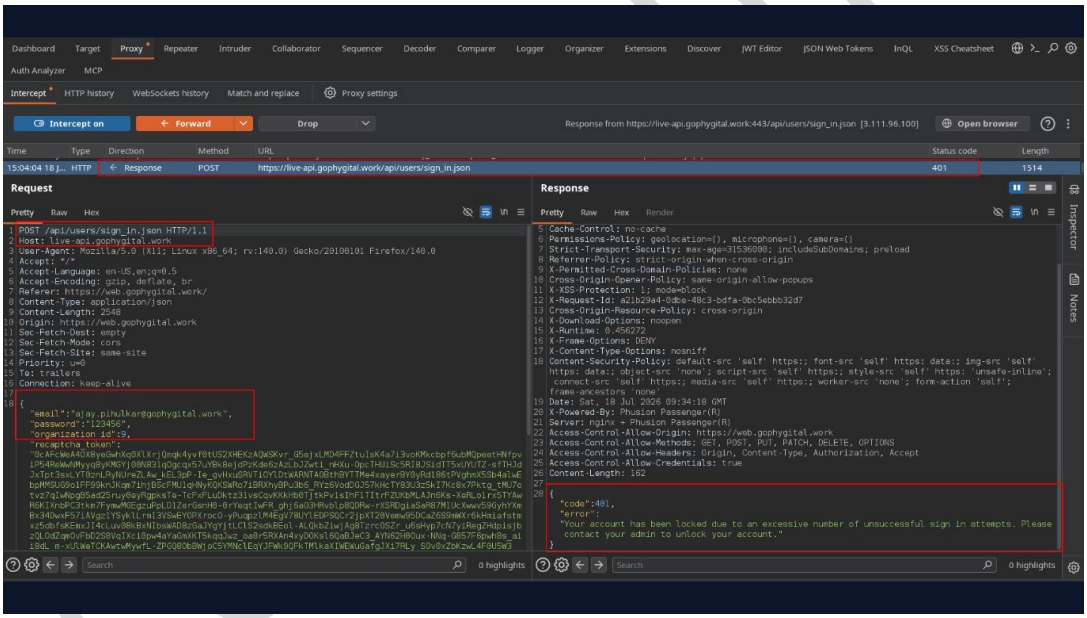
Step 1: Log in successfully once and intercept the authentication response using Burp Suite : https://live-api.gophygital.work/api/users/sign_in.json

Step 2: Save the successful login response in Burp Suite.

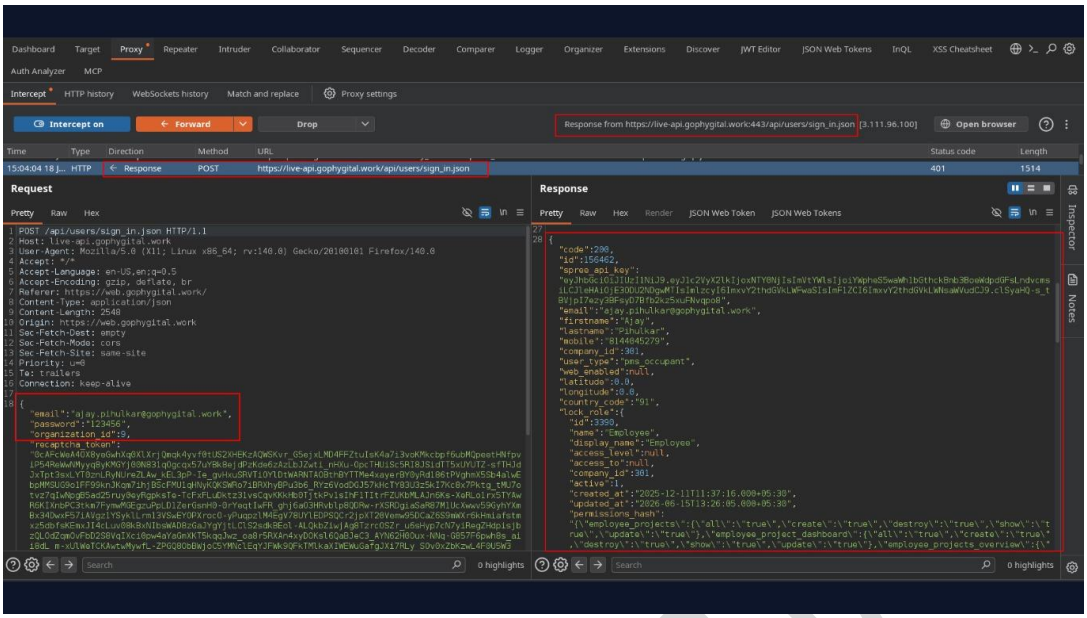
Step 3: Perform multiple failed login attempts until the application locks the account and displays "Contact Administrator".



Step 4: Attempt to log in again and intercept the server response.

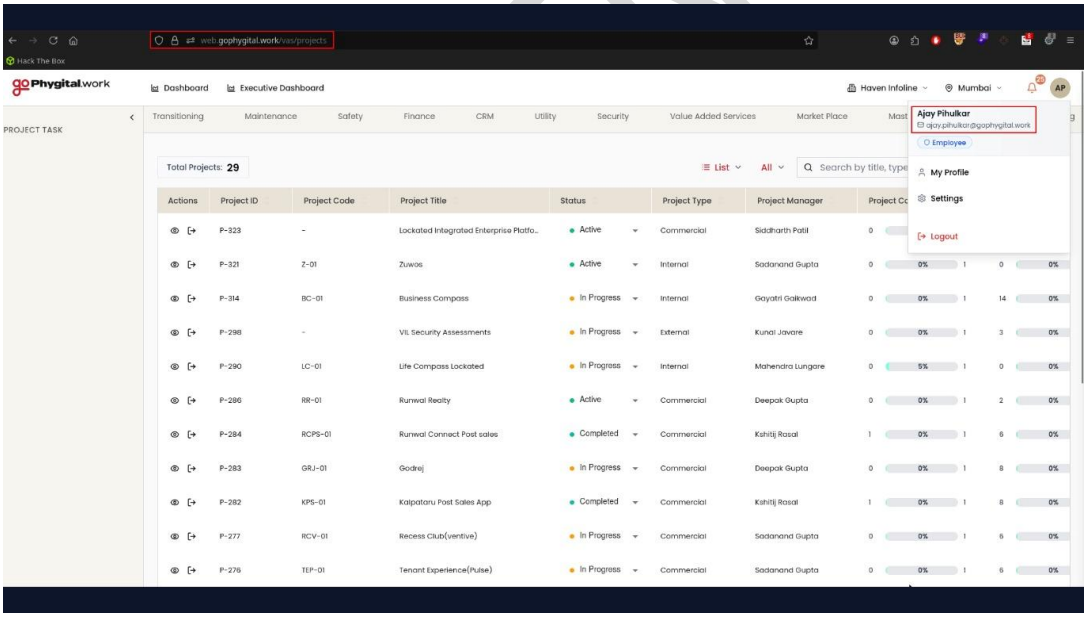


Step 5: Replay the previously captured successful authentication response or continue using the previously issued bearer token.



Step 6: Forward the manipulated response to the browser.

Step 7: Observe that the application grants authenticated access even though the account remains locked.



Recommendations:

- Immediately invalidate all active sessions and bearer tokens when an account is locked.
- Verify the user's current account status on every authenticated request.
- Reject requests made with tokens belonging to locked or disabled accounts.
- Maintain server-side session state or implement token revocation for JWTs.
- Force re-authentication after an account is unlocked.
-

Log and alert when requests are received using tokens associated with locked accounts.

5.25 Lack of Rate Limiting on Incident Creation Functionality

Affected Asset:

- https://web.gophygital.work/safety/incident
- https://web.gophygital.work/maintenance/ticket

Affected Parameter:

- Add Incident
- Add

CWE-ID: CWE-770: Allocation of Resources Without Limits or Throttling

Severity: Medium

Description:

The application does not implement adequate rate-limiting or request-throttling mechanisms on one or more functionalities. During testing, it was observed that a large number of requests could be submitted within a short period without triggering any restrictions, delays, account lockouts, CAPTCHA challenges, or other anti-automation controls.

The absence of rate limiting allows authenticated or unauthenticated users (depending on the affected functionality) to repeatedly invoke application actions without restriction. This may enable automated abuse, excessive resource consumption, and operational disruption.

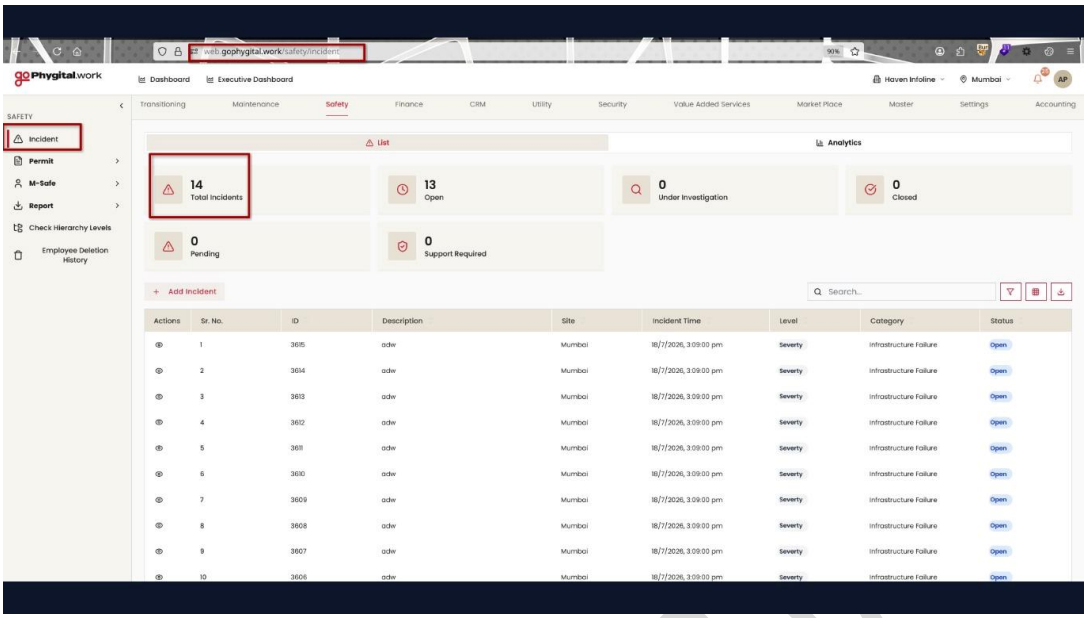
Impact:

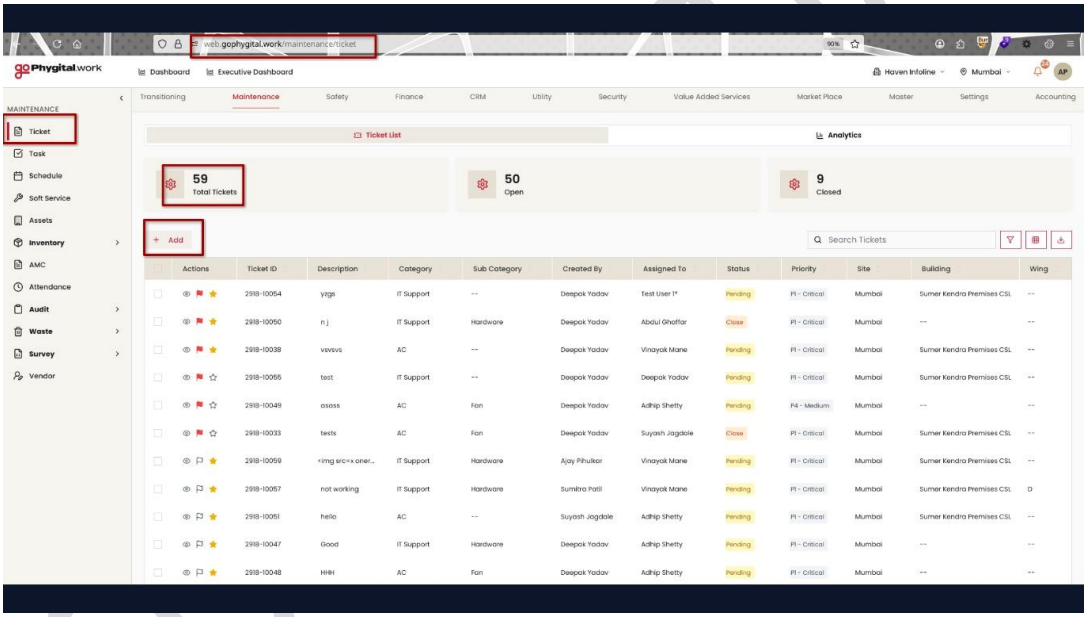
- Automated abuse of application functionality.
- Excessive consumption of server, database, and storage resources.
- Increased risk of denial-of-service (DoS) conditions through resource exhaustion.
- Large-scale generation of records, transactions, or uploads.
- Potential degradation of application performance and responsiveness.
- Increased storage, processing, backup, and infrastructure costs.
- Facilitation of automated attacks such as brute-force attempts, enumeration, spam submissions, and workflow abuse.
- Reduced availability and reliability of affected services.

References to Evidences and Proof of Concept:

Step 1: Log in to the application using a employee user account.

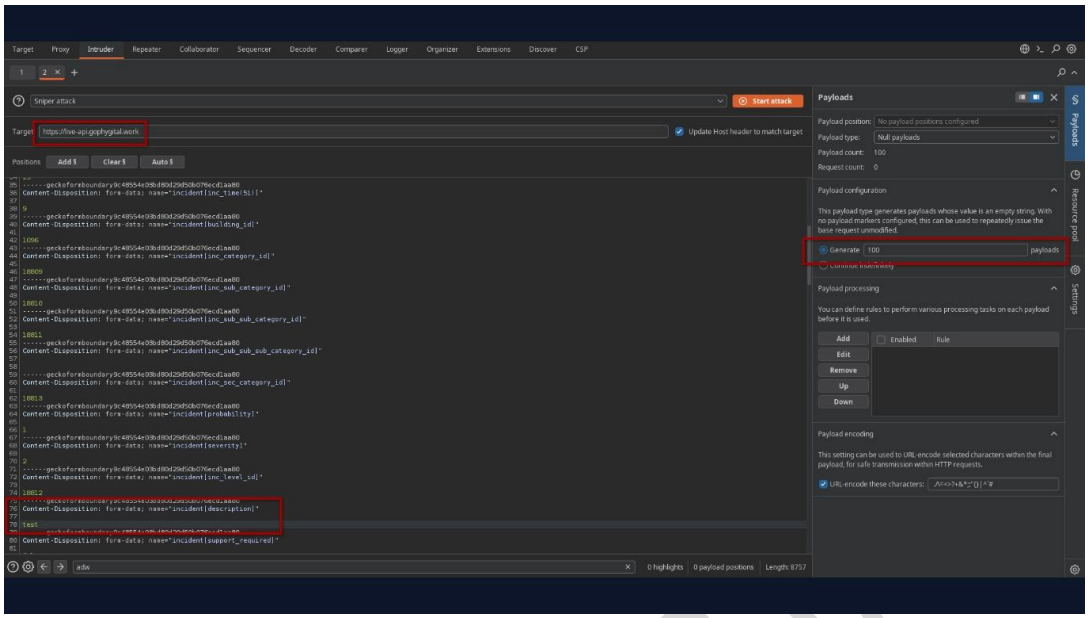
Step 2: Navigate to the functionality.

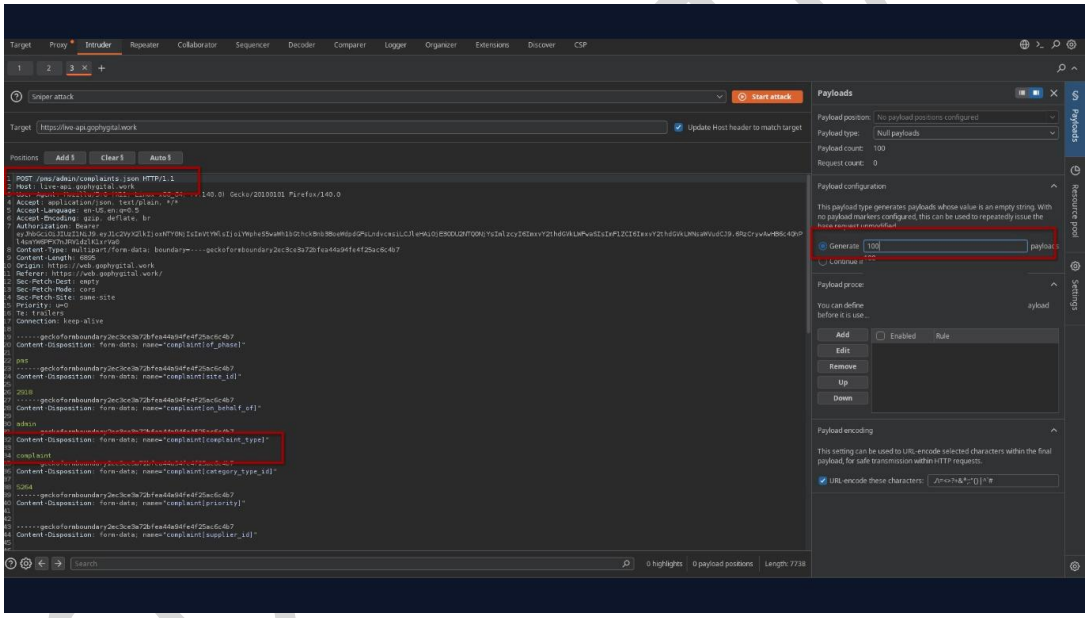




Step 3: Fill in the required details and attach a file using the file upload option.

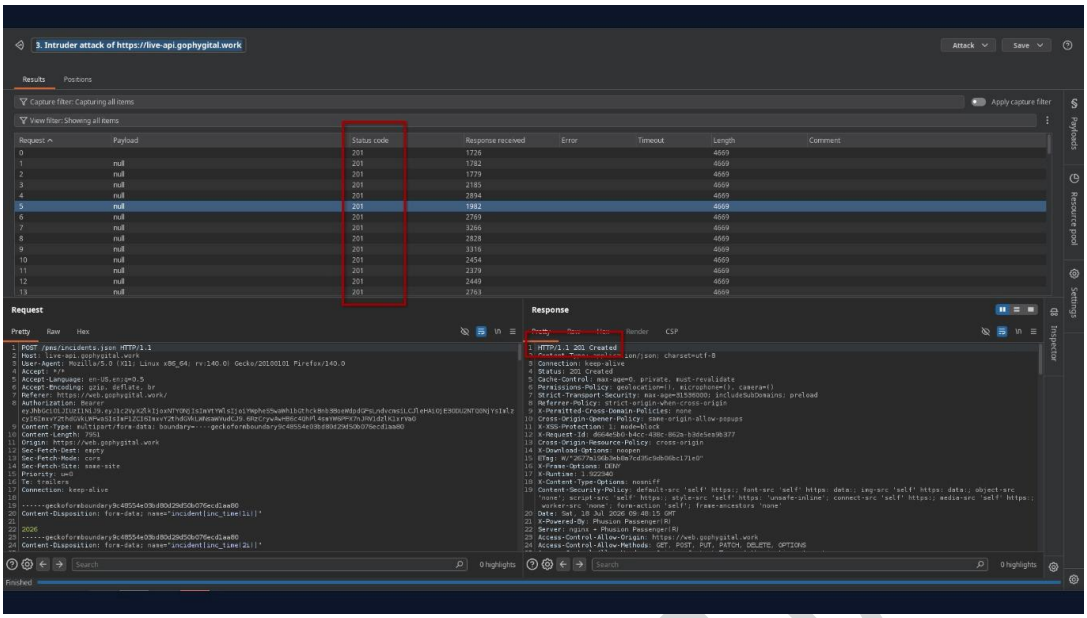
Step 4: Intercept the request using Burp Suite and send it to Burp Repeater or Intruder.

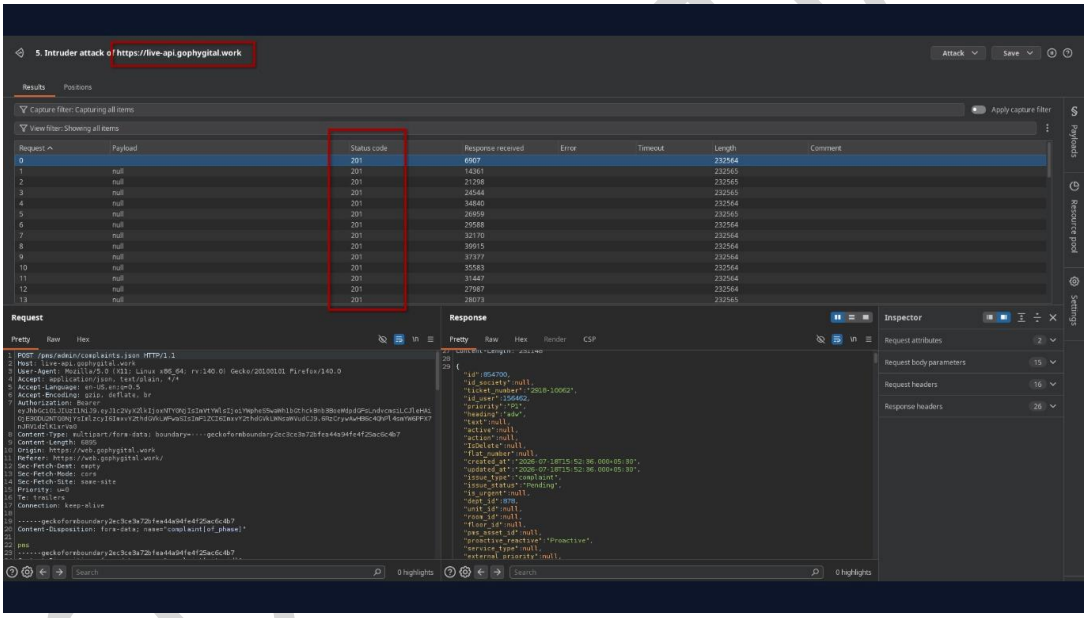




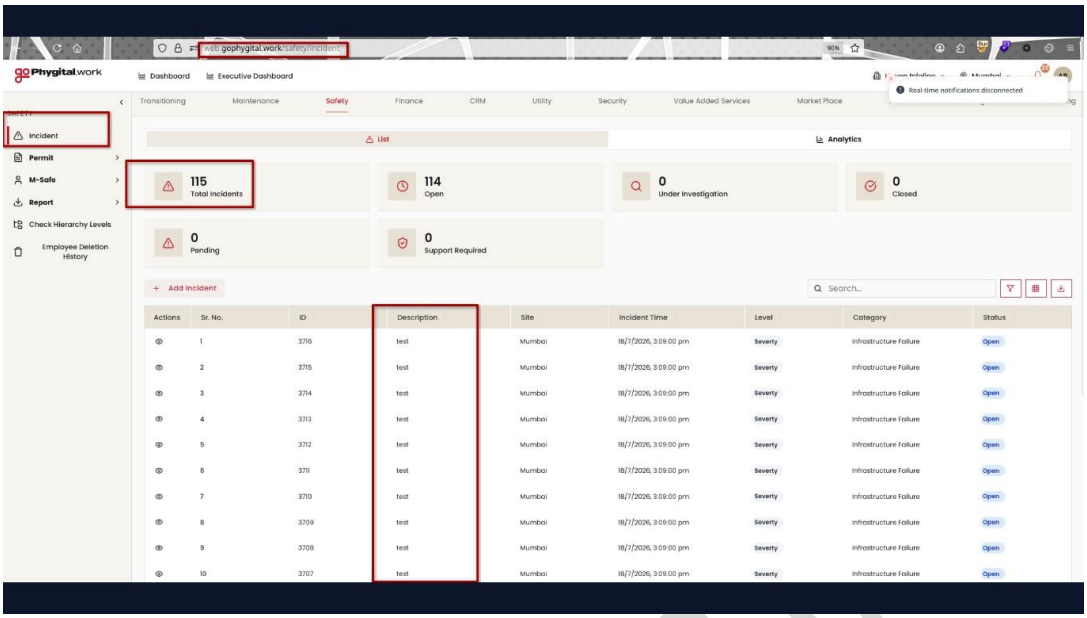
Step 5: Replay the request multiple times in rapid succession (e.g., 50–100 requests) without modifying the session.

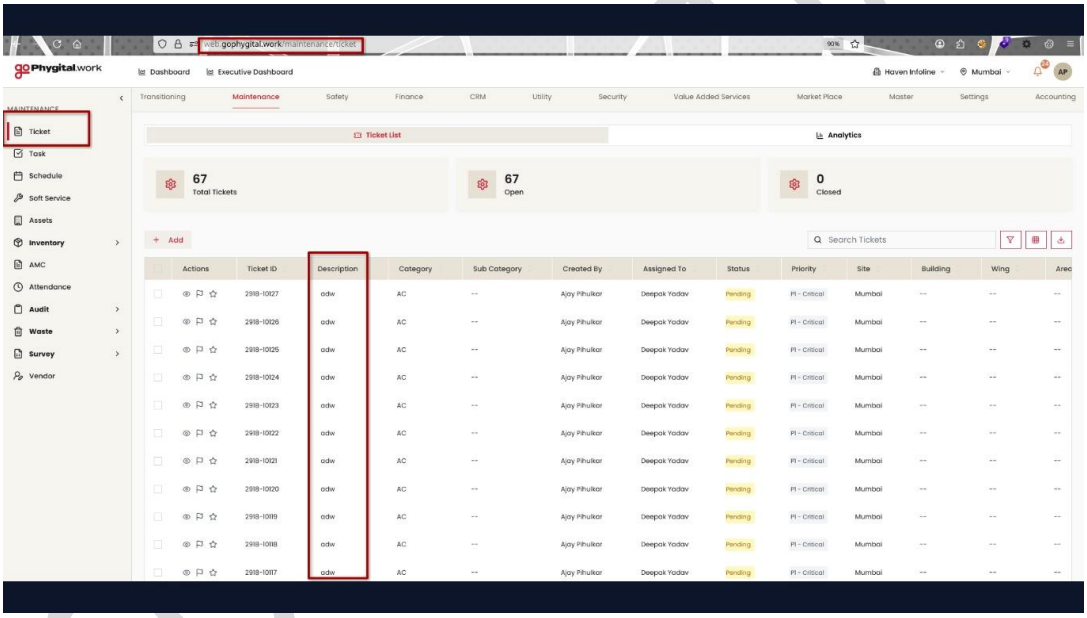
Step 6: Observe that the application processes all requests successfully and creates multiple records without enforcing any rate-limiting, throttling, or anti-automation controls.





Step 7: Confirm that a large number of requests and associated file uploads can be generated within a short period, demonstrating the potential for storage consumption and operational disruption.





Recommendations:

- Implement rate limiting on sensitive and resource-intensive endpoints.
- Apply request throttling based on user account, IP address, session, and API key where applicable.
- Introduce progressive delays after excessive requests.
- Define request quotas appropriate to business requirements.
- Monitor and alert on abnormal request volumes and automated activity.
- Log excessive request patterns and investigate potential abuse.
- Regularly review exposed endpoints to ensure appropriate anti-automation controls are in place.

5.26 Improper Server-Side Validation of Character Length Restrictions

Affected Asset: https://web.gophygital.work/safety/incident

Affected Parameter: incident[description]

CWE-ID: CWE-20: Improper Input Validation

Severity: Medium

Description:

The application enforces character length restrictions on certain input fields through client-side validation; however, these restrictions are not properly validated on the server side.

During testing, it was observed that fields configured with a maximum character limit of **240 characters** could be bypassed by intercepting the request and modifying the submitted value before it reached the server. The application accepted values significantly exceeding the intended limit and successfully stored them.

Furthermore, the oversized content was reflected within the application after submission, confirming that the server processed and persisted the unauthorized input without enforcing the defined validation rules.

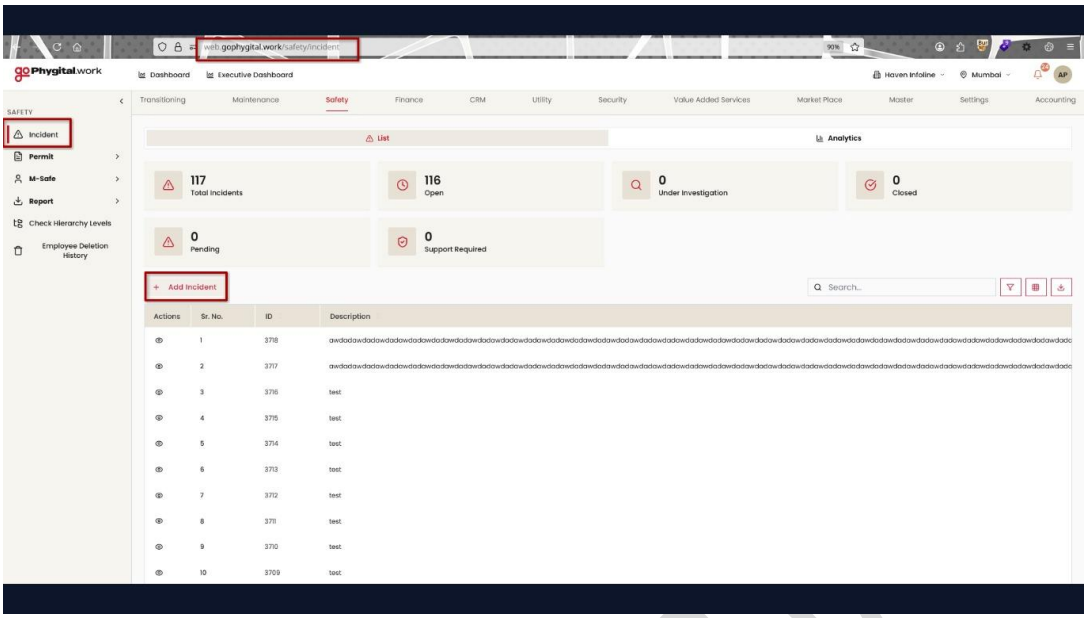
This issue indicates a lack of server-side validation and allows users to bypass business logic controls intended to restrict input length.

Impact:

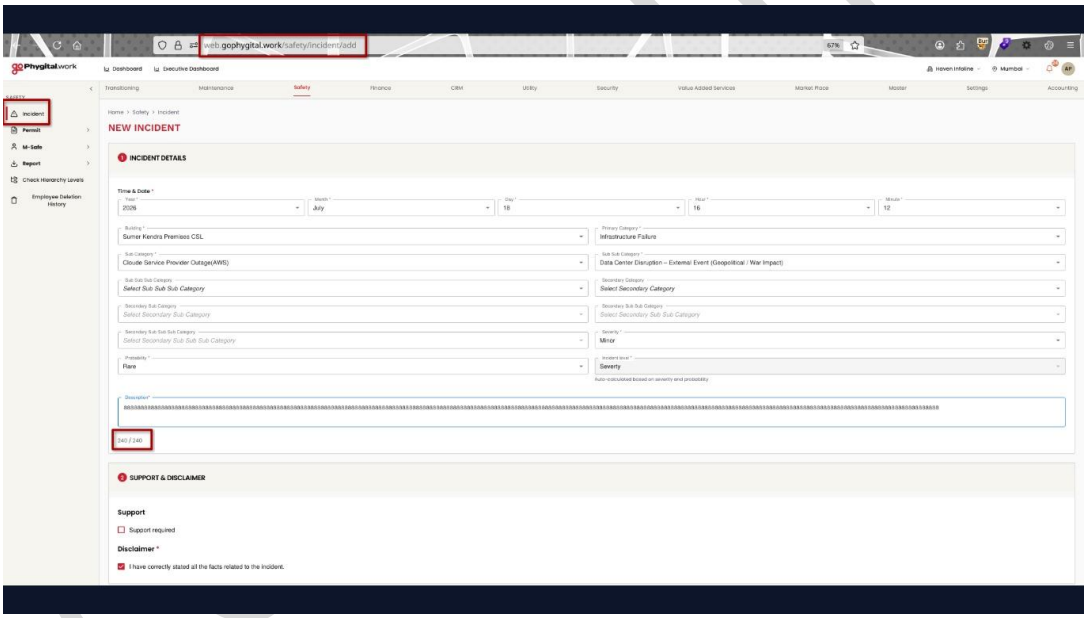
- Bypass of intended business validation controls.
- Storage of oversized and unexpected data within application records.
- Data integrity issues within application workflows.
- Potential database performance and storage concerns due to excessive input sizes.
- Display and formatting issues in application interfaces and reports.
- Increased risk of application instability where downstream components assume length restrictions are enforced.
- Potential impact on integrations, exports, notifications, and reporting mechanisms that rely on expected field lengths.

References to Evidences and Proof of Concept:

Step 1: Navigate to the Incident Creation functionality.



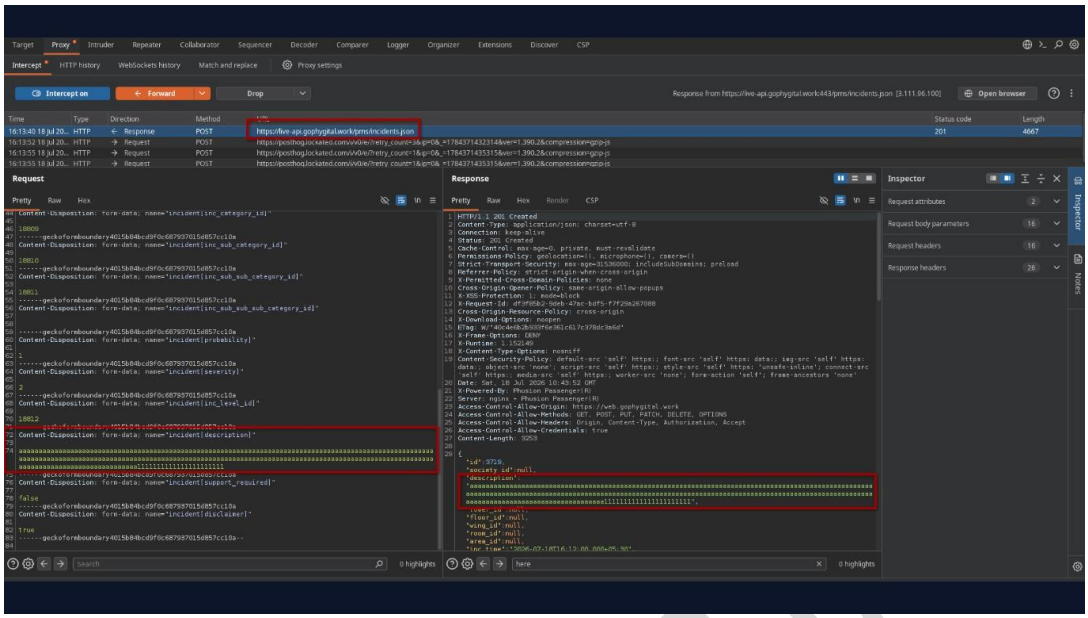
Step 2: Identify a field with a maximum character limit of 240 characters (e.g., Description field).



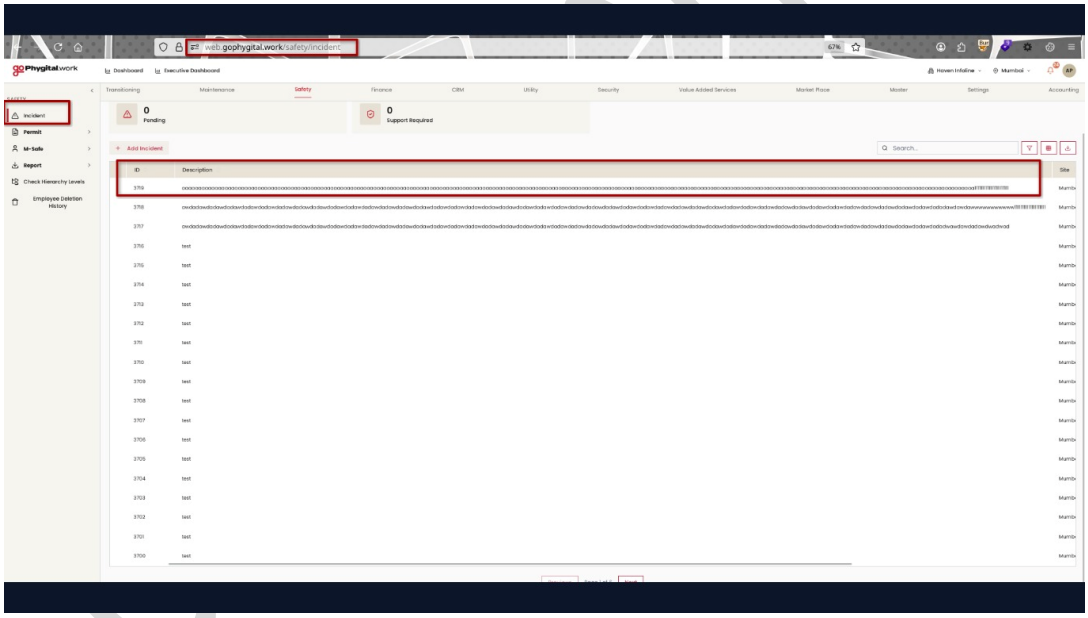
Step 3: Enter a valid value and intercept the submission request using Burp Suite.

Step 4: Modify the request by replacing the field value with a string significantly exceeding the allowed 240-character limit.

Step 5: Forward and observe that the application returns a successful response and stores the submitted data.



Step 6: Navigate to the created incident record and verify that the oversized content is displayed and persisted within the application.



Recommendations:

- Implement strict server-side validation for all input length restrictions.
- Reject requests containing values that exceed defined business limits.
- Ensure validation is enforced consistently across web interfaces, APIs, mobile applications, and import mechanisms.
- Apply centralized validation controls to prevent inconsistencies across different application components.
- Validate and sanitize all user-supplied input before processing or storage.
- Conduct a review of other input fields to identify similar validation bypass issues.

5.27 Server Version Disclosure via HTTP Response Headers

Affected Asset: https://web.gophygital.work/

Affected Parameter: HTTP Response Header (Server)

CWE-ID: CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

Severity: Low

Description:

The web server discloses its software name and exact version number through the HTTP Server response header.

During testing, the following response header was observed: HTTP/2 200 OK Server: nginx/1.28.0

This information reveals the underlying web server technology and version in use. While this disclosure does not directly result in unauthorized access, it provides attackers with valuable reconnaissance information that can be leveraged to identify known vulnerabilities, misconfigurations, or exploit paths associated with the disclosed software version.

Security best practices recommend minimizing unnecessary information exposure to reduce the attack surface and limit intelligence gathering opportunities.

Impact:

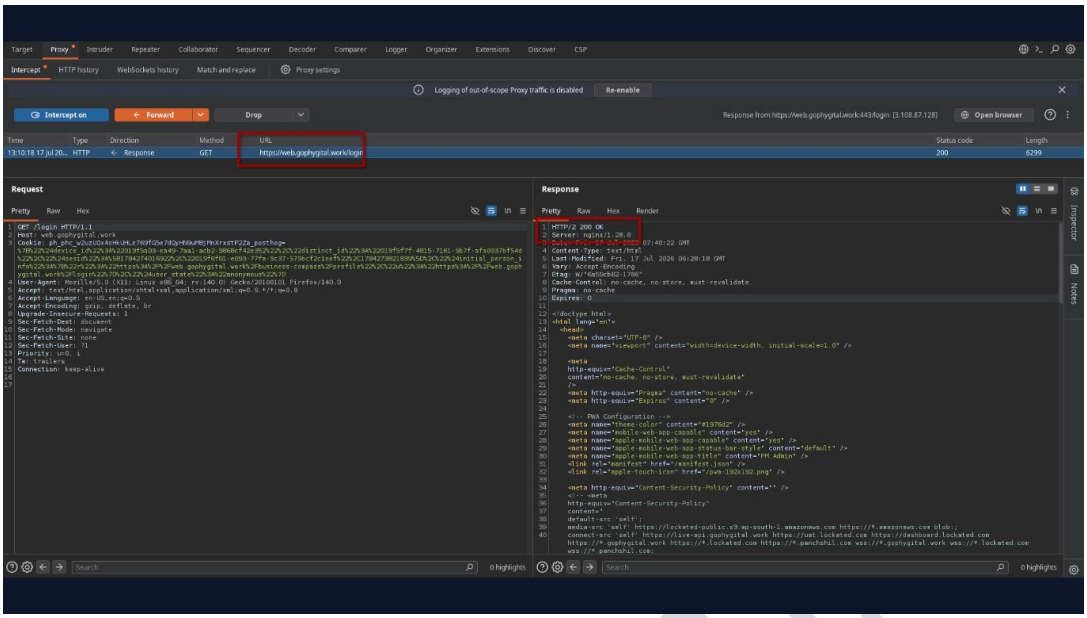
- Disclosure of underlying web server technology.
- Exposure of exact software version information.
- Assists attackers during reconnaissance and fingerprinting activities.
- May facilitate identification of publicly known vulnerabilities affecting the disclosed version.
- Provides additional information useful for targeted attacks.

References to Evidences and Proof of Concept:

Step 1: Send an HTTP request to the application.

Step 2: Inspect the HTTP response headers.

Step 3: Observe the header.



Step 4: Verify that the exact web server version is disclosed to unauthenticated users.

Recommendations:

- Remove or suppress detailed version information from HTTP response headers.
- Configure Nginx to hide version details using: server_tokens off;
- Where possible, return only a generic server banner or remove the header entirely.
- Regularly update and patch server software regardless of version disclosure settings.
- Review other response headers for unnecessary information leakage.

5.28 Username Enumeration via Distinct Password Reset Error Messages

Affected Asset:

- https://web.gophygital.work/forgot-password
- https://web.gophygital.work

Affected Parameter:

- Forget password
- Login panel

CWE-ID:

- CWE-203 – Observable Discrepancy
- CWE-204 – Observable Response Discrepancy

Severity: Low

Description:

The application is vulnerable to **Username Enumeration** because it returns different error messages based on the existence of a user account or organization. During testing, this behavior was observed at **two different locations** within the application:

1. **Login Panel** – The application returns different responses depending on whether the username/email or organization exists.
2. **Forgot Password** – The application displays specific error messages such as **"No user found"** and **"No organisation found"** when an invalid email address is submitted.

These distinguishable responses allow an attacker to determine whether a username, email address, or organization is registered with the application. An attacker can automate requests to enumerate valid accounts, which can then be used to facilitate credential stuffing, password spraying, phishing, or other targeted attacks.

Impact:

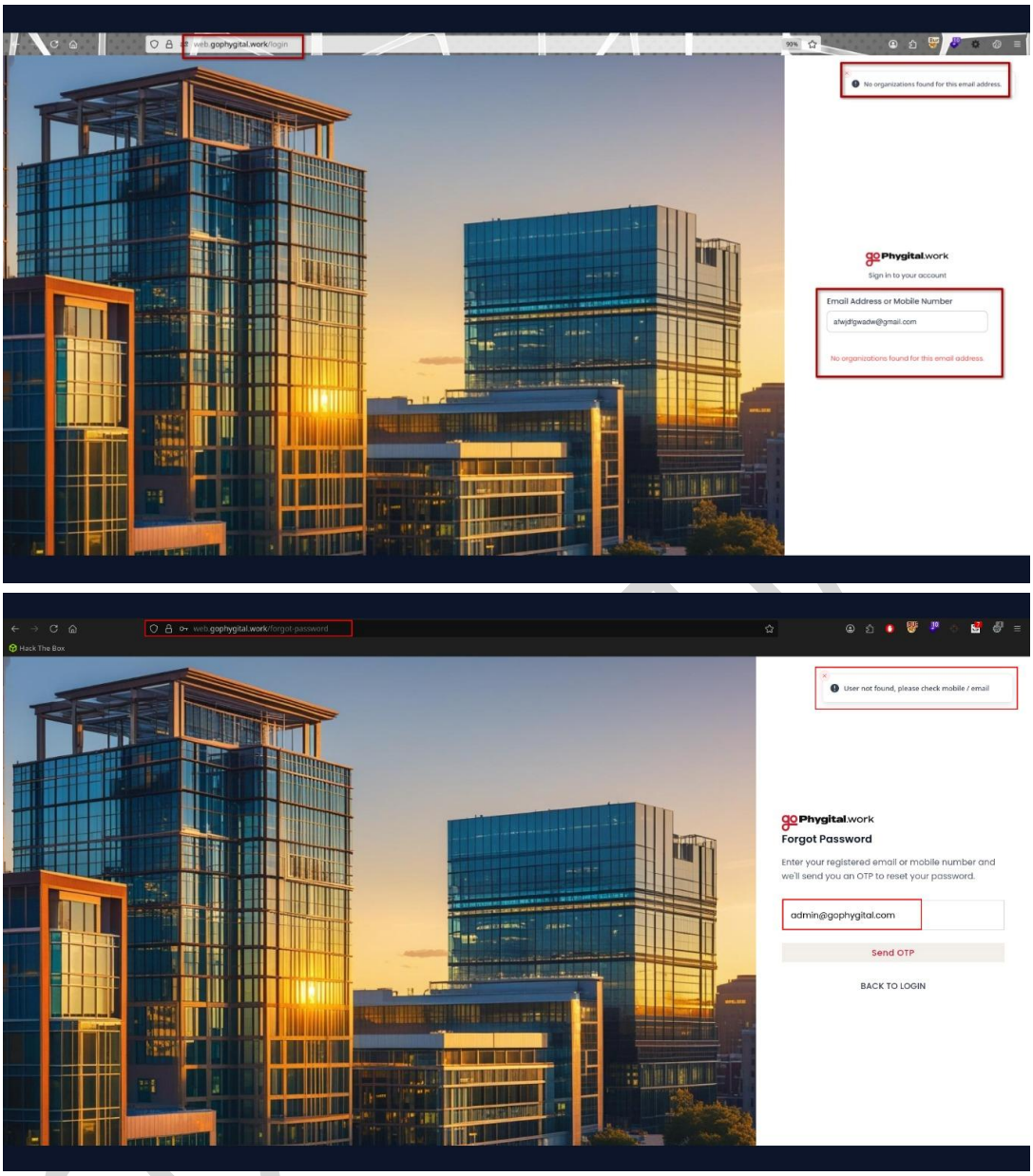
- Allows attackers to identify valid usernames, email addresses, and organizations.
- Enables automated enumeration of registered accounts.
- Increases the likelihood of successful credential stuffing and password spraying attacks.
- Assists attackers in conducting targeted phishing and social engineering attacks.
- Leaks unnecessary information about the application's registered users and organizations.

References to Evidences and Proof of Concept:

Step 1: Navigate to the application's login page.

Step 2: Submit a request using a valid username/email and an incorrect password.

Step 3: Observe the error message.



Step 4: Repeat the request using a non-existent username/email or organization.

Step 5: Observe that a different error message is returned, allowing differentiation between valid and invalid accounts.

Recommendations:

- Return a generic response for all authentication and password reset failures, for example:
  "Invalid username/email or password." (Login)
  "If an account exists for the provided email address, password reset instructions have been sent." (Forgot Password)

- Ensure that HTTP status codes, response bodies, and response times are consistent regardless of whether the account exists.
- Implement rate limiting and account lockout mechanisms to prevent automated enumeration.
- Consider using CAPTCHA after repeated failed authentication or password reset attempts.
- Log and monitor repeated login and password reset requests to detect enumeration attempts.

5.29 Account Enumeration via Authentication Response Messages

Affected Asset: https://web.gophygital.work/login

Affected Parameter: Email Address or Mobile Number

CWE-ID: CWE-203 – Observable Discrepancy

Severity: Low

Description:

The login page returns different responses based on the existence of the supplied email address or mobile number. When an invalid or unregistered email is entered, the application responds with:

"No organizations found for this email address."

This allows an unauthenticated attacker to determine whether an email address is registered with the application by observing the response message. Such behavior constitutes an **Account Enumeration** vulnerability.

An attacker can automate requests with a list of email addresses to identify valid users or organizations, which can later be leveraged in password spraying, credential stuffing, phishing, or other targeted attacks.

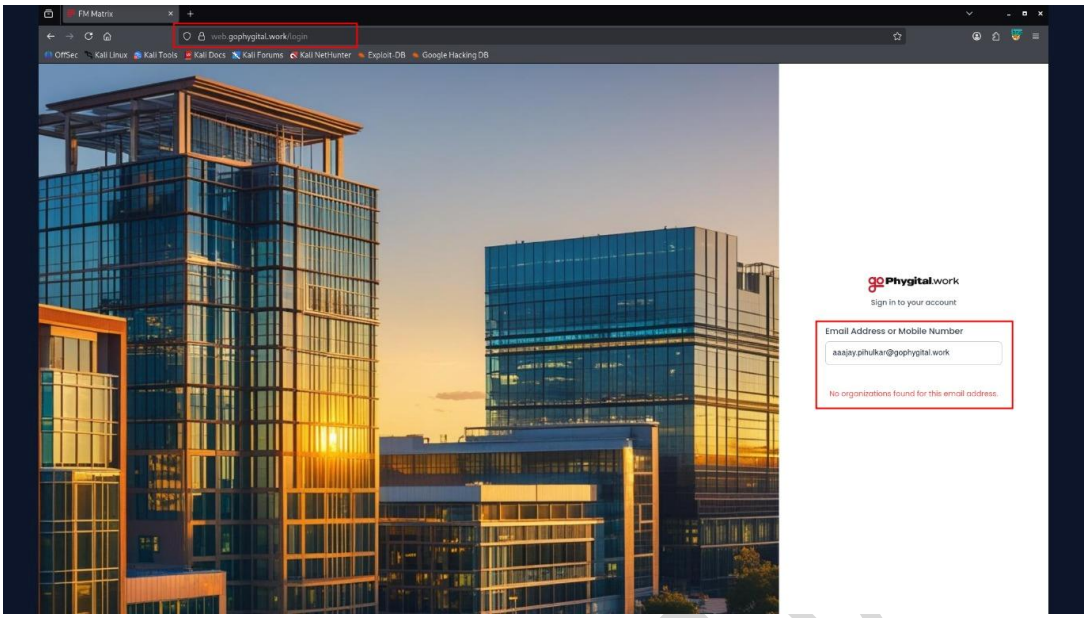
Impact:

- Allows attackers to identify valid email addresses or registered organizations.
- Facilitates password spraying and credential stuffing attacks.
- Assists attackers in targeted phishing campaigns.
- Enables user and organization reconnaissance.

References to Evidences and Proof of Concept:

Step 1: Navigate to https://web.gophygital.work/login.

Step 2: Enter an email address that is not associated with any organization (e.g., aaajay.philukar@gophygital.work).



Step 3: Observe the application response: No organizations found for this email address.

Recommendations:

- Return a generic authentication message for all login attempts, such as:
- **"If the provided credentials are valid, you will be able to continue."**
- **"Invalid email address or password."**

- Ensure identical HTTP status codes, response bodies, and response timing regardless of whether the account exists.
- Implement rate limiting and account lockout mechanisms to reduce automated enumeration attempts.
- Monitor authentication logs for high-volume login attempts targeting multiple email addresses.
- Consider adding CAPTCHA after repeated failed authentication attempts.

6. Pass Cases

6.1 Session Management: Session Token Exposure

Objective:

To verify that session identifiers and authentication tokens are securely protected and are not exposed through client-side storage mechanisms that could be accessed by malicious scripts.

Observations:

The application was assessed to determine whether session identifiers or authentication tokens were exposed through browser-accessible storage mechanisms such as document.cookie, localStorage, or sessionStorage.

Testing confirmed that:

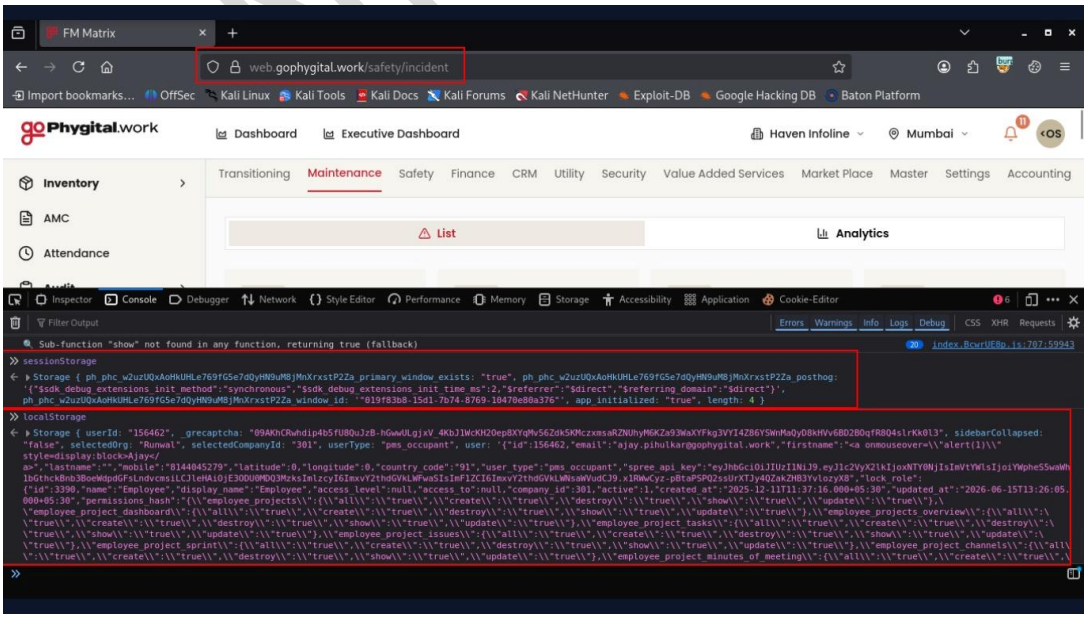
- document.cookie returned an empty value, indicating that authentication cookies are **not accessible via JavaScript** (consistent with the use of **HttpOnly** cookies).
- localStorage contained only non-sensitive application preferences (e.g., theme settings and UI table preferences).
- sessionStorage contained only non-sensitive application state information.
- No session IDs, JWTs, access tokens, refresh tokens, or other authentication credentials were found in client-side storage.

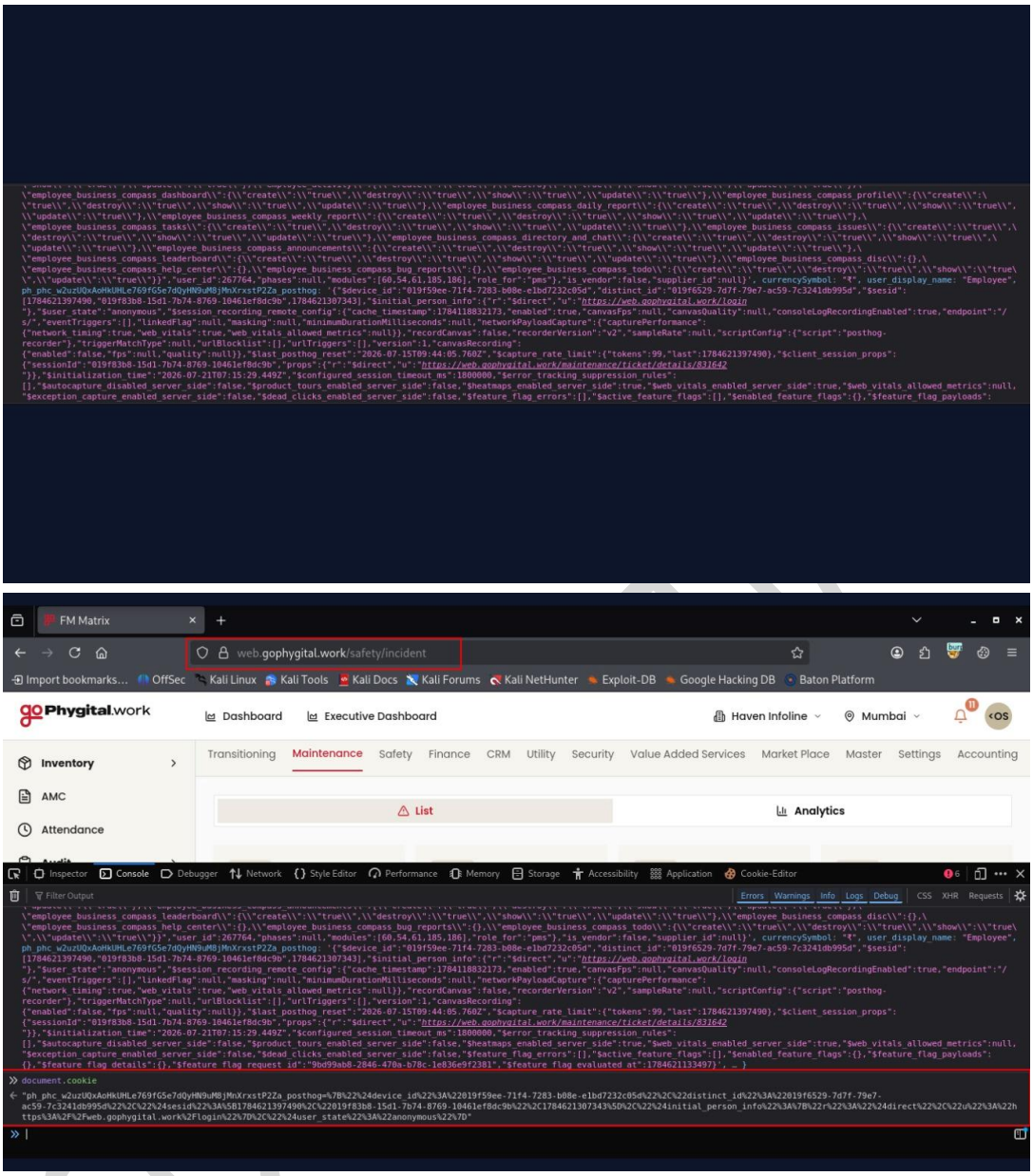
References to Evidences and Proof of Concept:

Step 1: Open this URL: https://web.gophygital.work/ and Authenticate using valid credentials

Step 2: Open the browser's Developer Tools and Navigate to the Console and Storage tabs.

Step 3: Execute the following commands: document.cookie localStorage sessionStorage





Step 4: Review the output for: Session identifiers JWTs Access tokens Refresh tokens Authentication cookies Verify whether any sensitive session information is accessible through JavaScript or browser storage.

Conclusion:

The application adequately protects session identifiers from client-side access. Authentication cookies are not exposed to JavaScript, and no sensitive session tokens were found in browser storage. No session token exposure was identified during testing. **This security control is considered to have passed.**

6.2 Authentication: Improper JWT Signature Validation

Objective:

To verify that the application validates the integrity and authenticity of JWTs by rejecting tokens with modified payloads or invalid cryptographic signatures.

Observations:

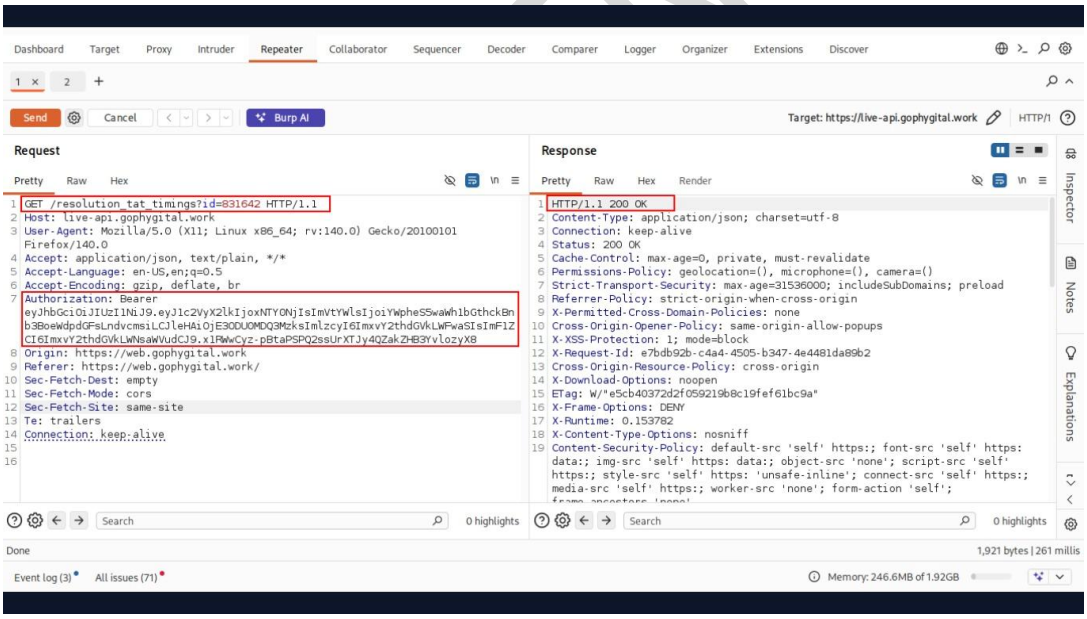
The application was tested to verify whether it properly validates the cryptographic signature of JSON Web Tokens (JWTs). A valid JWT was successfully accepted and returned the expected response (HTTP 200 OK). The JWT was then modified, resulting in an invalid signature. The application rejected the tampered token and returned **HTTP 401 Unauthorized** with the message **"Unauthorized: invalid token"**.

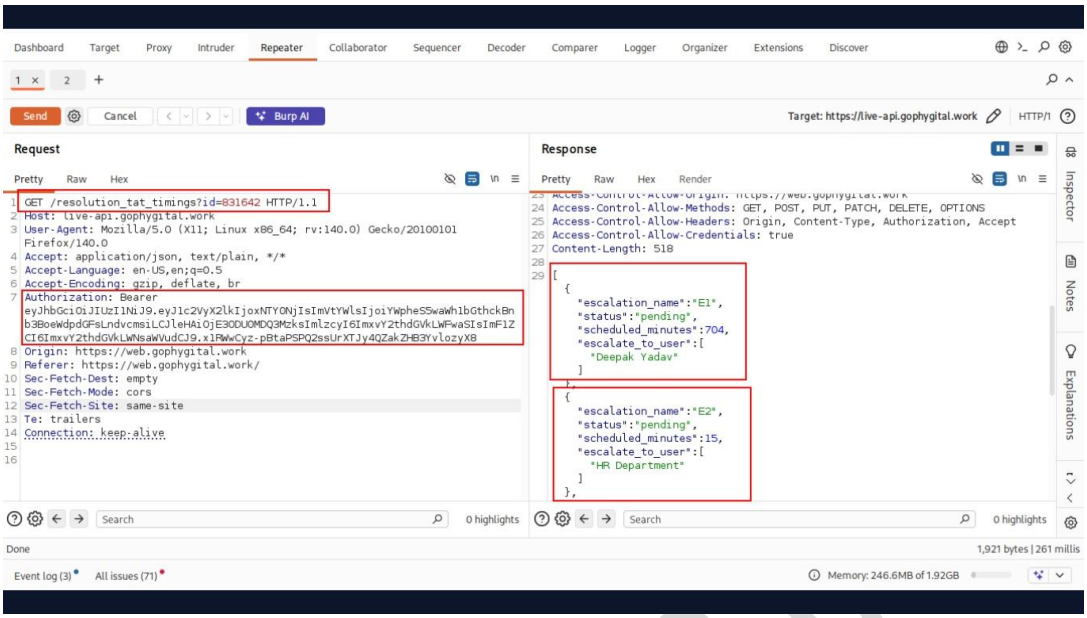
This behavior confirms that the server verifies the JWT signature before processing authenticated requests.

References to Evidences and Proof of Concept:

Step 1: Open this URL: https://web.gophygital.work/ and login with valid credentials

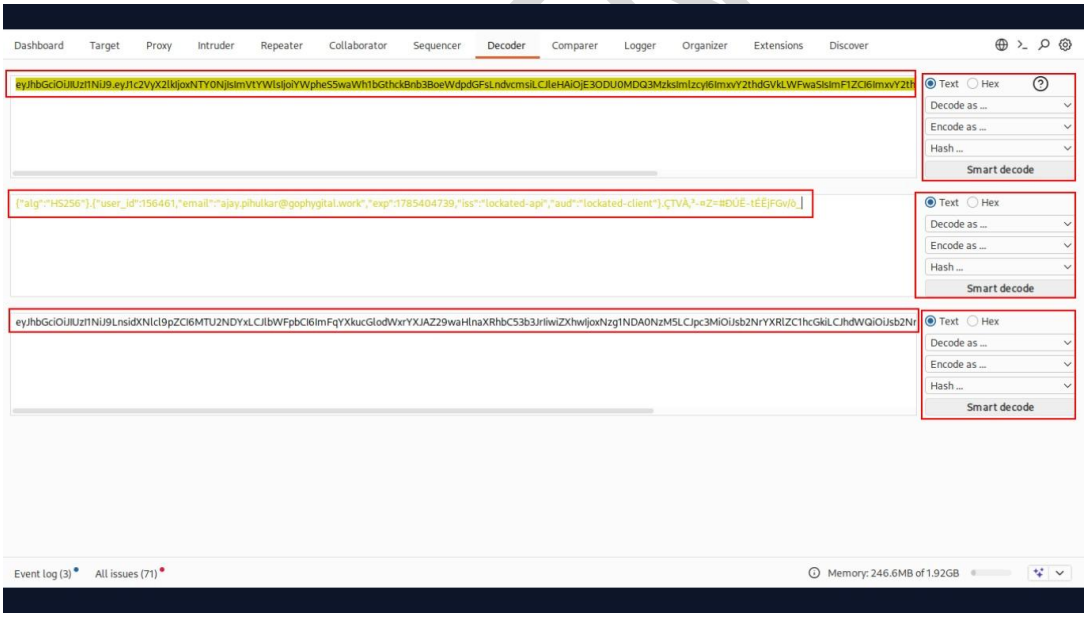
Step 2: Capture an authenticated request containing a valid JWT in the Authorisation: Bearer header. And send this to repeater



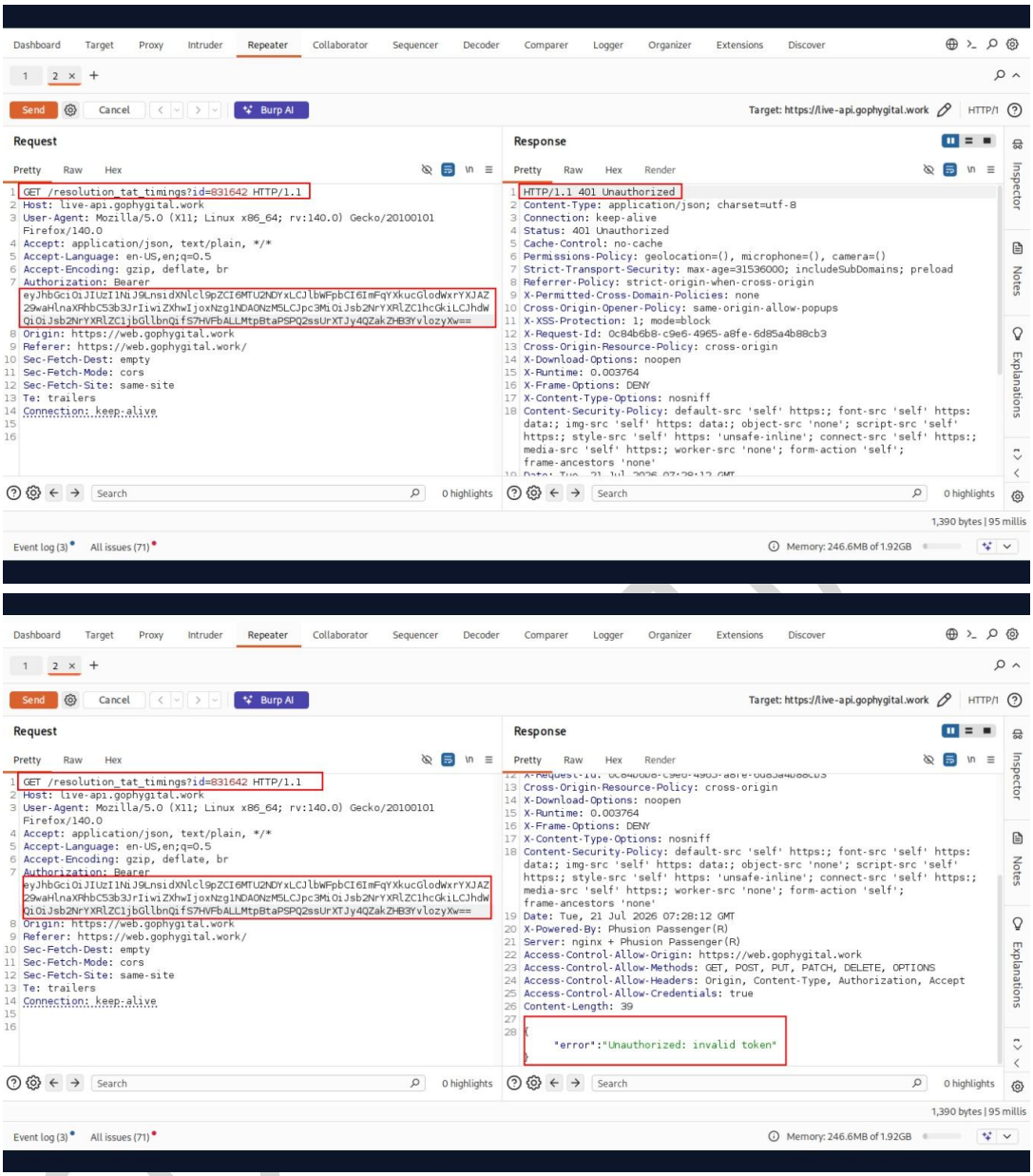


Step 3: Confirm that the request using the original JWT returns HTTP 200 OK.

Step 4: Decode the JWT and modify the payload or signature without generating a valid replacement signature.



Step 5: Replay the request using the tampered JWT.



Step 6: Observe the server response.

Conclusion:

The application correctly validates JWT signatures and rejects tokens whose signatures have been modified or invalidated. No evidence of improper JWT signature validation or authentication bypass was identified. **This security control is considered to have passed.**

6.3 Wrong Algorithms Usage Depending on Context

Objective:

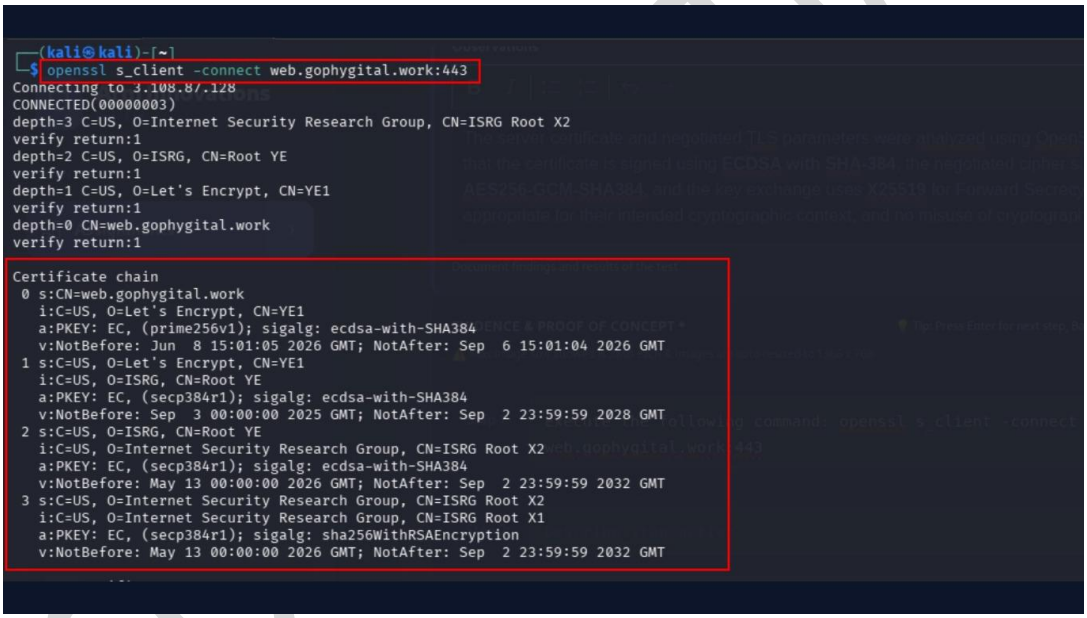
To verify that cryptographic algorithms are used appropriately according to their intended purpose and current security best practices, including certificate signatures, key exchange mechanisms, and negotiated cipher suites.

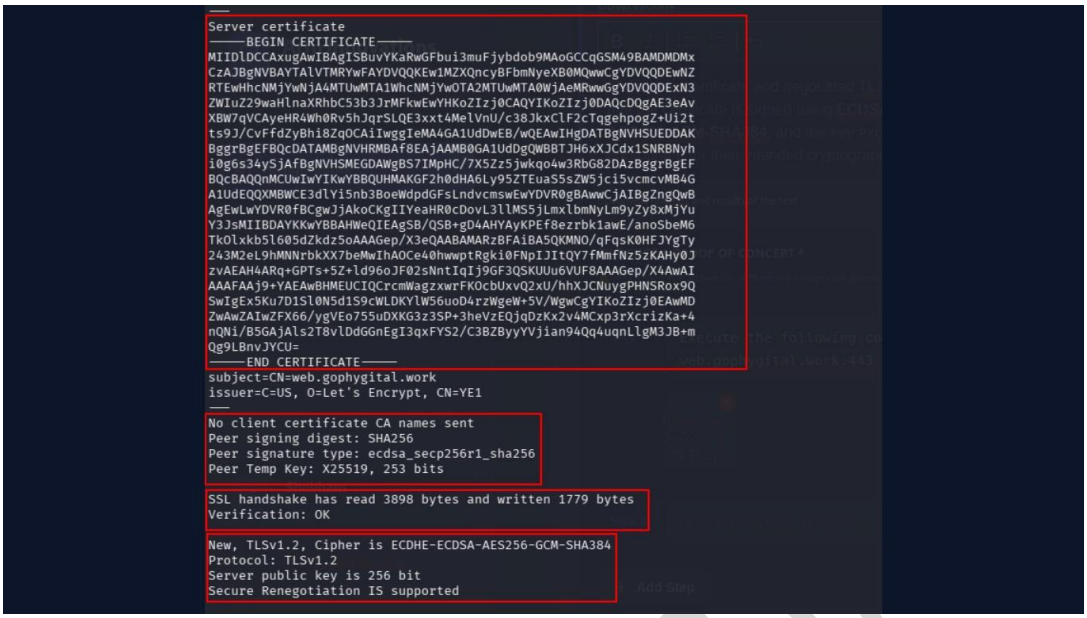
Observations:

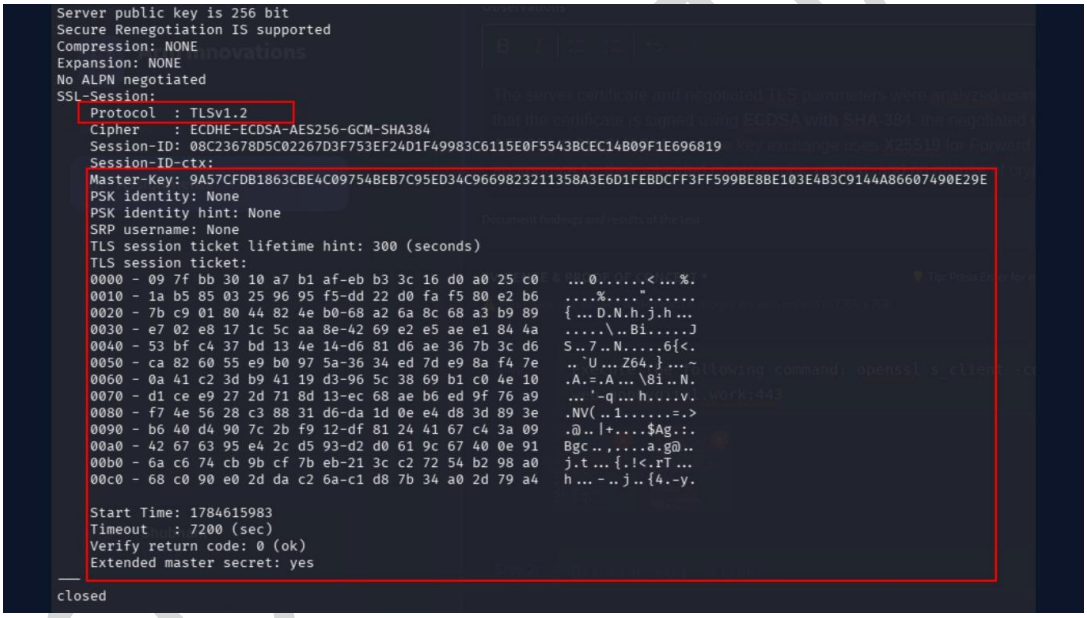
The server certificate and negotiated TLS parameters were analyzed using OpenSSL. The assessment verified that the certificate is signed using **ECDSA with SHA-384**, the negotiated cipher suite is **ECDHE-ECDSA-AES256-GCM-SHA384**, and the key exchange uses **X25519** for Forward Secrecy. These algorithms are appropriate for their intended cryptographic context, and no misuse of cryptographic primitives was identified.

References to Evidences and Proof of Concept:

Step 1: Execute the following command: openssl s_client -connect web.gophygital.work:443







Step 2: Review the following information: Protocol version Negotiated cipher suite Certificate signature algorithm Key exchange algorithm

Step 3: Verify that secure and context-appropriate cryptographic algorithms are used.

Conclusion:

The server uses appropriate cryptographic algorithms for certificate signing, key exchange, and encrypted communication. The negotiated TLS configuration follows current security best practices, and no inappropriate algorithm usage was identified.

6.4 Weak Algorithms Usage

Objective:

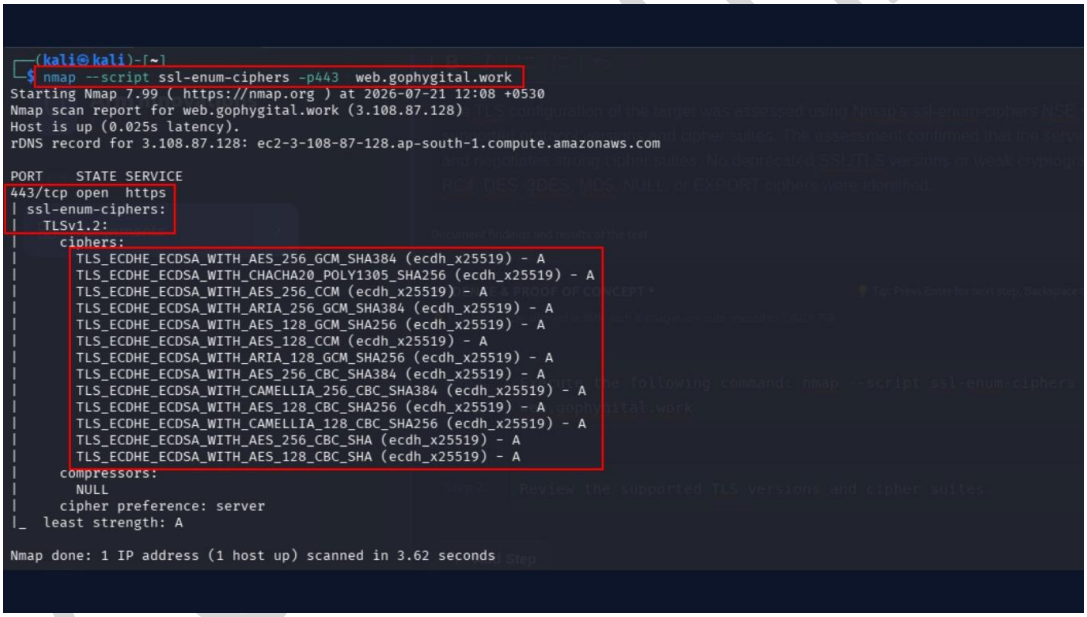
To verify that the application does not support weak or deprecated cryptographic protocols and cipher suites that could compromise the confidentiality and integrity of encrypted communications.

Observations:

The TLS configuration of the target was assessed using Nmap's ssl-enum-ciphers NSE script to identify supported protocol versions and cipher suites. The assessment confirmed that the server supports only **TLS 1.2** and negotiates strong cipher suites. No deprecated SSL/TLS versions or weak cryptographic algorithms such as RC4, DES, 3DES, MD5, NULL, or EXPORT ciphers were identified.

References to Evidences and Proof of Concept:

Step 1: Execute the following command: nmap --script ssl-enum-ciphers -p443 web.gophygital.work



Step 2: Review the supported TLS versions and cipher suites.

Step 3: Verify that deprecated protocols and weak cipher suites are not supported.

Conclusion:

The server is configured with strong TLS cipher suites and does not support deprecated protocols or weak cryptographic algorithms. No weak algorithm usage was identified during the assessment.

6.5 Sensitive Data Encryption at Rest and In Transit

Objective:

To verify that:

- Sensitive information is encrypted during transmission using TLS.
- Sensitive information stored on the server is encrypted (where applicable).

Observations:

The application's network traffic was captured and analyzed using Wireshark. The TLS handshake was successfully established before any application data was exchanged. Subsequent traffic was transmitted as encrypted **TLS Application Data** over HTTPS, indicating that data is protected while in transit.

The capture does not provide visibility into how data is stored on the server; therefore, encryption of **data at rest** could not be verified through this test.

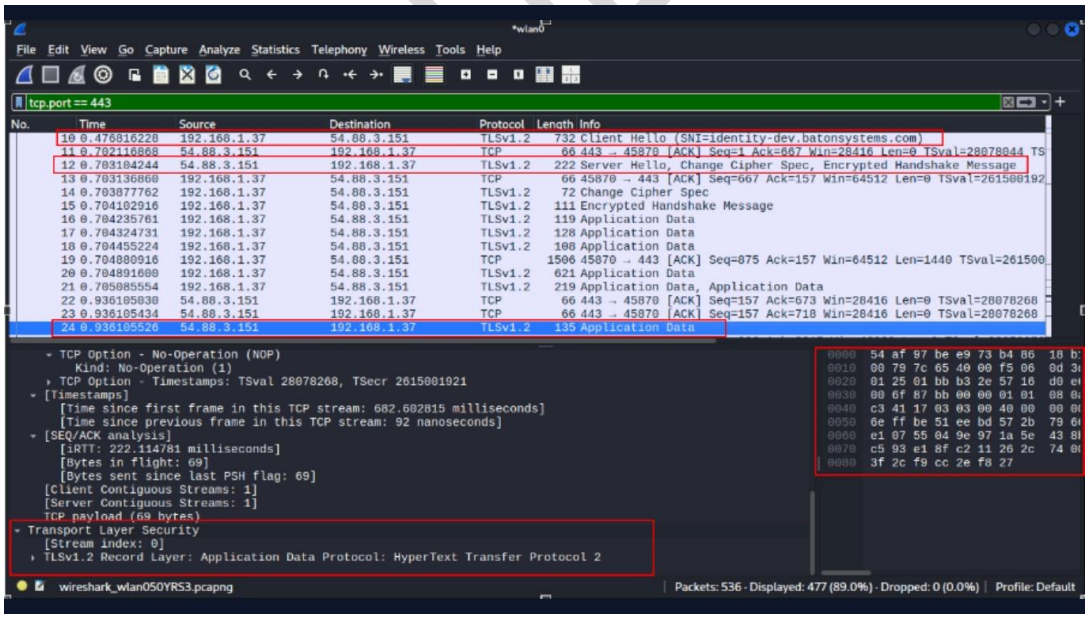
References to Evidences and Proof of Concept:

Step 1: Launch Wireshark and begin capturing network traffic.

Step 2: Open this url: https://web.gophygital.work/ and login with valid credentials

Step 3: Apply a filter such as: tcp.port == 443

Step 4: Verify that: A TLS handshake (Client Hello / Server Hello) occurs. Application traffic is transmitted as encrypted TLS Application Data



Step 5: Inspect packets to confirm that sensitive information is not visible in plaintext.

Conclusion:

The application properly encrypts data **in transit** using TLS, protecting sensitive information during network communication. Based on the available evidence, **no plaintext transmission of sensitive data was observed**. However, **encryption of data at rest cannot be confirmed from network traffic alone and requires additional server-side assessment.**

6.6 Check for Brute Force Protection

Objective:

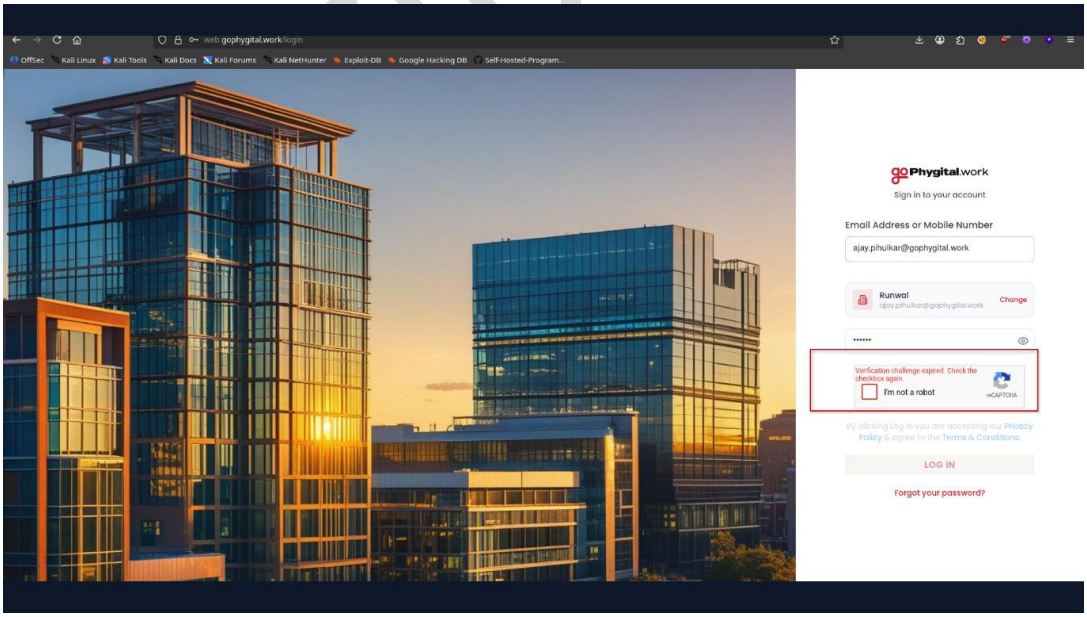
To verify whether the application implements adequate protection mechanisms against brute force attacks by preventing automated or repeated authentication attempts through CAPTCHA enforcement or other server-side security controls.

Observations:

- The application's login functionality was assessed for resistance against brute force attacks.
- During authentication testing, the application required users to complete a CAPTCHA challenge before processing login attempts.
- Automated login attempts using Burp Suite were unable to proceed without providing a valid CAPTCHA response.
- The CAPTCHA was validated by the server, preventing repeated automated authentication requests from being processed successfully.
- The implemented CAPTCHA mechanism significantly increased the difficulty of conducting credential guessing and brute force attacks through automated tools.
- No evidence was found that the CAPTCHA protection could be bypassed during testing, and no weakness in the brute force protection mechanism was identified.

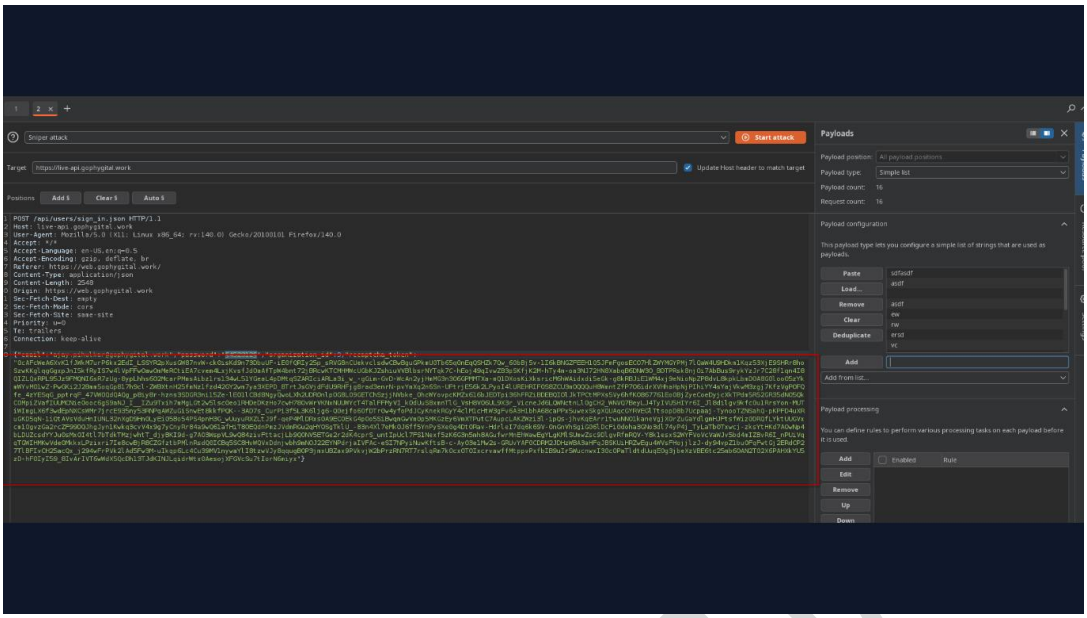
References to Evidences and Proof of Concept:

Step 1: Navigate to the application's login page. https://web.gophygital.work/login



Step 2: Intercept the authentication request using Burp Suite.

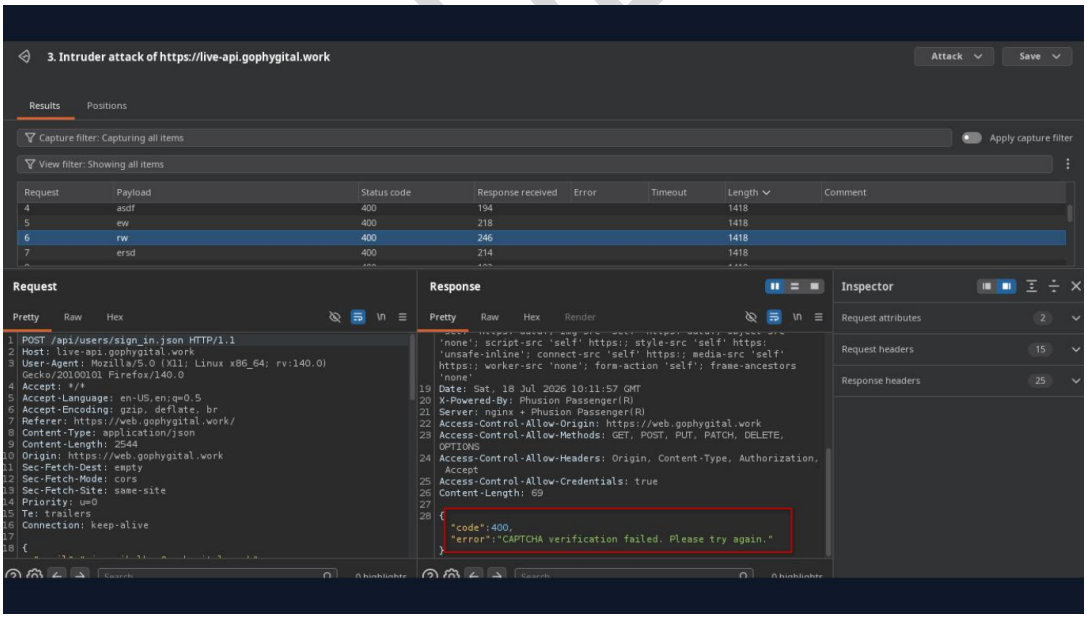
Step 3: Attempt to perform multiple automated login requests using Burp Suite Intruder with different username and password combinations.



Step 4: Observe whether the application enforces CAPTCHA validation before processing authentication requests.

Step 5: Verify whether authentication requests without a valid CAPTCHA are rejected by the server.

Step 6: Confirm that automated brute force attempts cannot proceed without successfully solving the CAPTCHA challenge.



Conclusion:

The application's authentication mechanism was tested to evaluate its resistance against brute force attacks. During testing, the application consistently enforced CAPTCHA validation before processing authentication requests, preventing automated login attempts from being executed successfully. No evidence of CAPTCHA bypass or ineffective brute force protection was identified during the assessment. Based on the testing performed, the application implements

effective protection against automated brute force attacks, and **no vulnerability was identified in this test case**.

6.7 Check for account lockout

Objective:

To verify that the application implements an effective account lockout mechanism to protect user accounts from brute-force attacks. The test ensures that repeated failed login attempts trigger account lockout, preventing unauthorized access through password guessing.

Observations:

- The application allows a limited number of consecutive failed login attempts.
- After five consecutive invalid password attempts, the account is automatically locked.
- An appropriate account lockout message is displayed to the user.
- The account cannot be accessed even when valid credentials are entered while the lockout is active.
- The implemented account lockout mechanism effectively mitigates brute-force attacks.

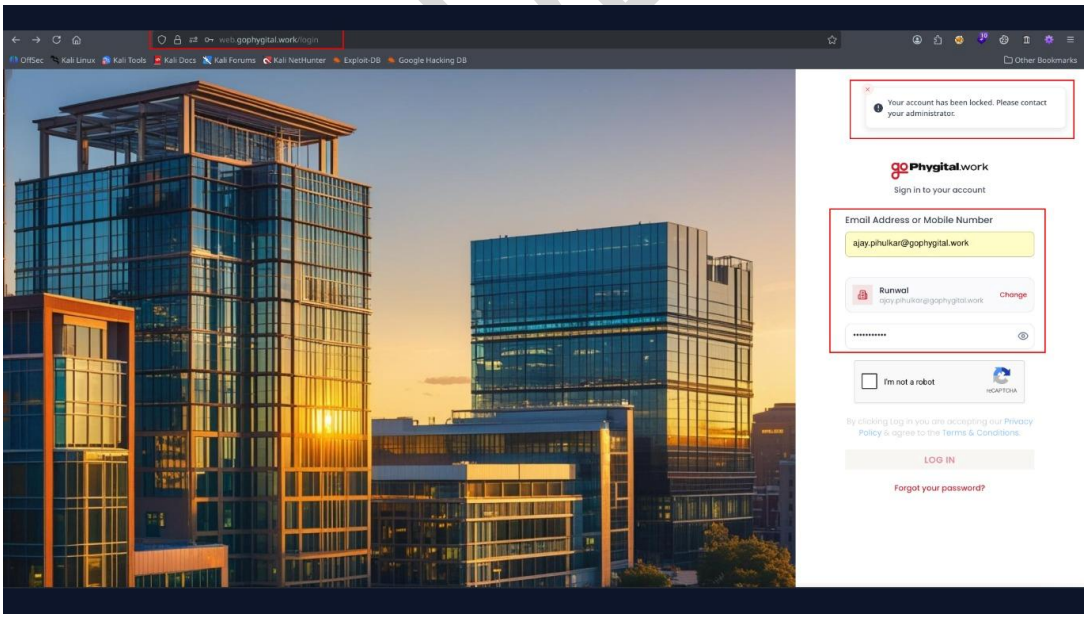
References to Evidences and Proof of Concept:

Step 1: Navigate to the application login page.

Step 2: Enter a valid email address and invalid password.

Step 3: Repeat the login attempt with an invalid password five consecutive times.

Step 4: Verify that the application locks the account after the fifth failed login attempt and displays an appropriate account lockout message.



Conclusion:

The application correctly implements an account lockout mechanism by locking the account after five consecutive failed login attempts, effectively reducing the risk of brute-force attacks.

6.8 Check credentials only delivered over HTTPS

Objective:

To verify that user credentials are transmitted only over secure HTTPS connections, ensuring they are protected from interception during authentication.

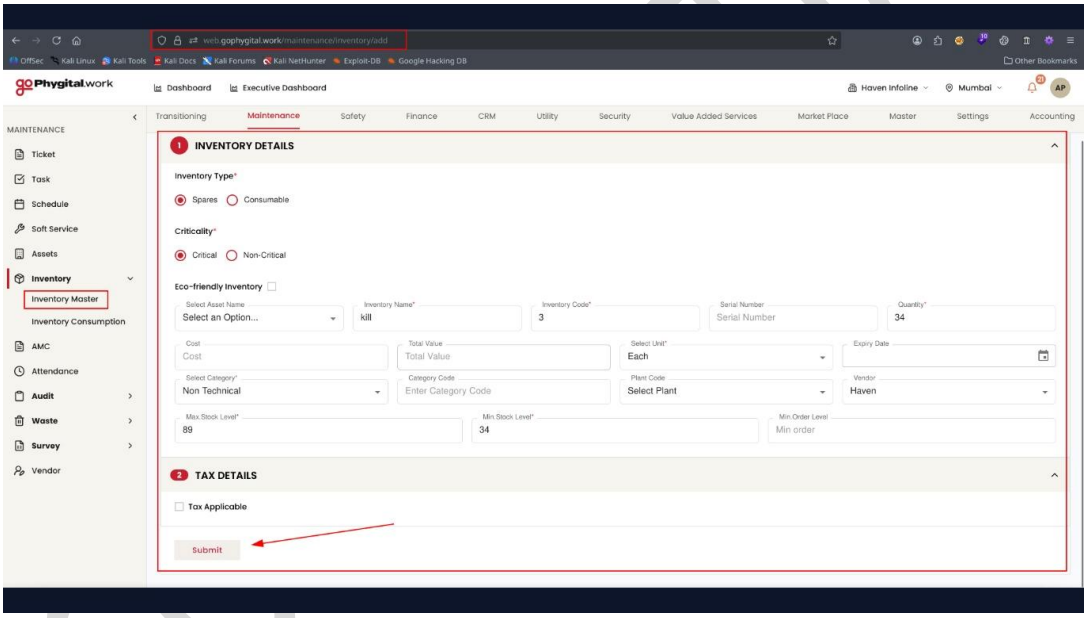
Observations:

- User credentials were submitted through a secure HTTPS connection.
- The request was transmitted over HTTPS using HTTP/1.1.
- No credentials were observed being transmitted over an unencrypted HTTP connection.
- The application ensures that authentication data is encrypted during transmission.

References to Evidences and Proof of Concept:

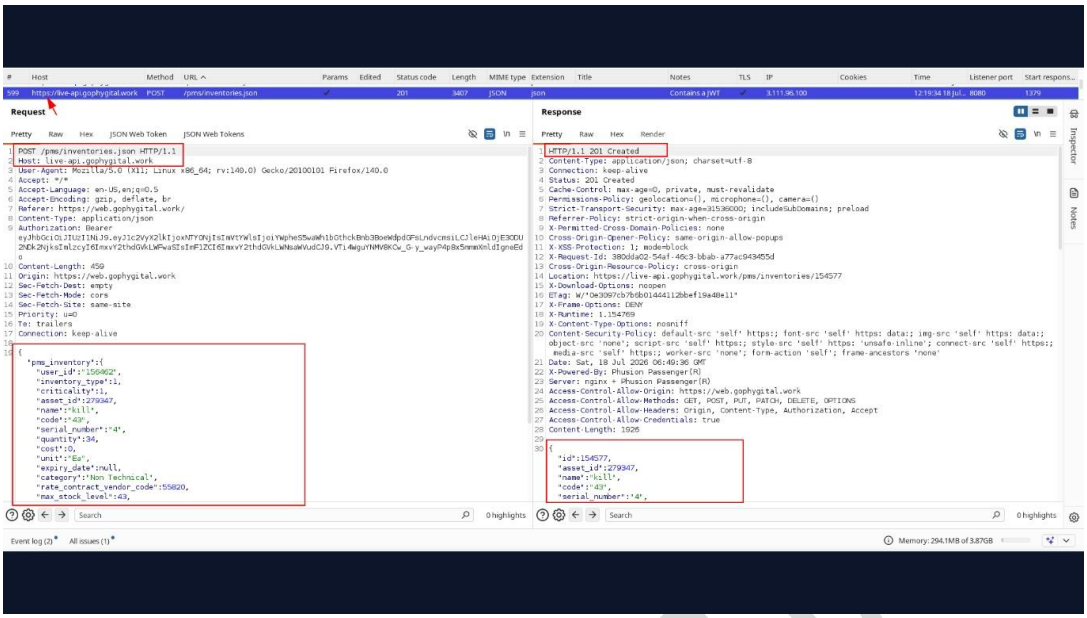
Step 1: Access the application login page.

Step 2: Enter valid credentials in the Inventory Master form.



Step 3: Intercept the login request using Burp Suite.

Step 4: Verify that the Inventory Master Details request is transmitted over the HTTPS protocol.



Step 5: Confirm that the request is sent over an encrypted HTTPS (HTTP/1.1) connection.

Conclusion:

The application securely transmits user credentials over HTTPS, ensuring that authentication data is encrypted during transit. No insecure transmission of credentials over HTTP was observed during testing.

6.9 Check for Format String Vulnerability

Objective:

To verify whether the application improperly interprets user-supplied input as format string specifiers, which could lead to unintended behavior, information disclosure, application crashes, or other security issues.

Observations:

- The application was tested for Format String vulnerabilities by submitting common format string test inputs through multiple user-controlled input fields.
- Testing was performed on the **Search** field available on the **Tasks** page:

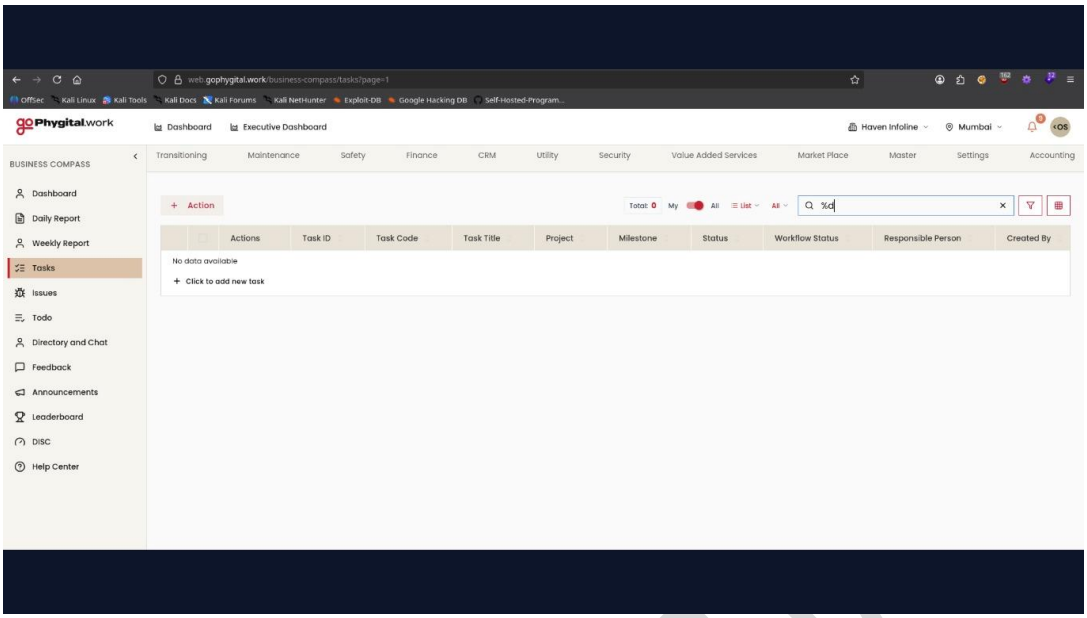
Additional testing was performed on the **Username** field available on the **Profile** page:

- Similar testing was also conducted across other available input fields within the application.
- The submitted format string test values were accepted and stored or displayed as plain text where applicable.
- No format specifiers were interpreted or processed by the application.
- No unexpected behavior, application errors, server exceptions, crashes, or information disclosure were observed during testing.
- The application consistently treated the supplied input as ordinary text across all tested input fields.
- No Format String vulnerability was identified during testing.

References to Evidences and Proof of Concept:

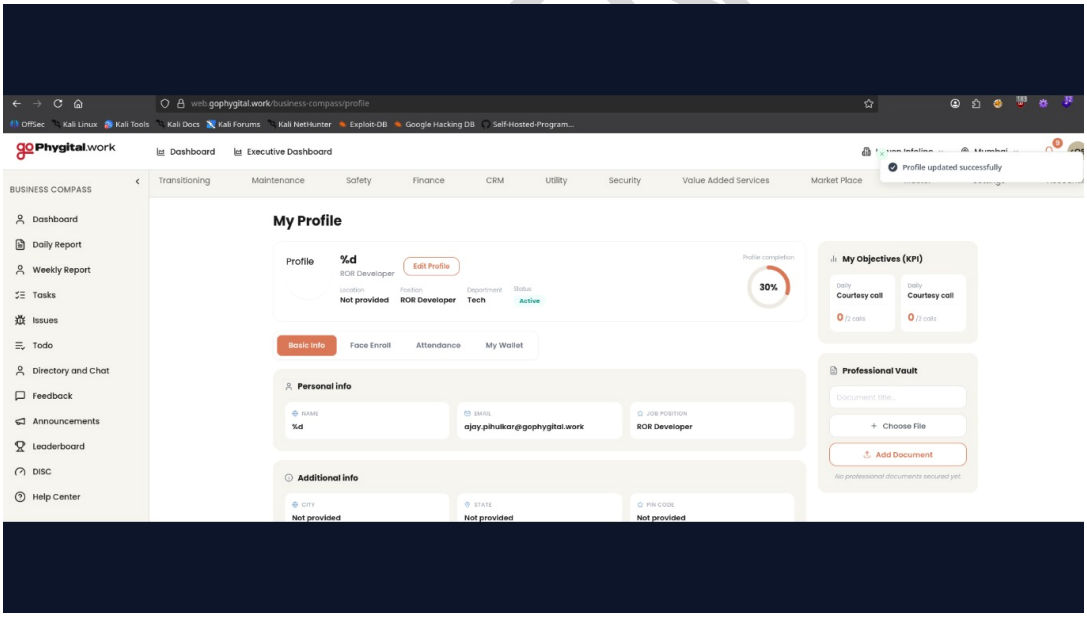
Step 1: Navigate to the Tasks page: https://web.gophygital.work/business-compass/tasks?page=1

Step 2: Enter common format string test values into the Search field and observe the application's response.



Step 3: Navigate to the Profile page: https://web.gophygital.work/business-compass/profile

Step 4: Update the Username field using common format string test inputs and save the changes.



Step 5: Repeat the same testing across other available user input fields within the application.

Step 6: Observe whether the application interprets any format string specifiers, generates server errors, discloses internal information, or exhibits unexpected behavior.

Step 7: Verify that all submitted inputs are treated as plain text and do not affect the application's normal functionality.

Conclusion:

The application was assessed for Format String vulnerabilities by submitting common format string test inputs through the Search field, Username field, and other available user-controlled

input fields. All submitted values were processed as plain text and were not interpreted as format specifiers. No application errors, information disclosure, crashes, or abnormal behavior were observed during testing. Based on the testing performed, the application is not vulnerable to Format String Injection, and no security issue was identified in this test case.

6.10 Check for HTTP Verb Tampering

Objective:

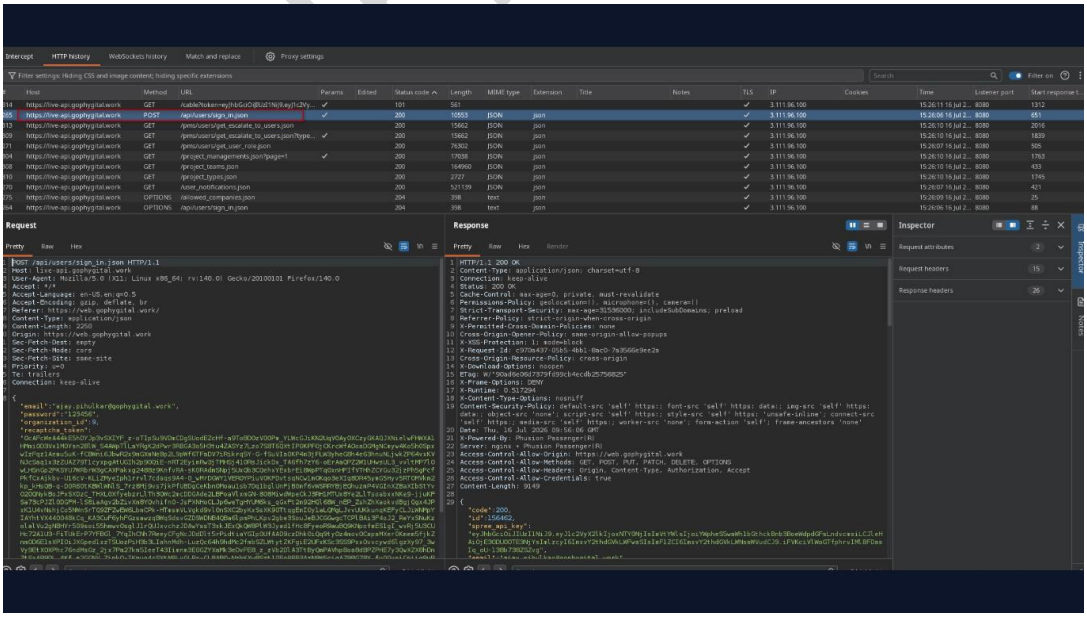
To verify whether the application properly enforces HTTP methods for its endpoints and prevents unauthorized access or unintended functionality by rejecting requests that use unsupported or unexpected HTTP verbs.

Observations:

- The application was tested for HTTP Verb Tampering by modifying the HTTP method of valid application requests.
- The login endpoint /api/users/sign_in.json, which normally accepts a **POST** request, was selected for testing.
- The original **POST** request was modified to use other HTTP methods, including **GET**, **PUT**, **DELETE**, **PATCH**, **HEAD**, and **OPTIONS** where applicable.
- The application responded with **HTTP 404 Not Found** or otherwise rejected the modified requests, indicating that unsupported HTTP methods were not accepted for the endpoint.
- Similar testing was performed on other accessible application endpoints.
- The tested endpoints consistently rejected requests using unsupported HTTP methods and did not perform any unintended actions.
- No authentication bypass, authorization bypass, or unexpected application behavior was observed during testing.
- No HTTP Verb Tampering vulnerability was identified.

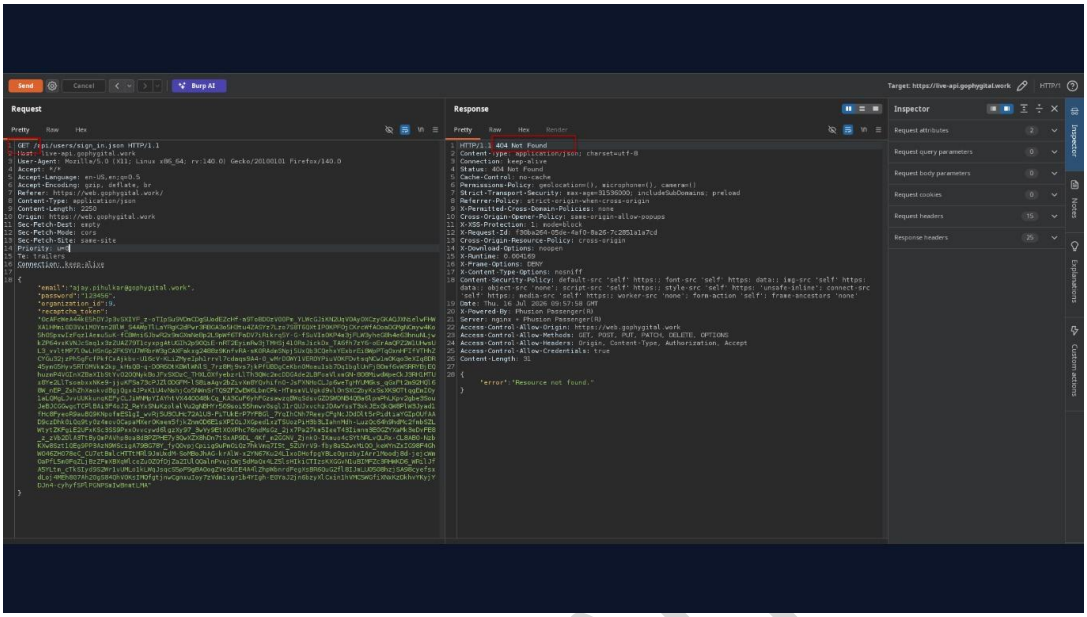
References to Evidences and Proof of Concept:

Step 1: Intercept a valid POST request to the login endpoint: /api/users/sign_in.json



Step 2: Send the original request to Burp Suite Repeater.

Step 3: Modify the HTTP method from POST to alternative methods such as GET, PUT, DELETE, PATCH, HEAD, and OPTIONS.



Step 4: Send each modified request and observe the server response.

Step 5: Verify that the application rejects unsupported HTTP methods and does not process the request.

Step 6: Repeat the same testing against other accessible application endpoints.

Step 7: Confirm that no unauthorized functionality, authentication bypass, or unintended server-side action occurs when unsupported HTTP methods are used.

Conclusion:

The application was tested for HTTP Verb Tampering by modifying valid requests to use unsupported HTTP methods. The login endpoint (/api/users/sign_in.json) and other tested endpoints consistently rejected requests that used unexpected HTTP verbs, returning **HTTP 404 Not Found** or otherwise refusing to process the request. No unauthorized functionality, authentication bypass, or unintended server-side processing was observed. Based on the testing performed, the application correctly enforces supported HTTP methods, and no HTTP Verb Tampering vulnerability was identified.

6.11 Check Session Token Randomness

Objective:

To verify whether the application generates unique and sufficiently unpredictable Bearer tokens for authenticated sessions, reducing the risk of session prediction, token guessing, or unauthorized session impersonation.

Observations:

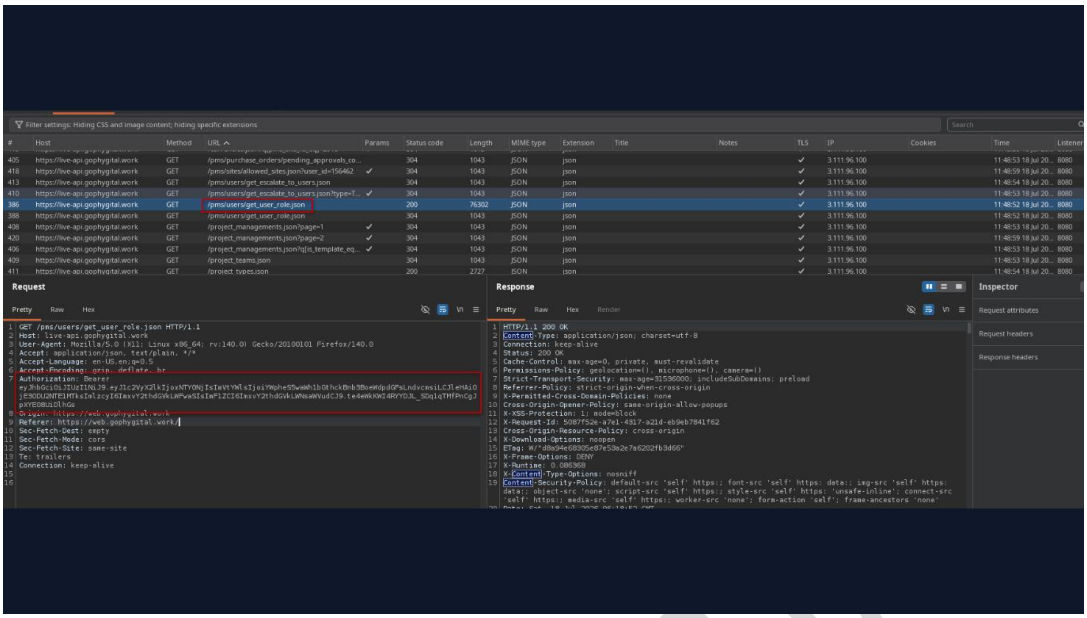
- The application uses **Bearer Token** authentication for accessing protected API endpoints after successful user authentication.
- Multiple authentication sessions were established, and the Bearer tokens issued by the application were collected and compared.
- The Bearer tokens associated with the authenticated endpoint **/pms/users/get_user_role.json** were analyzed.
- Each authentication session generated a unique JWT Bearer token.
- Comparison of the issued tokens showed no evidence of token reuse or obvious sequential patterns.
- The JWT payload contained standard authentication claims, while each token was protected by a valid signature.
- Based on the observed behavior, no indication of predictable or weak session token generation was identified during testing.

References to Evidences and Proof of Concept:

Step 1: Navigate to the application's login page and authenticate using valid user credentials. https://web.gophygital.work/login

Step 2: Access authenticated functionality and intercept a request containing the Bearer Token.

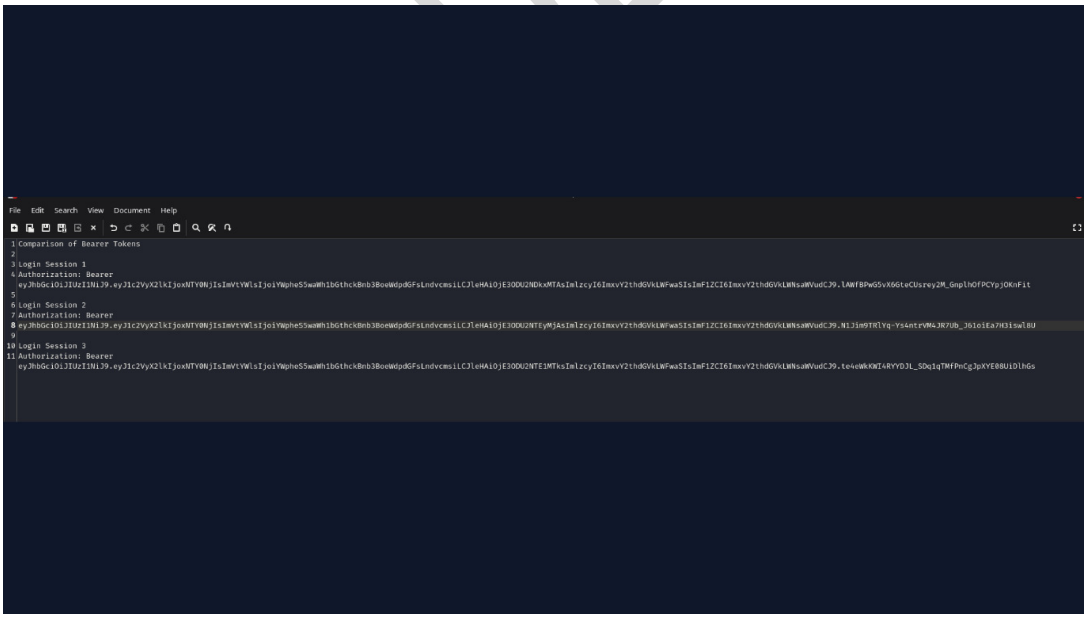
Step 3: Capture the request for the endpoint /pms/users/get_user_role.json.



Step 4: Repeat the authentication process multiple times to obtain Bearer tokens from separate authenticated sessions.

Step 5: Compare the collected Bearer tokens for uniqueness, observable patterns, and consistency.

Step 6: Verify that each authenticated session receives a distinct Bearer token and that no obvious token prediction or reuse is observed.



Conclusion:

The application's session token generation mechanism was evaluated by comparing multiple Bearer tokens obtained from separate authenticated sessions through the **/pms/users/get_user_role.json** endpoint. Each authentication session generated a distinct JWT Bearer token, and no evidence of token reuse or obvious predictability was observed during

testing. Based on the assessment performed, the application appears to generate unique session tokens for authenticated users, and **no vulnerability was identified in this test case**.

6.12 Mass Assignment Validation

Objective:

To verify that the application is protected against Mass Assignment vulnerabilities by ensuring that users cannot modify or assign security-sensitive attributes through client-controlled requests or responses. The assessment focused on validating that privileged fields such as role and isAdmin are ignored or rejected by the server unless the requesting user is explicitly authorized.

Observations:

Testing was performed by intercepting authenticated requests and manually adding privileged parameters such as role: "admin" and isAdmin: true to the request body. Additional testing involved modifying privilege-related fields in the server response before it was processed by the client.

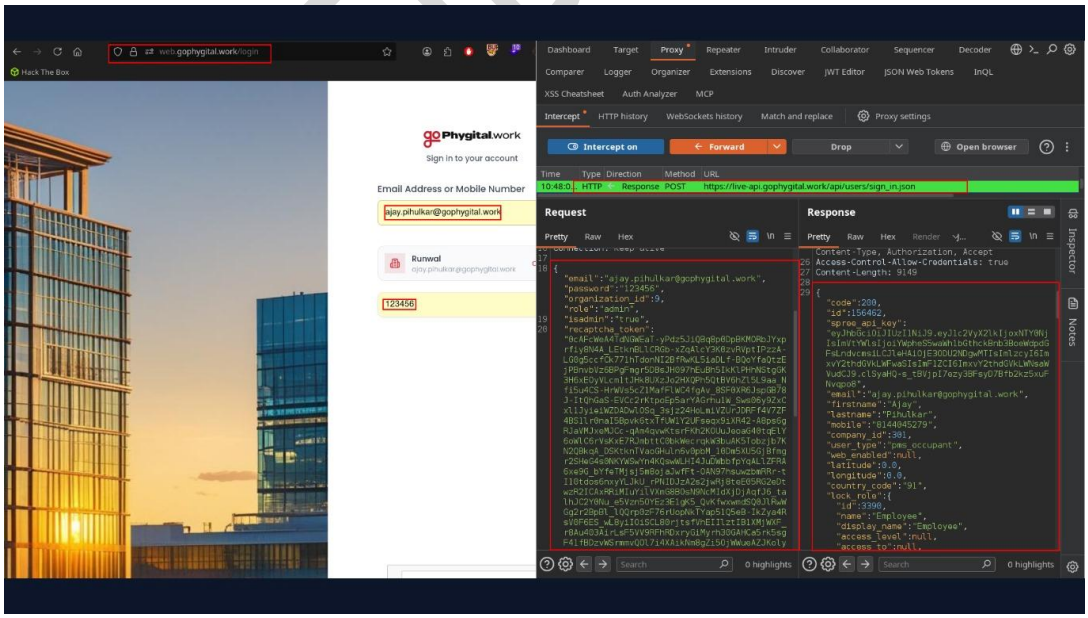
The application correctly enforced server-side validation and authorization. The injected parameters were ignored (or the request was rejected), and the user's privileges remained unchanged. Similarly, modifying privilege-related values in the response did not result in any unauthorized access or privilege escalation because the server continued to enforce authorization independently of client-side data.

References to Evidences and Proof of Concept:

Step 1: Log in as a low-privileged user.

Step 2: Intercept a user update or profile request using Burp Suite.

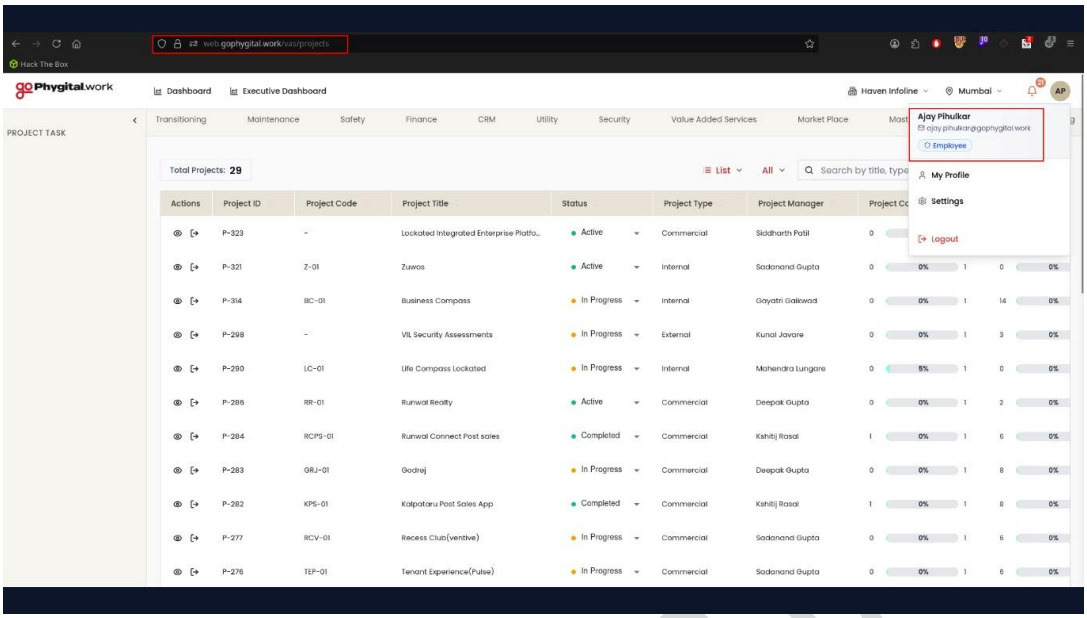
Step 3: Add privileged parameters such as: { "role": "admin", "isAdmin": true }



Step 4: Forward the modified request to the server.

Step 5: Verify that the server ignores the unauthorized fields or rejects the request.

Step 6: Intercept the server response and modify privilege-related fields before they reach the browser.



Step 7: Attempt to access administrative functionality.

Step 8: Verify that no additional privileges are granted and access controls continue to be enforced by the server.

Conclusion:

The application is **not vulnerable to Mass Assignment**. Server-side validation and authorization correctly prevent users from modifying security-sensitive attributes such as user roles or administrative flags through client-controlled requests or manipulated responses. Authorization decisions are enforced on the backend, preventing privilege escalation even when client-side data is tampered with.

6.13 Check For NULL/Invalid Session Token (Bearer Token Validation)

Objective:

To verify whether the application properly validates Bearer tokens by rejecting requests containing missing, null, or invalid authentication tokens and preventing unauthorized access to protected resources.

Observations:

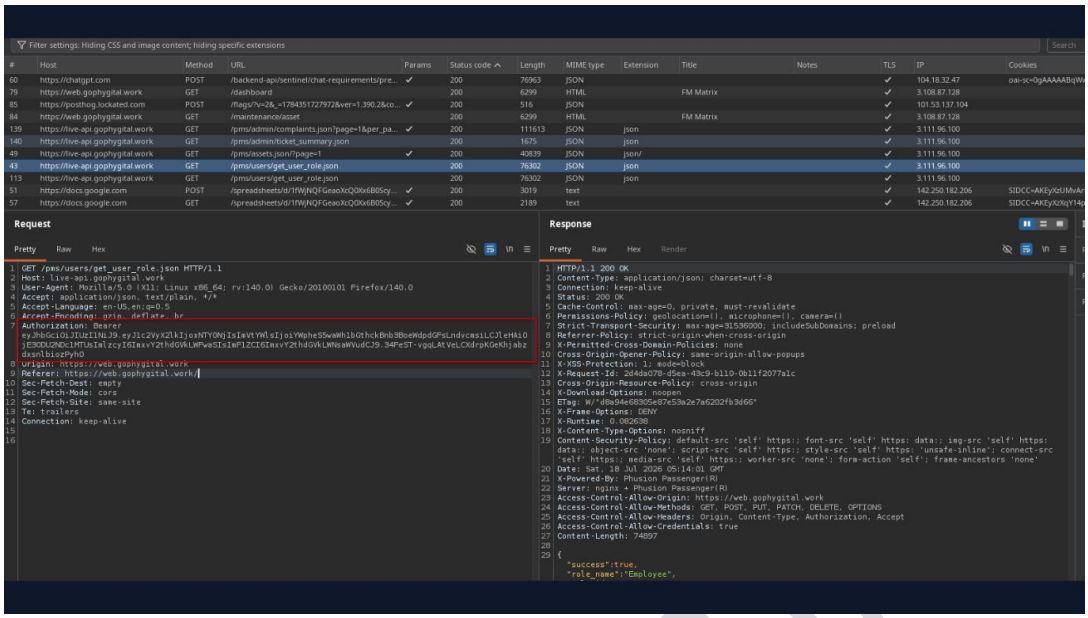
- The application uses **Bearer Token** authentication for accessing protected API endpoints after successful user authentication.
- An authenticated API request containing a valid Bearer token was intercepted using Burp Suite.
- The **Authorization** header containing the Bearer token was removed from the intercepted request.
- The modified request was sent to the server for processing.
- The server responded with **HTTP 401 Unauthorized**, indicating that the request was rejected due to the absence of a valid authentication token.
- The same test was repeated against multiple authenticated API endpoints, and each request consistently returned **HTTP 401 Unauthorized** when the Bearer token was removed.
- No protected API endpoint was accessible without a valid Bearer token, demonstrating proper server-side authentication enforcement.

References to Evidences and Proof of Concept:

Step 1: Navigate to the application's login page and authenticate using valid user credentials. https://web.gophygital.work/login

Step 2: Access authenticated functionality to generate requests containing a valid Bearer Token.

Step 3: Intercept an authenticated API request using Burp Suite. Tested Endpoint: /pms/users/get_user_role.json

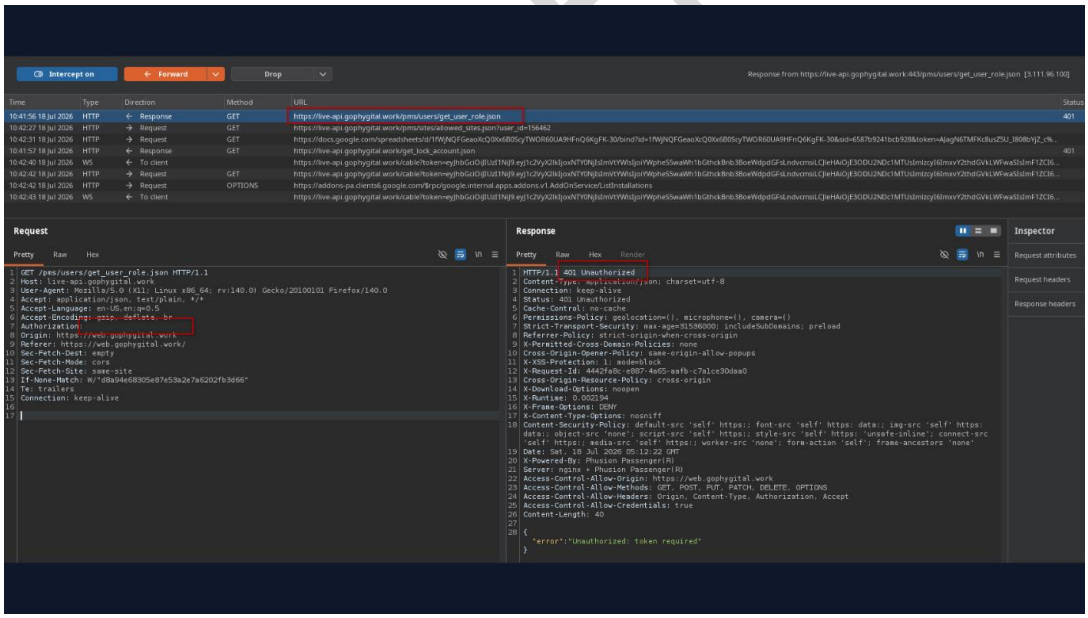


Step 4: Send the intercepted request to Burp Repeater.

Step 5: Remove the Authorization: Bearer header from the request.

Step 6: Send the modified request to the server.

Step 7: Verify that the server rejects the request by returning HTTP 401 Unauthorized.



Step 8: Repeat the same test on additional authenticated API endpoints and confirm that all protected endpoints consistently reject requests without a valid Bearer token.

Conclusion:

The application was tested to verify server-side validation of Bearer tokens by removing the **Authorization** header from authenticated API requests. The tested endpoint (/pms/users/get_user_role.json) and all other authenticated endpoints consistently returned **HTTP 401 Unauthorized** when the Bearer token was omitted. This demonstrates that protected

resources are inaccessible without valid authentication credentials and that the application correctly enforces authentication on protected APIs. **No vulnerability was identified in this test case.**

6.14 Check for OTP Flooding

Objective:

To verify that the application implements adequate security controls to prevent OTP flooding by restricting excessive OTP generation requests. The test ensures that repeated OTP requests are rate-limited, preventing abuse and protecting the OTP delivery mechanism from misuse.

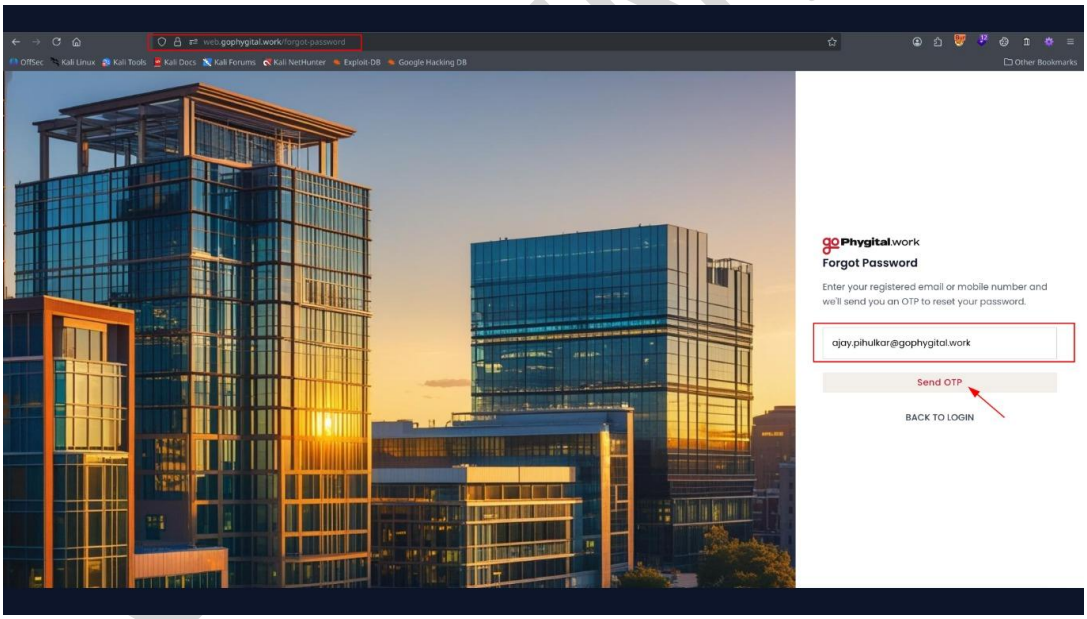
Observations:

- The application initially accepted a limited number of OTP requests.
- After the configured threshold was reached, the server responded with "Too Many Requests" for subsequent requests.
- No additional OTPs were generated after rate limiting was triggered.
- The application successfully enforced server-side rate limiting on the OTP generation endpoint.

References to Evidences and Proof of Concept:

Step 1: Navigate to the Forgot Password page.

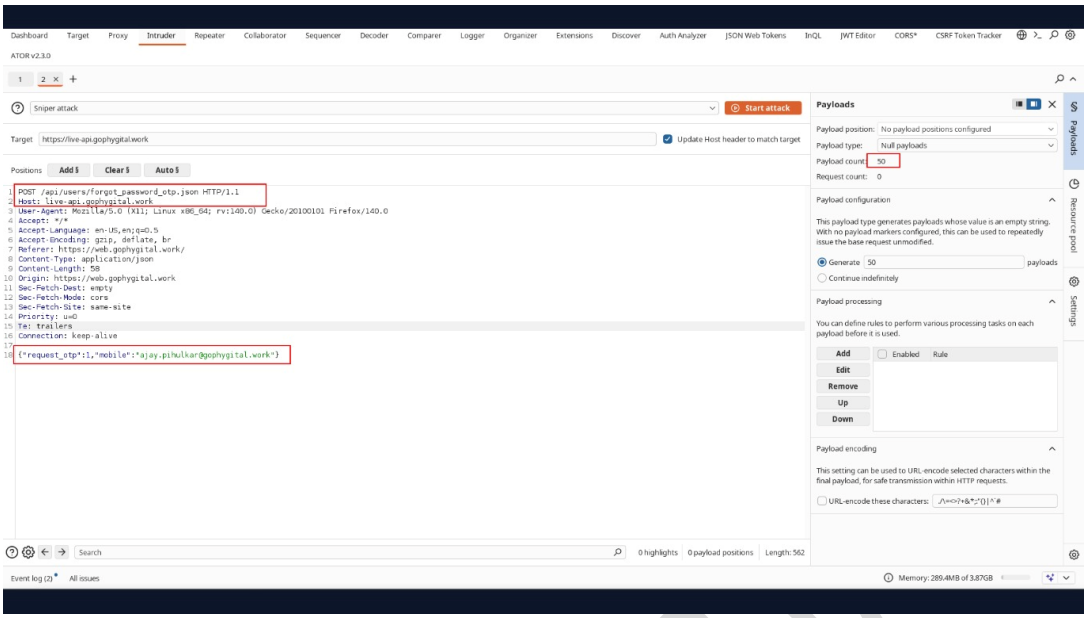
Step 2: Enter a valid registered email address and click send OTP.



Step 3: Intercept the OTP generation request using Burp Suite.

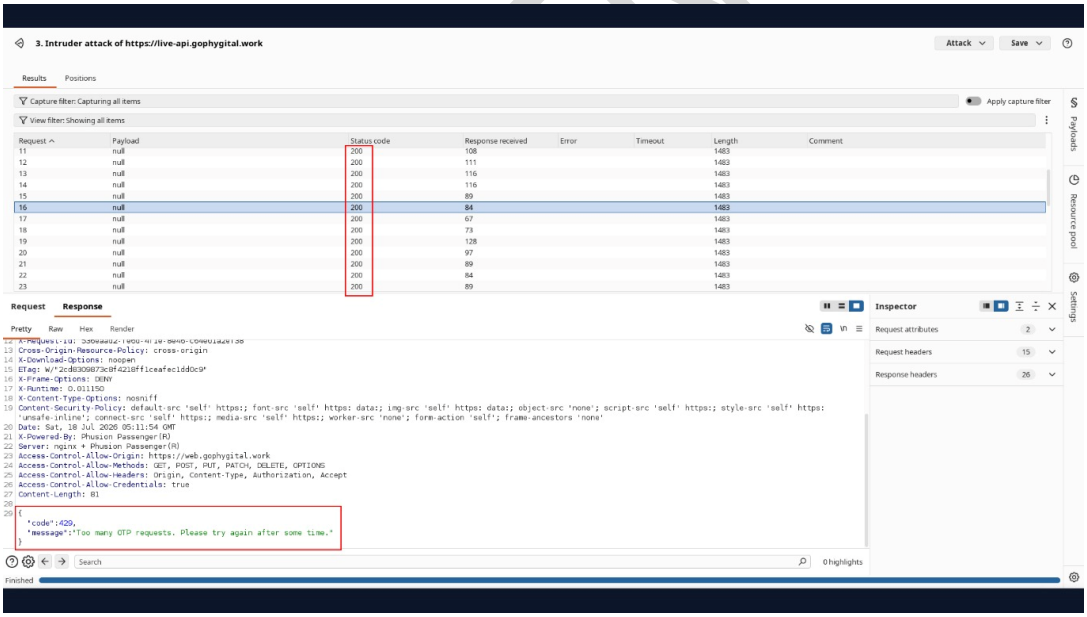
Step 4: Send the captured request to Intruder.

Step 5: Configure a Null Payload attack with 50 requests.



Step 6: Start the Intruder attack.

Step 7: Monitor the server responses and verify that, after multiple requests, the application returns a "Too Many Requests" response.



Step 8: Confirm that additional OTP generation requests are blocked.

Conclusion:

The application successfully enforces rate limiting on OTP generation requests. Excessive OTP requests are blocked with a "Too Many Requests" response, preventing OTP flooding attacks.

6.15 HTTP Parameter Pollution (HPP)

Objective:

To verify whether the application securely handles duplicate HTTP parameters and prevents HTTP Parameter Pollution attacks that could lead to authentication bypass, parameter manipulation, or unintended application behavior.

Observations:

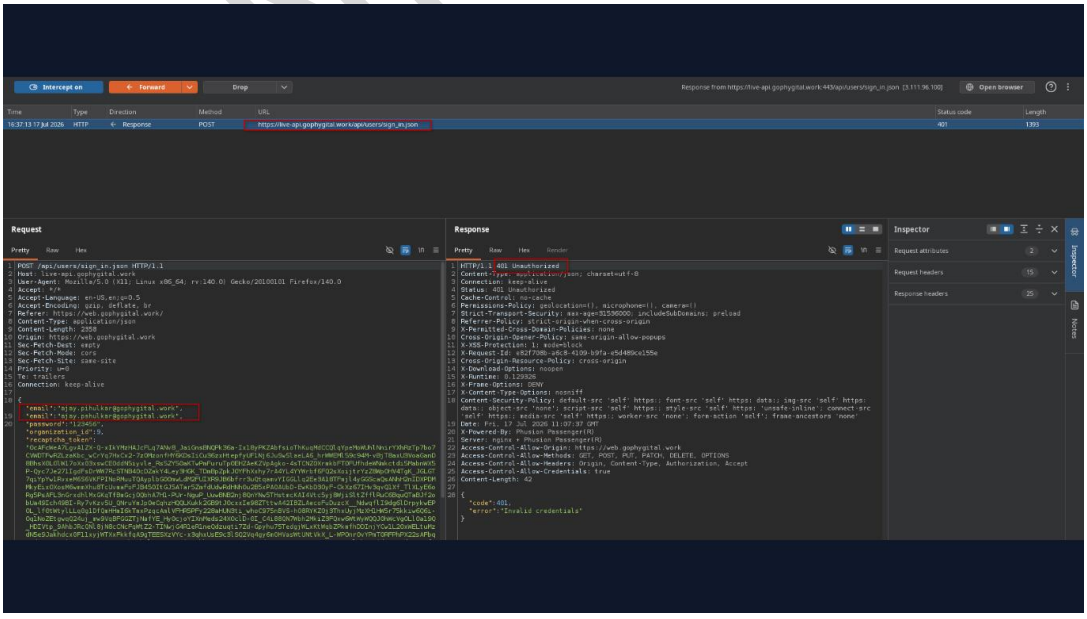
- The application's login functionality was tested for HTTP Parameter Pollution by intercepting the authentication request using Burp Suite.
- The intercepted request contained login credentials in **JSON** format.
- The **email** parameter was duplicated within the JSON request while keeping the remaining request structure unchanged.
- The modified request was forwarded to the server for processing.
- The server responded with **HTTP 401 Unauthorized**, indicating that the malformed request was not accepted for authentication.
- No evidence was observed that duplicate parameters altered the authentication logic, bypassed security controls, or resulted in unauthorized access.
- The application handled the modified request securely and did not exhibit behavior indicative of HTTP Parameter Pollution vulnerabilities.

References to Evidences and Proof of Concept:

Step 1: Navigate to the application's login page. https://web.gophygital.work/login

Step 2: Capture the login request using Burp Suite Proxy.

Step 3: Duplicate the email parameter within the intercepted JSON request while preserving the remaining request structure.



Step 4: Send the modified request to the server.

Step 5: Observe the server response and verify whether the duplicate parameter affects the authentication process.

Step 6: Confirm that the application rejects the manipulated request and does not allow authentication or unexpected behavior due to duplicate parameters.

Conclusion:

The application was assessed for HTTP Parameter Pollution by introducing a duplicate **email** parameter into the intercepted **JSON** authentication request. The server responded with **HTTP 401 Unauthorized**, demonstrating that the modified request was not accepted for authentication. No evidence was found that duplicate parameters could influence the application's authentication logic, bypass security controls, or manipulate request processing. Based on the testing performed, the application was found to handle duplicate JSON parameters securely, and **no vulnerability was identified in this test case**.

6.16 Session Termination After Logout

Objective:

To verify whether the application properly terminates authenticated user sessions after logout by invalidating the active session and preventing access to previously authenticated pages without re-authentication.

Observations:

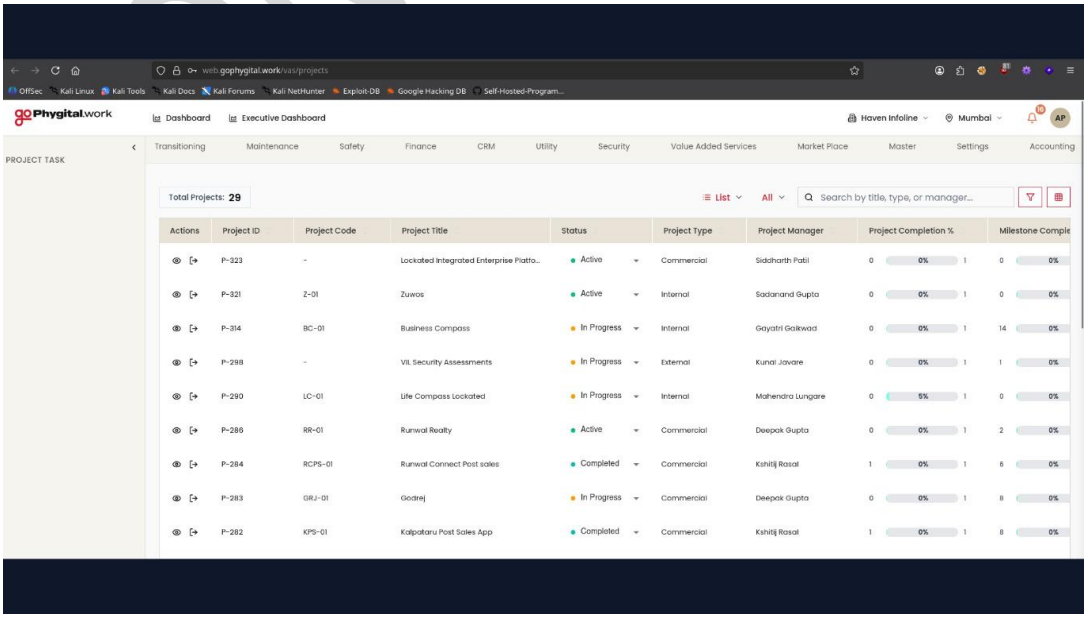
- The logout functionality of the application was tested after establishing a valid authenticated session.
- After successful authentication, an authenticated page was accessed to confirm that the session was active.
- The user then logged out using the application's logout functionality.
- After logout, the browser's **Back** button was used in an attempt to revisit the previously authenticated page.
- The application consistently redirected the user to the login page instead of displaying the authenticated content.
- No previously authenticated pages were accessible after logout, indicating that the session had been successfully terminated.
- Based on the observed behavior, the application properly invalidates user sessions following logout and prevents unauthorized access using the browser history.

References to Evidences and Proof of Concept:

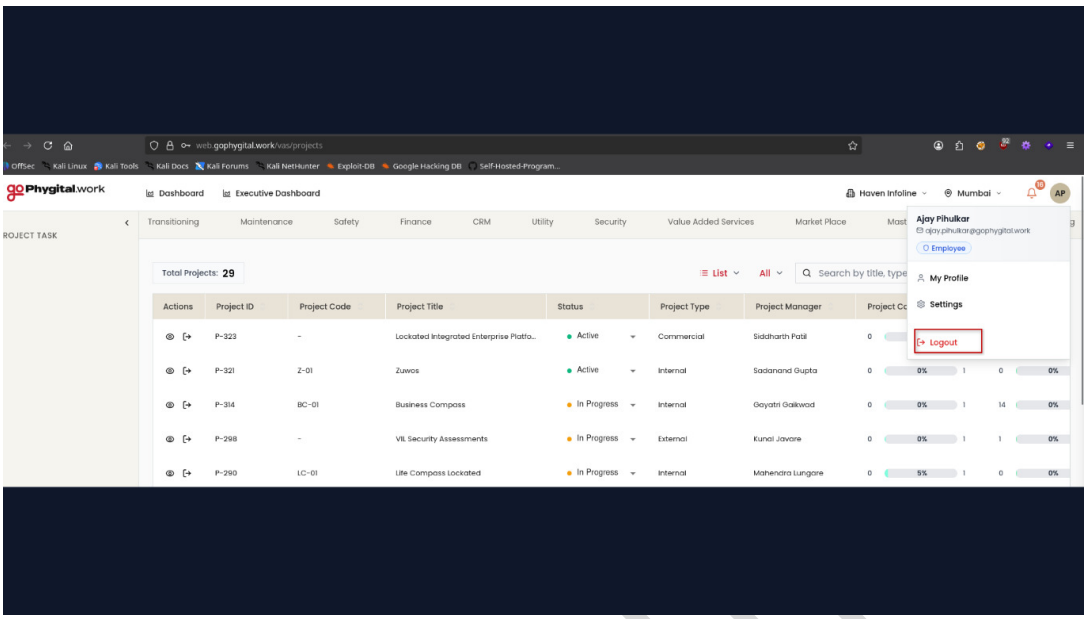
Step 1: Navigate to the application's login page and authenticate using valid user credentials.

Step 2: After successful authentication, access an authenticated page. https://web.gophygital.work/vas/projects

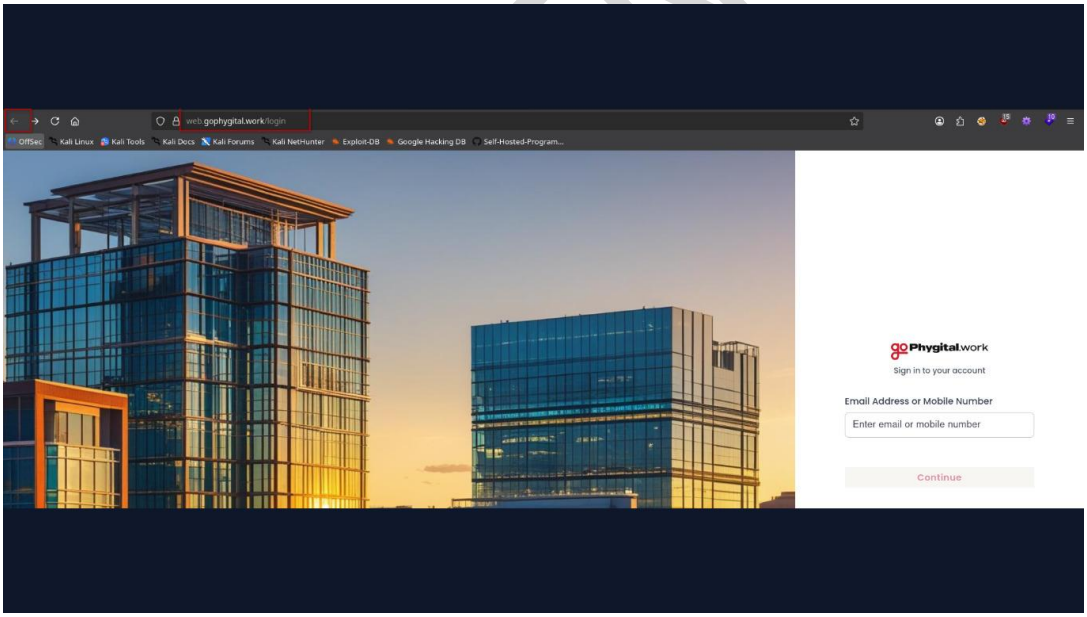
Step 3: Verify that the authenticated page is accessible.



Step 4: Log out of the application using the provided logout functionality.



Step 5: After logout, click the browser's Back button to attempt to revisit the previously authenticated page.



Step 6: Observe whether the application allows access to the authenticated page or redirects the user to the login page.

Step 7: Verify that access to authenticated content requires re-authentication after logout.

Conclusion:

The application's logout functionality was tested to verify proper session termination. After logging out, attempts to revisit a previously authenticated page using the browser's **Back** button consistently redirected the user to the login page, preventing access to protected resources. This behavior indicates that the application properly invalidates authenticated sessions upon

logout and enforces re-authentication before granting access to protected content. **No vulnerability was identified in this test case.**

6.17 Check HTTP Strict Transport Security (HSTS)

Objective:

Verify that the web application implements HTTP Strict Transport Security (HSTS) by including the Strict-Transport-Security response header, ensuring that browsers enforce secure HTTPS connections and prevent protocol downgrade attacks.

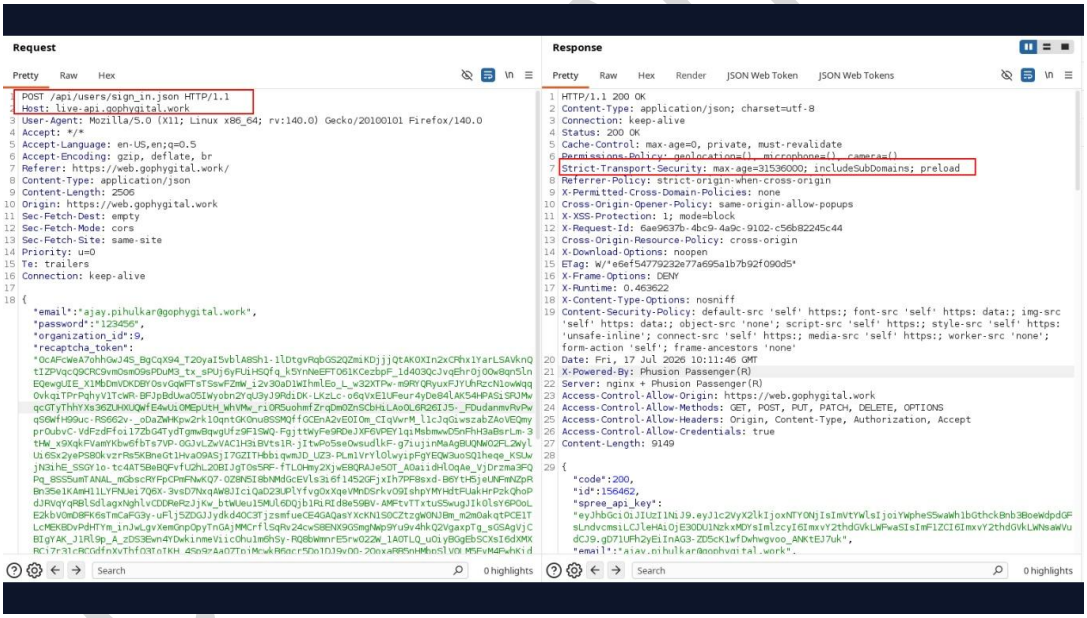
Observations:

- The application's HTTP responses include the Strict-Transport-Security header.
- The HSTS header is returned over HTTPS responses.
- The header instructs browsers to communicate with the application only over HTTPS.
- No issues related to the HSTS configuration were observed during testing.

References to Evidences and Proof of Concept:

Step 1: Navigate to the web application over HTTPS and capture the HTTP response using Burp Suite.

Step 2: In Burp Suite, open the HTTP Response and inspect the response headers.



Step 3: Observe that the response contains the Strict-Transport-Security header with an appropriate value, confirming that HTTP Strict Transport Security (HSTS) is enabled.

Conclusion:

The application implements HTTP Strict Transport Security (HSTS) by including the Strict-Transport-Security response header in HTTPS responses, ensuring that browsers enforce secure communication over HTTPS.

6.18 Check that CAPTCHA Cannot Be Replayed After Validation

Objective:

Verify that a validated CAPTCHA cannot be reused in subsequent requests and that the application requires a new CAPTCHA for each authentication or verification attempt.

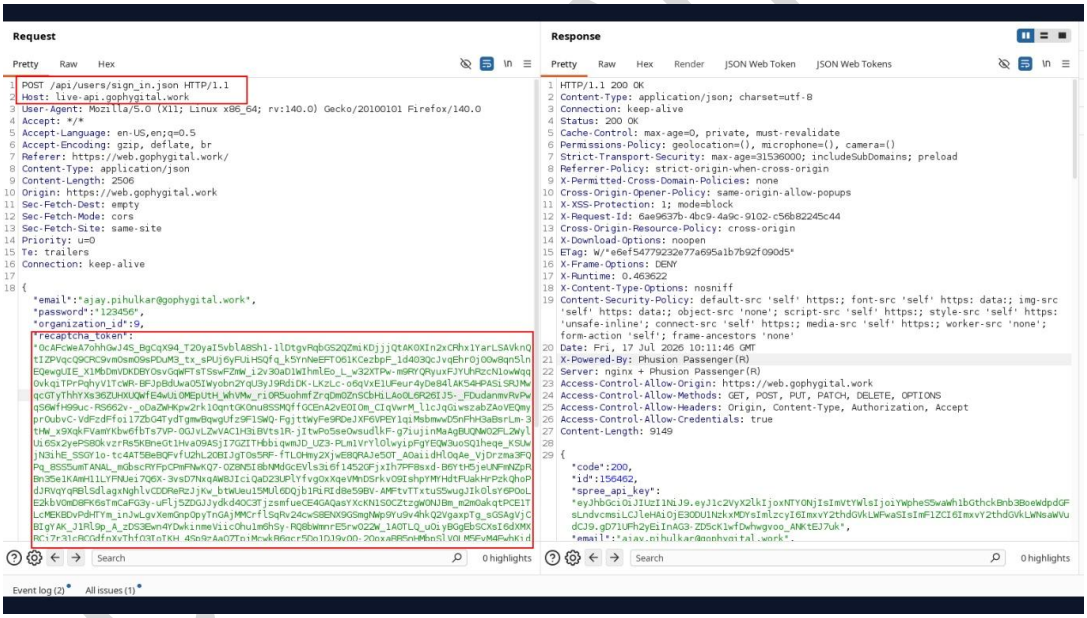
Observations:

- The CAPTCHA request and corresponding validation request were captured using Burp Suite.
- A previously validated CAPTCHA value was replayed in a subsequent request.
- The application rejected the reused CAPTCHA and required a new CAPTCHA to proceed.
- CAPTCHA replay attacks were successfully prevented.

References to Evidences and Proof of Concept:

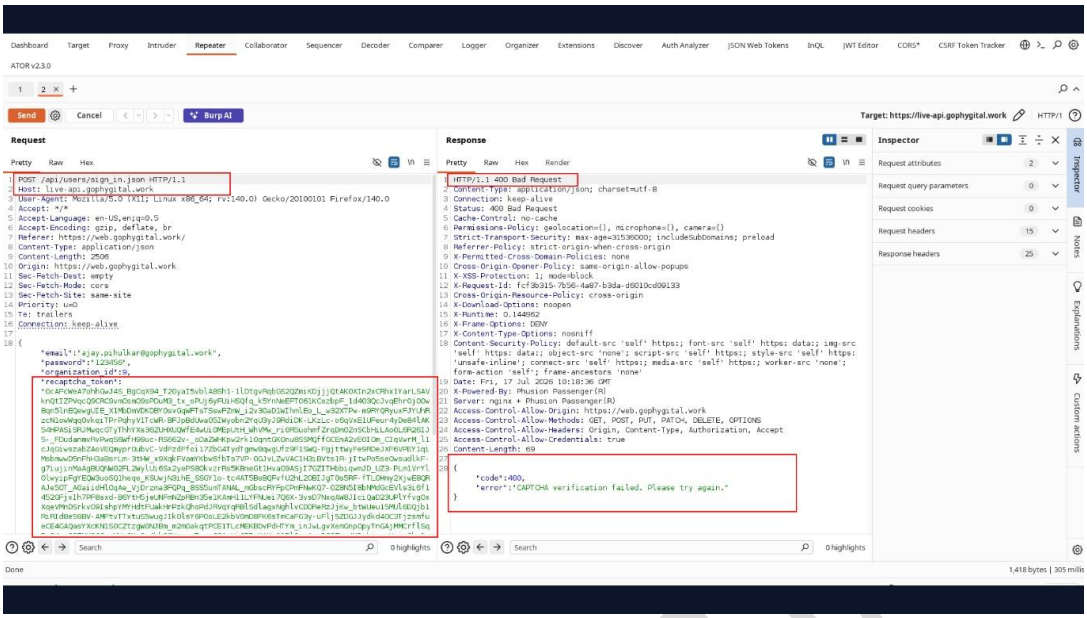
Step 1: Navigate to the application's Login page and enter valid credentials

Step 2: Complete the CAPTCHA, capture the HTTP request using Burp Suite, and submit the request successfully.



Step 3: Send the captured request to Repeater.

Step 4: Replay the same request containing the previously validated CAPTCHA value without generating a new CAPTCHA.



Step 5: Observe that the application rejects the replayed CAPTCHA request

Conclusion:

The application prevents CAPTCHA replay by rejecting previously validated CAPTCHA values and requiring a new CAPTCHA for each authentication or verification attempt.

6.19 Check that the login form is delivered over HTTPS

Objective:

Verify that the login page and credential submission process are served only over HTTPS to protect sensitive information during transmission.

Observations:

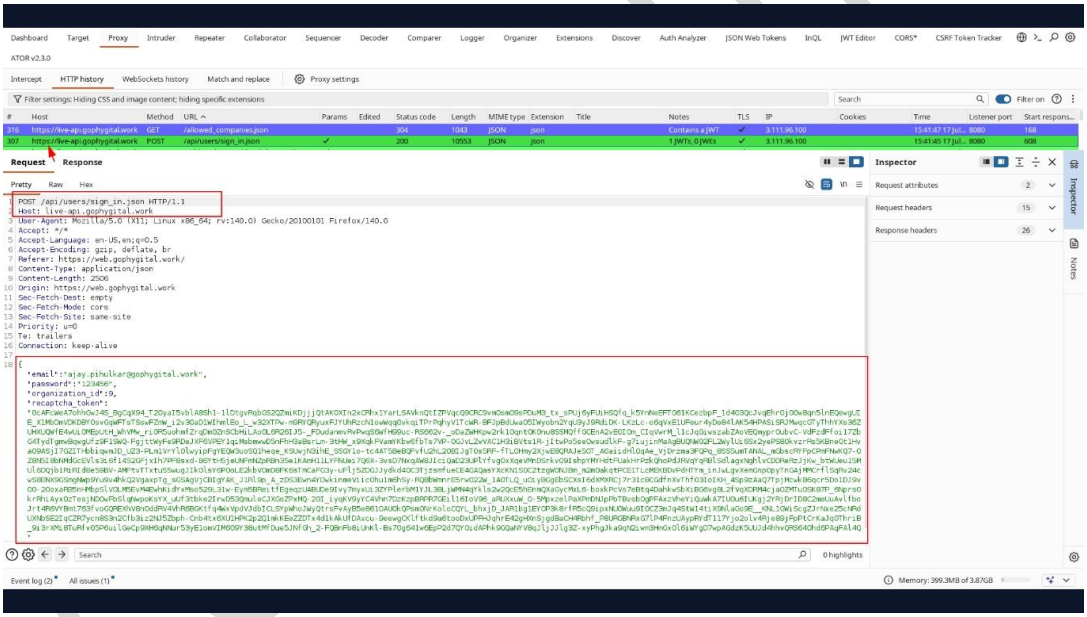
- The login page was served over HTTPS.
- Login credentials were transmitted through an encrypted HTTPS connection.
- Attempts to access the login page over HTTP were automatically redirected to HTTPS.

References to Evidences and Proof of Concept:

Step 1: Accessed the application login page.

Step 2: Entered valid credentials in the login form.

Step 3: Verified in Burp Suite that the login page request was transmitted over the HTTPS protocol.



Step 4: Observed that the request was transmitted over an encrypted HTTPS connection using HTTP/1.1

Conclusion:

The login form and authentication requests are delivered over a secure HTTPS connection. User credentials are transmitted through an encrypted channel, and no insecure transmission of authentication data was observed during testing.

6.20 Check SSL/TLS Versions and Cipher Suites

Objective:

Verify that the web server supports only secure SSL/TLS protocol versions, strong cryptographic algorithms, and robust cipher suites. Ensure that weak or deprecated protocols and ciphers are not enabled.

Observations:

- The server supports TLS 1.2.
- All supported cipher suites are rated A by Nmap.
- Secure cipher suites such as AES-GCM, ChaCha20-Poly1305, and AES-CCM are enabled.
- ECDHE key exchange is used, providing Perfect Forward Secrecy (PFS).
- No weak or deprecated cipher suites (e.g., RC4, DES, 3DES, NULL, EXPORT, or MD5) were detected.

References to Evidences and Proof of Concept:

Step 1: Execute the following Nmap command to enumerate the supported SSL/TLS cipher suites.

Step 2: nmap --script ssl-enum-ciphers -p 443 web.gophygital.work

```
└─$ nmap --script ssl-enum-ciphers -p 443 web.gophygital.work
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-17 15:06 +0530
Nmap scan report for web.gophygital.work (3.108.87.128)
Host is up (0.071s latency).
rDNS record for 3.108.87.128: ec2-3-108-87-128.ap-south-1.compute.amazonaws.com

PORT    STATE SERVICE
443/tcp open  https
| ssl-enum-ciphers:
|   TLSv1.2:
|     ciphers:
|       TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256 (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_AES_256_CCM (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_ARIA_256_GCM_SHA384 (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_AES_128_CCM (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_ARIA_128_GCM_SHA256 (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_CBC_SHA384 (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_CBC_SHA256 (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
|     compressors:
|       NULL
|     cipher preference: server
|_  least strength: A

Nmap done: 1 IP address (1 host up) scanned in 6.20 seconds
```

Step 3: Observe that the scan output lists only Grade A cipher suites for TLS 1.2, with no weak or deprecated ciphers supported.

Conclusion:

The web server is configured to support only strong SSL/TLS cipher suites. No weak or deprecated cryptographic algorithms were identified.

6.21 Integrity and Security of CAPTCHA

Objective:

To verify whether the application's CAPTCHA mechanism is securely implemented and validated on the server side, ensuring that modified, invalid, or tampered CAPTCHA values cannot be used to bypass the authentication process.

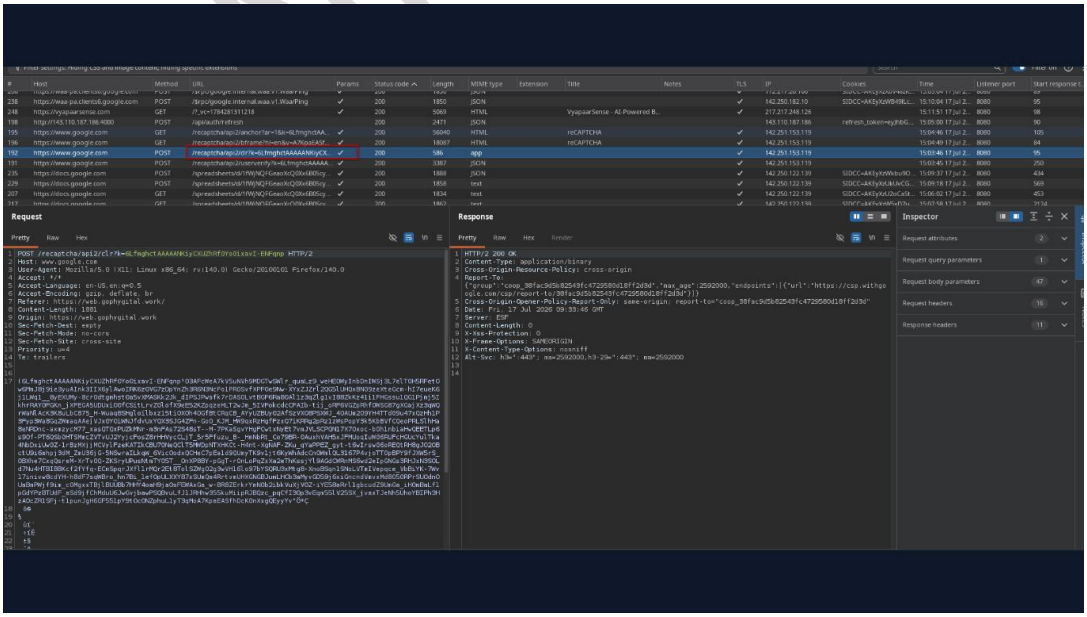
Observations:

- The CAPTCHA validation mechanism on the application's login page was assessed by intercepting the authentication request using Burp Suite.
- The captured request containing the CAPTCHA value was forwarded to Burp Repeater for manual testing.
- The CAPTCHA token/value was deliberately modified by removing one or more characters before resubmitting the request.
- The server responded with **HTTP 400 Bad Request**, indicating that the modified CAPTCHA value was detected as invalid.
- The application performed CAPTCHA validation on the server side and rejected the tampered request.
- No evidence was found that an altered CAPTCHA value could bypass the CAPTCHA verification mechanism.
- Based on the observed behavior, the CAPTCHA implementation effectively validates request integrity before processing authentication.

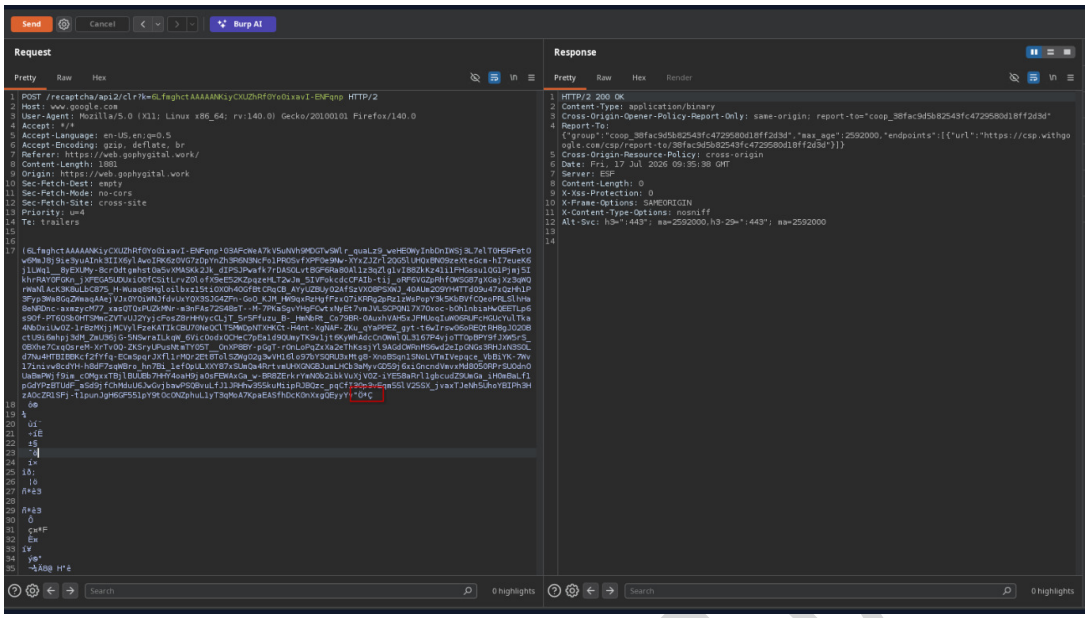
References to Evidences and Proof of Concept:

Step 1: Navigate to the application's login page: https://web.gophygital.work/login

Step 2: Capture the login request containing the CAPTCHA value using Burp Suite Proxy.



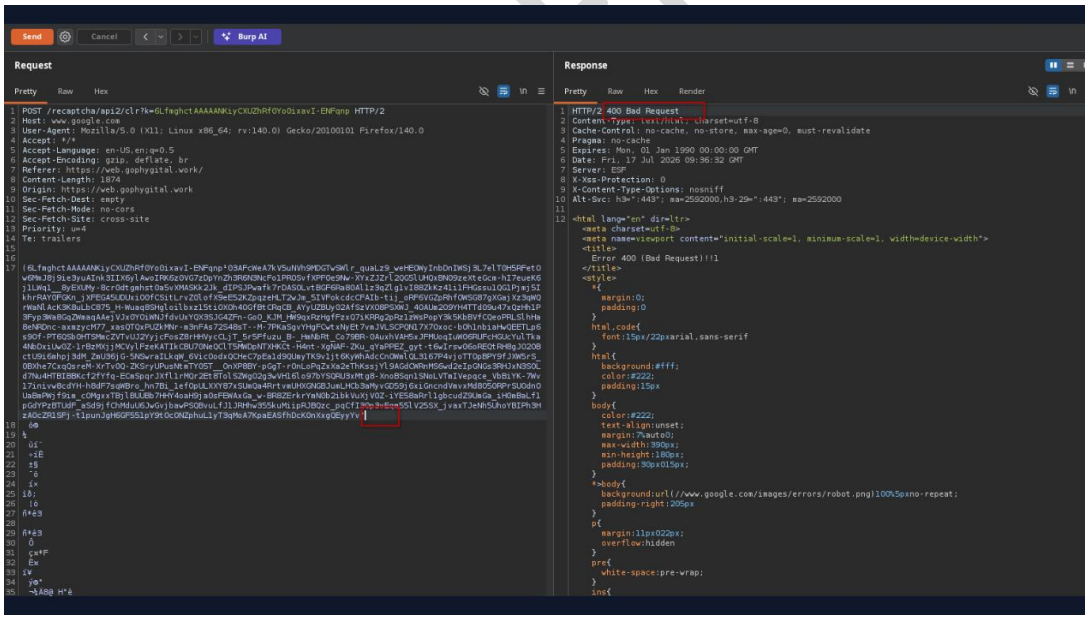
Step 3: Forward the captured request to Burp Repeater.



Step 4: Modify the CAPTCHA value by removing one or more characters from the request.

Step 5: Send the modified request to the server.

Step 6: Observe the server response and verify that the request is rejected with HTTP 400 Bad Request.



Step 7: Confirm that the application validates the CAPTCHA on the server side and does not accept modified or tampered CAPTCHA values.

Conclusion:

The CAPTCHA implementation was tested by modifying the CAPTCHA value within the intercepted authentication request and replaying it using Burp Suite Repeater. The application correctly rejected the tampered CAPTCHA with an **HTTP 400 Bad Request** response, demonstrating that CAPTCHA validation is performed on the server side and that altered CAPTCHA values are not accepted. Based on the testing performed, the CAPTCHA integrity and

validation mechanisms were found to be functioning as intended, and **no vulnerability was identified in this test case**.

6.22 Check for Cache Management on HTTP

Objective:

To verify whether the application prevents sensitive or authenticated content from being cached by browsers or intermediary proxies through the implementation of appropriate HTTP cache-control mechanisms.

Observations:

The authenticated page **https://web.gophygital.work/vas/projects** was accessed and the HTTP response was analyzed.

- The application returned the following cache control headers:
- Cache-Control: no-cache, no-store, must-revalidate
- Pragma: no-cache
- Expires: 0

- These headers instruct browsers and intermediary caches not to store sensitive content and to always revalidate cached responses before reuse.
- Cache control directives were also present within the HTML meta tags, providing additional browser-side cache prevention.
- No insecure cache directives (such as public or max-age) were observed.
- The application appears to implement appropriate cache management controls for sensitive content.
- No security weaknesses related to HTTP cache management were identified during testing.

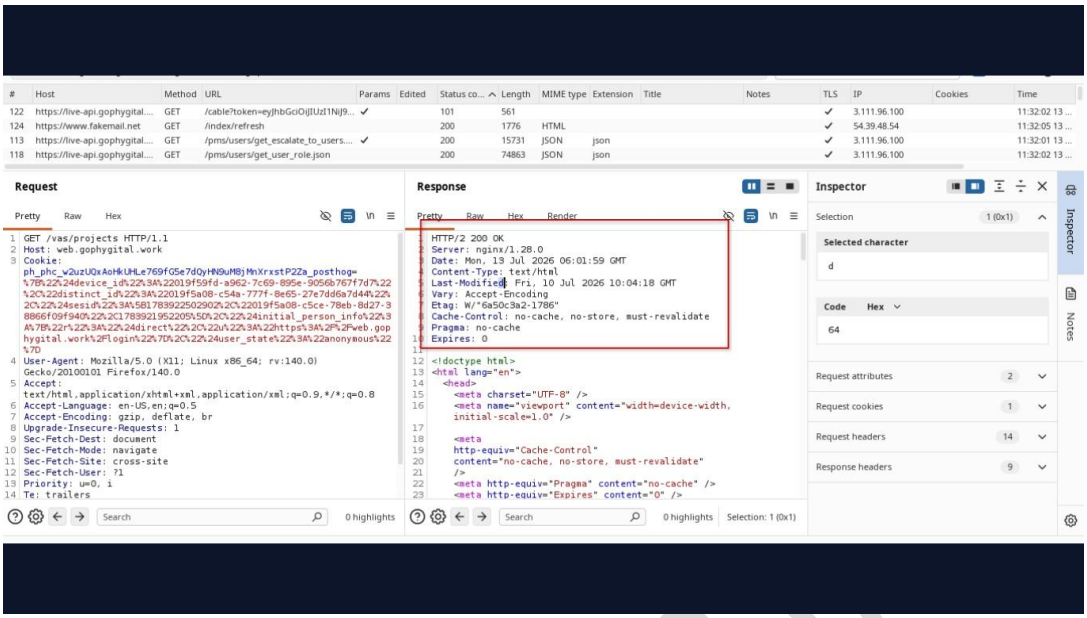
References to Evidences and Proof of Concept:

Step 1: Navigate to the following URL: https://web.gophygital.work/vas/projects

Step 2: Authenticate to the application and access the protected page.

Step 3: Intercept the HTTP response using Burp Suite.

Step 4: Review the response headers for cache-related directives.



Step 5: Verify the presence of secure cache-control headers, including: ◦ Cache-Control: no-cache, no-store, must-revalidate ◦ Pragma: no-cache ◦ Expires: 0

Step 6: Confirm that no insecure cache directives permitting storage of sensitive content are present.

Step 7: Verify that the application instructs browsers and intermediary caches not to retain sensitive responses.

Conclusion:

The application correctly implements HTTP cache management controls by returning appropriate cache-control headers that prevent browsers and intermediary caches from storing sensitive or authenticated content. The response includes Cache-Control: no-cache, no-store, must-revalidate, along with Pragma: no-cache and Expires: 0, which align with secure caching practices. No vulnerability was identified in this test case.

Appendix A: Recommended Security Headers

Below is a highly recommended security header. Consider adding it to the web server configuration to protect users as well the application from malicious attacks.

Header	Description and Recommendation
Content Security Policy (CSP)	The CSP header allows a whitelist defining approved sources of content for the site to be loaded. By restricting the sources from where browsers of your site can load assets (files such as js and css), CSP can act as a countermeasure to XSS attacks. For more information, and implementation hints, refer to: https://scotthelme.co.uk/content-security-policy-an-introduction/
HTTP Strict Transport Security (HSTS)	The target website is being served from not only HTTP but also HTTPS and it lacks of HSTS policy implementation. HTTP Strict Transport Security (HSTS) is a web security policy mechanism whereby a web server declares that complying user agents (such as a web browser) are to interact with it using only secure HTTP (HTTPS) connections. The HSTS Policy is communicated by the server to the user agent via a HTTP response header field named "Strict-Transport-Security". HSTS Policy specifies a period of time during which the user agent shall access the server in only secure fashion.
X-XSS-Protection	The application is missing X-XSS-Protection header which means that this website could be at risk of a Cross-site Scripting (XSS) attacks.
Secure Flag Not Set	This cookie does not have the Secure flag set. When a cookie is set with the Secure flag, it instructs the browser that the cookie can only be accessed over secure SSL/TLS channels. This is an important security protection for session cookies.
HTTP Only Flag Not Set	If the HttpOnly flag (optional) is included in the HTTP response header, the cookie cannot be accessed through client side script (again if the browser supports this flag).

Appendix B: Types of Assessments

Type of Penetration Testing Approach	Description
Red Team Assessments	A Red Team Assessment comes closest to simulating a real-world attack. Here, the scope of work is usually the entire organization, limited only by deciding whether the Red Team comes onsite also or does all their hacking remotely. The exercise is timebound and the results are very often eye-opening.
Bug Bounties	Bug bounties are now the norm, and we propose that you could run a private bug bounty program for a limited duration and a limited scope to understand the value you can get from it. In traditional types of security assessment (see the list below), you pay for consultant effort, and not just for the results. With bug bounties, you pay only for the results.
Black Box Approach	Here, we only know the URL of the website. Enumeration of technologies, mapping of the website, identification of fault injection points, determining input validation vulnerabilities, or logical security vulnerabilities, and the OWASP top 10 attacks are all part of this exercise.
Grey Box Approach	Often enough, a web application involves authentication and authorization of components. To be able to test these, we request for a dummy user account with the least level of privileges within the application. Using this account, we can log in and test for various flaws in the authentication scheme, as well as an attempt to escalate our privileges and bypass authorization restrictions.
War Dialing	Is a Penetration Test technique in which a list of telephone numbers are called to search for computers, systems, and fax machines. The purpose is to gain as much business-critical data as possible with the help of this technique.
Wireless Hacking	Wireless hacking is done to gather all loopholes possible in an organization's wireless infrastructure. This is done with an intention to gain unauthorized access and to try and exploit as many resources as available.
Social Engineering	Controls can be put on systems and devices but the same does not hold true for the objects using these systems (employees/temporaries). Social Engineering is the method by which all hackers try to get confidential and business critical information by using various techniques. This test focuses on exploiting and finding out all the possible loopholes pertaining to this domain so that your organization is geared up to face social engineering attacks in life
War Driving	War driving can be carried out to test the range and strength of your organization's wireless network's signal. This will help to gauge the extent of the threat exposure area for your organization.
Business Risk Based Approach	Traditional Penetration Testing approach only focuses on the technical vulnerabilities. But Business Risk based approach not only focuses on the

	technical vulnerabilities but also on the risks presumed to the organization's business. First, test cases pertaining to the business threat model are developed and Penetration test is carried out focusing majorly on these cases. This method has many advantages over the traditional Penetration Test methodology. And one of the biggest advantages it has is that of being business focused.
Source Code Review	Source code review focuses on detecting the vulnerabilities early in the Software Development Life Cycle (SDLC) such as Dataflow attacks, Cross Site Scripting (XSS), Injection (SQL, File, XPATH, reflection, etc.). File Inclusion/execution and Information Leakage and this methodology will help the organization to close the loopholes during the development and testing phase.
Internal Penetration Testing	Many organizations secure their organization from outside threats but leave their internal network security comparatively weak. And it has been proved repeatedly that the organization's security was compromised from within the network. Internal Penetration Test focuses on identifying these loopholes and recommending solutions to make the internal network secure to thwart internal threats and attacks.
Vulnerability Assessment	Vulnerability Assessment is the process of identifying, quantifying, and prioritizing the vulnerabilities of the components of IT infrastructure.
Stress Testing	As applications move to the Web, into a server-based environment, it becomes increasingly important to be able to gauge the performance and load capability of an application. Stress testing involves testing the web-application's ability to handle the load/stress resulting from an increase in hits, which may be a result of the sudden change affecting the organization's business activity directly.
Denial of Service Testing	A very serious and significant threat that web applications face is from DoS attacks. In this attack, the attacker sends so many packets to a website that it cannot service the legitimate users that are trying to access it. This leads to denial of service to the legitimate users, thus affecting the business directly. So, assessment needs to be carried out to check for the organization's preparedness to face such an attack.

*****************************End of Report*************************